GExperts Help

USER MANUAL

©2026 GExperts Team

1 GExperts 4

1.1 Introduction 5
1.2 Readme.txt 6
1.3 License Agreement 10
1.4 C++ and C# Compatibility 10
1.5 Debug Window and DbugIntf.pas 12
1.6 Change Log (2006-2015) 14
1.7 Credits 17
1.8 Configuration 18
1.8.1 GExperts Configuration 18
1.8.2 Editor Experts 20
1.8.3 General Configuration 22
1.8.4 IDE Enhancements 23
1.8.5 Code Editor Enhancements 25
1.8.6 Editor Toolbar 25

2 Experts 28

2.1 ASCII Chart 29
2.2 Backup Project 29
2.3 Class Browser 31
2.4 Clean Directories 32
2.5 Clipboard History 33
2.6 Code Librarian 34
2.7 Code Proofreader 35
2.8 Component Grid 36
2.9 Components to Code 37
2.10 Copy Component Names 38
2.11 Edit Unit Search Path 39
2.12 Editor Bookmarks 41
2.13 Expert Manager 43
2.14 Favorite Files 44
2.15 Find Component Reference 45
2.16 Formatter 46
2.17 Grep Regular Expressions 55
2.18 Grep Results 60

©2026 GExperts Team

2.19 Grep Search 62
2.20 Hide/Show Non-Visual Components 64
2.21 IDE Menu Shortcuts 64
2.22 Macro Library 65
2.23 Macro Templates 66
2.24 Message Dialog 68
2.25 Open File 69
2.26 PE Information 70
2.27 Perfect Layout 72
2.28 Procedure List 72
2.29 Project Dependencies 73
2.30 Project Option Sets 74
2.31 Property Value Constants 75
2.32 Rename Components 76
2.33 Replace Components 77
2.34 Set Component Properties 81
2.35 Set FocusControl 82
2.36 Set Tab Order 83
2.37 Source Export 83
2.38 Substitution Macros 85
2.39 ToDo Grid 86
2.40 To Do List 91
2.41 Uses Clause Manager 92

Index 99

©2026 GExperts Team

GExperts

1 GExperts

1.1 Introduction

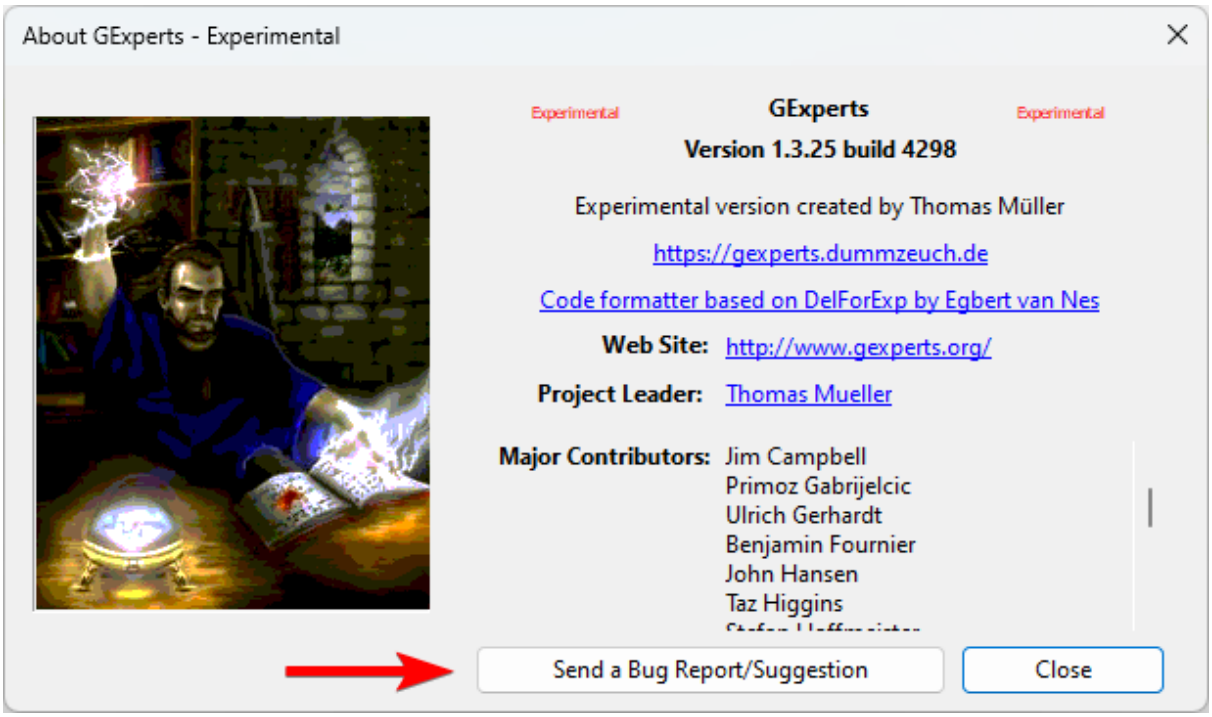
GExperts Version 1.3.25 (Updated 2024-08-24)

GExperts is a free set of tools built to increase the productivity of Delphi and C++Builder programmers by adding several features to the IDE. GExperts is developed as Open Source software and encourages user contributions to the project.

GExperts would not exist without the help and advice of many people. For more credits see the change log and commit history, or open the GExperts about box. Please take a moment to read the GExperts License Agreement. Also, remember that the **Readme.txt** file included with GExperts will often contain information not included in this help file, including the latest changes, known bugs, and plans for the future.

The latest version of GExperts and the FAQ are always available at: https://gexperts.dummzeuch.de. You can send bug reports and suggestions using the Feedback Wizard in the GExperts about box.

For other GExperts related queries use the GExperts sub-forum in Delphi Praxis at https://en.delphipraxis.net/forum/31-gexperts/



©2026 GExperts Team

1.2 Readme.txt

GExperts 1.3.25
Open Source Programming Tools for Delphi and C++Builder

Source code, the FAQ, and the latest news are available at:
  https://gexperts.dummzeuch.de/
  https://sourceforge.net/projects/gexperts/
Additional (older) information at
  http://www.gexperts.org/

Please send all bug reports and suggestions using the feedback wizard
available in the GExperts about box.

For other GExperts related queries use the GExperts subforum in Delphi Praxis at
https://en.delphipraxis.net/forum/31-gexperts/

INTRODUCTION

GExperts is a set of tools built to increase the productivity of
Delphi and C++Builder programmers by adding several features to the
IDE. GExperts is developed as Open Source software and is contributed
as freeware to the development community. Users are encouraged to
download the source code and submit bug fixes and patches as well as
new features for inclusion in the GExperts distribution.

THE EXPERTS

GExperts contains numerous experts. See the online help
(GExperts.chm) for information on the usage of the experts.

The source code formatter expert is based on DelForExp by Egbert van Nes
who contributed it to GExperts.
http://web.archive.org/web/20121115141650/http://www.aew.wur.nl/UK/Delforexp

KNOWN LIMITATIONS/BUGS

- The following features are not supported under Delphi 8 - RAD Studio 10.2:
  Inner classes, class helpers, anonymous methods, and generics in the
  Class Browser, and some of the old editor tab enhancements, such as
  multiline editor tabs. Note that the IDE converts some high ANSI and MBCS
  characters to UTF-8 UNICODE when loaded in the code editor, so you may
  experience problems with those files (especially in Delphi 2007 and earlier).
- The compiler replacement option of the Code Proofreader is only partially
  working under Delphi 7 - RAD Studio 10. It does not correct when an identifier is
  terminated by pressing a symbol key such as a period or open parenthesis,
  but works fine when the symbol is terminated by a space (IDE limitation).
- Due to either native Open Tools API limitations or bugs, the following
  items can not be supported:
  Delphi 8 Only:

©2026 GExperts Team

```
  - Rename Components (IDE bug setting the Name property)
  - Jumping to a form search match from Grep Results when the source is open
    (IDE bug)
Delphi 2005 Only:
  - The Project Option Sets environment options (IDE bug)
  - Set Tab Order: Selecting components in the desired tab order (IDE bug)
Delphi 8 - 2007:
  - Any interaction at all with the WinForms/.NET form designer/components
    (no WinForms support for IOTAFormEditor/IOTAComponent)
  - Replace Components for VCL.NET (No direct access to components)
  - Components to Code for VCL.NET (No direct access to components)
Delphi 8 - RAD Studio 10:
  - It is no longer possible for addins to override some built-in IDE
    shortcuts.  You may need to configure your GExperts shortcuts (Prev/Next
    Identifier, Procedure List, etc.) to not conflict with your selected
    keymapping.  Also, the Macro Library can not always intercept the
    Shift+Ctrl+R keystroke to automatically grab keyboard macros, so you may
    need to create macros using the recording functions in the toolbar.
Delphi 2009 - 2010 Update 1:
  - Record and playback in the Marco Library does not work.  This is an IDE
    bug handling editor macro streams (QC 78289).
Delphi XE3 - 10.3 Update 3:
  - Replace Components fails for components that contain sub-components, e.g.
    TTable with static fields. This is an IDE bug (RSP25645).
       Until this IDE bug has been fixed, this functionality has been disabled.


INSTALLATION
----------------------
GExperts is distributed as a self-installing executable that should
automatically install itself into the IDE.  If you are installing under
Vista as a non-administrator, you may need to manually register the GExperts
DLL with the IDE after installation.  This is most easily done using the
Expert Manager tool in the GExperts start menu group.  If you prefer, you
can also manually register the DLL with the IDE using the Windows registry
editor (RegEdit.exe).  Create a key similar to the following (the
version number appearing before "\Experts\" is IDE dependent):
HKEY_CURRENT_USER\Software\Embarcadero\BDS\23.0\Experts\ (RAD Studio 12)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\22.0\Experts\ (RAD Studio 11)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\21.0\Experts\ (RAD Studio 10.4)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\20.0\Experts\ (RAD Studio 10.3)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\19.0\Experts\ (RAD Studio 10.2)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\18.0\Experts\ (RAD Studio 10.1)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\17.0\Experts\ (RAD Studio 10)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\16.0\Experts\ (RAD Studio XE8)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\12.0\Experts\ (RAD Studio XE5)
HKEY_CURRENT_USER\Software\Embarcadero\BDS\9.0\Experts\  (RAD Studio XE2)
HKEY_CURRENT_USER\Software\CodeGear\BDS\6.0\Experts\     (RAD Studio 2009)
HKEY_CURRENT_USER\Software\Borland\BDS\5.0\Experts\      (RAD Studio 2007)
HKEY_CURRENT_USER\Software\Borland\Delphi\7.0\Experts\   (Delphi 7)
HKEY_CURRENT_USER\Software\Borland\Delphi\6.0\Experts\   (Delphi 6)

Then add a new string value that points to your GExperts DLL (X is the IDE
version such as 6, 7, 8, 2005, 2006, 2007, 2009, 2010, XE1, XE8, 10, etc.):
GExperts=C:\Program Files\GExperts\GExpertsRSX.dll
GExperts=C:\Program Files\GExperts\GExpertsDelphiX.dll
GExperts=C:\Program Files\GExperts\GExpertsBDSX.dll or
```


©2026 GExperts Team

```
GExperts=C:\Program Files\GExperts\GExpertsDX.dll or

Before installing GExperts, it is strongly recommended that you install
the latest updates and patches for your IDE available from:
  http://www.embarcadero.com/


UNINSTALLATION
----------------------
Uninstallation of GExperts can be done from the Control Panel's
Add/Remove Programs or Programs [and Features] tool.


CHANGE LOG
----------------------
Sorry, I have not maintained the change log since I took over from
Eric Berry. There were many. In particular of course support for the
latest released versions of RAD Studio / Delphi.
-- Thomas Mueller
You can see the changes from 2006 to 2015 in the file
changelog(2006-2015).txt


FUTURE ENHANCEMENTS?
----------------------
- [Unassigned] Persistent editor bookmarks
- [Unassigned] Integrate file auto-save code
- [Unassigned] Add compile date to version information and sync
  FileVersion with ProductVersion
- [Unassigned] Add GX_MessageBox warnings to the Pascal only experts
  when used under C++Builder
- [Unassigned] Replace SortGrid with a TListView and make the various
  TListViews sortable
- [Unassigned] Allow adding some common editor and object inspector
  actions to the editor toolbar?
- [Unassigned] Add support for hiding the toolbar on all dockable experts
- [Unassigned] Add word wrapped printing to the To Do List, etc.


MISCELLANEOUS NOTES
----------------------
- If a DFM is opened as text from the Grep Results (for example), you
  may experience an IDE bug where it forgets the value of the Text DFM
  flag on the form and converts it to a binary format.
- The Code Proofreader can only perform corrections when the editor caret
  is directly after the word to correct following a change.  As a result,
  pressing enter will not trigger corrections.
- Changing the configured GExperts top-level menu item location requires
  an IDE restart before the new setting is active.
- The editor toolbar does not support separators when aligned left/right.
- If a DFM is opened as text, a subsequent Grep Search can not scan the
  DFM's associated source files.  Delphi does not provide access to the
  source file buffer.  Under C++Builder you might also see duplicated
  matches for the DFM file when it is being viewed as text.
- Perfect Layout was designed for Delphi 3, and as such will probably
  never support docking or undocking of windows in later version IDEs.
  Please use the desktop saving feature of your IDE instead.
```


©2026 GExperts Team

- Nothing in GExperts is aware of or takes into account conditional
  defines, so code that is defined out will still be parsed by GExperts.
- Source Export only supports background colors when exporting to HTML.

GEXPERTS LICENSE

GExperts is copyright 1996-2015 by GExperts, Inc, Erik Berry, and several other authors who have submitted their code for inclusion. This license agreement only covers code written by GExperts, Inc. and Erik Berry. You should contact the other authors concerning their respective copyrights and conditions.

The rules governing the use of GExperts and the GExperts source code are derived from the official Open Source Definition, available at http://opensource.org/osd

The conditions and limitations are as follows:

Usage of GExperts binary distributions is permitted for all developers. You may not use the GExperts source code to develop proprietary or commercial products including plugins or libraries for those products. You may use the GExperts source code in an Open Source project, under the terms listed below.

You may not use the GExperts source code to create and distribute custom versions of GExperts under the "GExperts" name. If you do modify and distribute custom versions of GExperts, the binary distribution must be named differently and clearly marked so users can tell they are not using the official GExperts distribution. A visible and unmodified version of this license must appear in any modified distribution of GExperts.

Custom distributions of GExperts must include all of the custom changes as a patch file that can be applied to the original source code. This restriction is in place to protect the integrity of the original author's source code. No support for modified versions of GExperts will be provided by the original authors or on the GExperts mailing lists.

All works derived from GExperts must be distributed under a license compatible with this license and the official Open Source Definition, which can be obtained from http://www.opensource.org/.

Please note that GExperts, Inc. and the other contributing authors hereby state that this package is provided "as is" and without any express or implied warranties, including, but not without limitation, the implied warranties of merchantability and fitness for a particular purpose. In other words, we accept no liability for any damage that may result from using GExperts or programs that use the GExperts source code.

1.3 License Agreement

GExperts is copyright 1996-2026 by GExperts, Inc., Erik Berry, and several other authors who have submitted their code for inclusion. This license agreement only covers code written by GExperts, Inc., Erik Berry, and code specifically contributed for use in GExperts. You should contact the other third-party authors concerning their respective copyrights and conditions.

The rules governing the use of GExperts and the GExperts source code are derived from the official Open Source Definition, available at http://www.opensource.org/. The conditions and limitations are as follows:

- Usage of GExperts binary distributions is permitted for all developers. You may not use the GExperts source code to develop proprietary or commercial products including plugins or libraries for those products. You may use the GExperts source code in an Open Source project, under the terms listed below.

- You may not use the GExperts source code to create and distribute custom versions of GExperts under the "GExperts" name. If you do modify and distribute custom versions of GExperts, the binary distribution must be named differently and clearly marked so users can tell they are not using the official GExperts distribution. A visible and unmodified version of this license must appear in any modified distribution of GExperts.

- Custom distributions of GExperts must include all of the custom changes as a patch file that can be applied to the original source code. This restriction is in place to protect the integrity of the original author's source code. No support for modified versions of GExperts will be provided by the original authors or on the GExperts mailing lists.

- All works derived from GExperts must be distributed under a license compatible with this license and the official Open Source Definition, which can be obtained from http://www.opensource.org/.

- Please note that GExperts, Inc. and the other contributing authors hereby state that this package is provided "as is" and without any express or implied warranties, including, but not without limitation, the implied warranties of merchantability and fitness for a particular purpose. In other words, we accept no liability for any damage that may result from using GExperts or programs that use the GExperts source code.

1.4 C++ and C# Compatibility

The following tools should work with C++ code:

- IDE Menu Shortcuts
- PE Information
- Grep Search / Grep Results
- Procedure List (no support for old C-style parameter lists)
- Message Dialog
- Expert Manager
- ASCII Chart
- Clipboard History
- Perfect Layout (deprecated)
- Replace Components

©2026 GExperts Team

- Set Tab Order
- Clean Directories
- Favorite Files (although one icon shows Delphi)
- Code Librarian (although two icons show Delphi)
- Source Export
- Code Proofreader (except compiler-assisted replacement)
- Components to Code
- Backup Project (it only includes C++ source files in the project file)
- To Do List
- Rename Components
- Copy Component Names
- Editor Experts
  - Insert Date/Time
  - Comment Code
  - Uncomment Code
  - Locate/Move to Matching Delimiter
  - Macro Templates (except for many %MACROS% that require source parsing)
  - Sort Selected Lines
  - Previous/Next Identifier Reference
  - Reverse Statement
  - Change Case
  - Select Identifier
  - Align Lines
  - Change Case
- IDE Enhancements
  - Multi-line Tabs
  - Show Tabs as Buttons
  - Component Palette Tab Names in Popup Menu
  - Add a Windows Menu
- Code editor enhancements and the editor toolbar
- Macro Library
- Open File
- Find Component Reference
- Set Component Properties
- Hide/Show Non-Visual Components
- Set FocusControl

The following tools will only work with Delphi source code:

- Class Browser
- Project Dependencies (only Pascal dependencies in the project file are found)
- Several of the editor toolbar buttons, such as Used Units, Unit Positions, etc.
- Uses Clause Manager

C# Compatibility

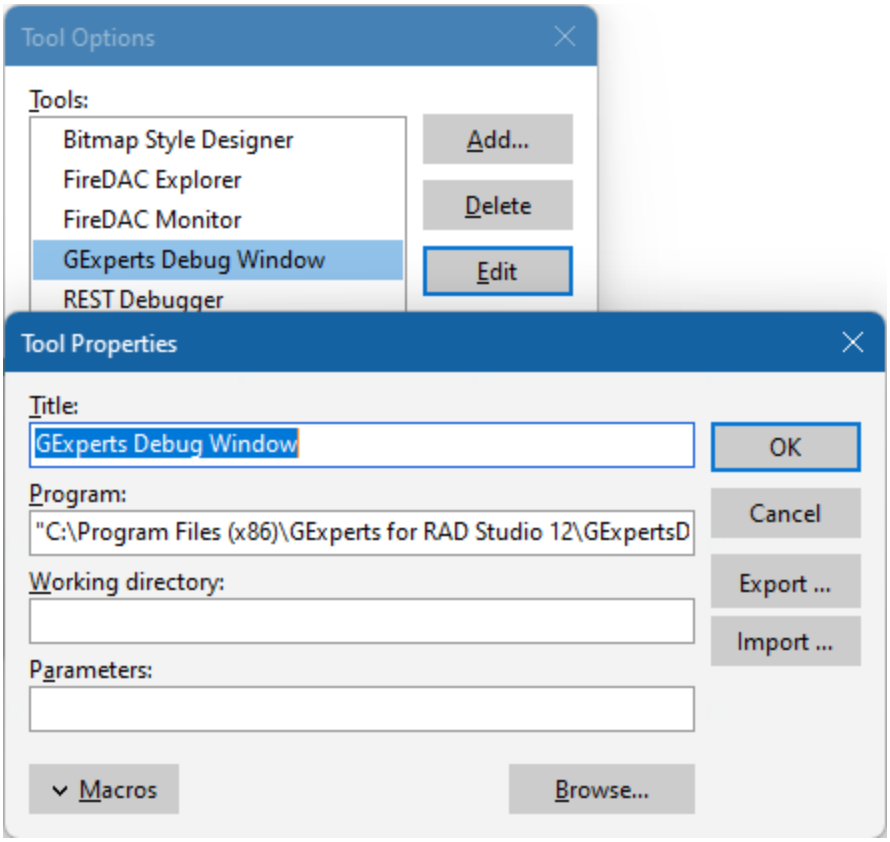
None of the tools that require source code parsing will work with C# code, because GExperts does not contain a C# code parser. The state of the other tools with respect to C# is that they may work, but there has been only limited testing to verify correct results, since RAD Studio no longer supports C# projects.

©2026 GExperts Team

1.5 Debug Window and DbugIntf.pas

The debug window is useful in those situations where it is difficult to use the regular IDE debugger (debugging or data collection with end users, debugging timing sensitive routines, paint routines, etc.). To use the debug window, include the unit `DbugIntf.pas` that comes with GExperts in your unit's uses clause and then in the spot you need a debug statement inserted, type `SendDebug('Debug note');`. You may want to wrap your debug statements in a conditional define such as `{$IFDEF DEBUG}` and create an IDE Code Template to easily insert new debug statements into your code.

Usage:
The GExperts Debug Window is a separate program (**GExpertsDebugWindow.exe**) that sits in the task tray of the Windows shell. You can add it to the Tools menu in Delphi, launch from the start menu, or when you send a debug message, GExperts will try to start the it automatically.



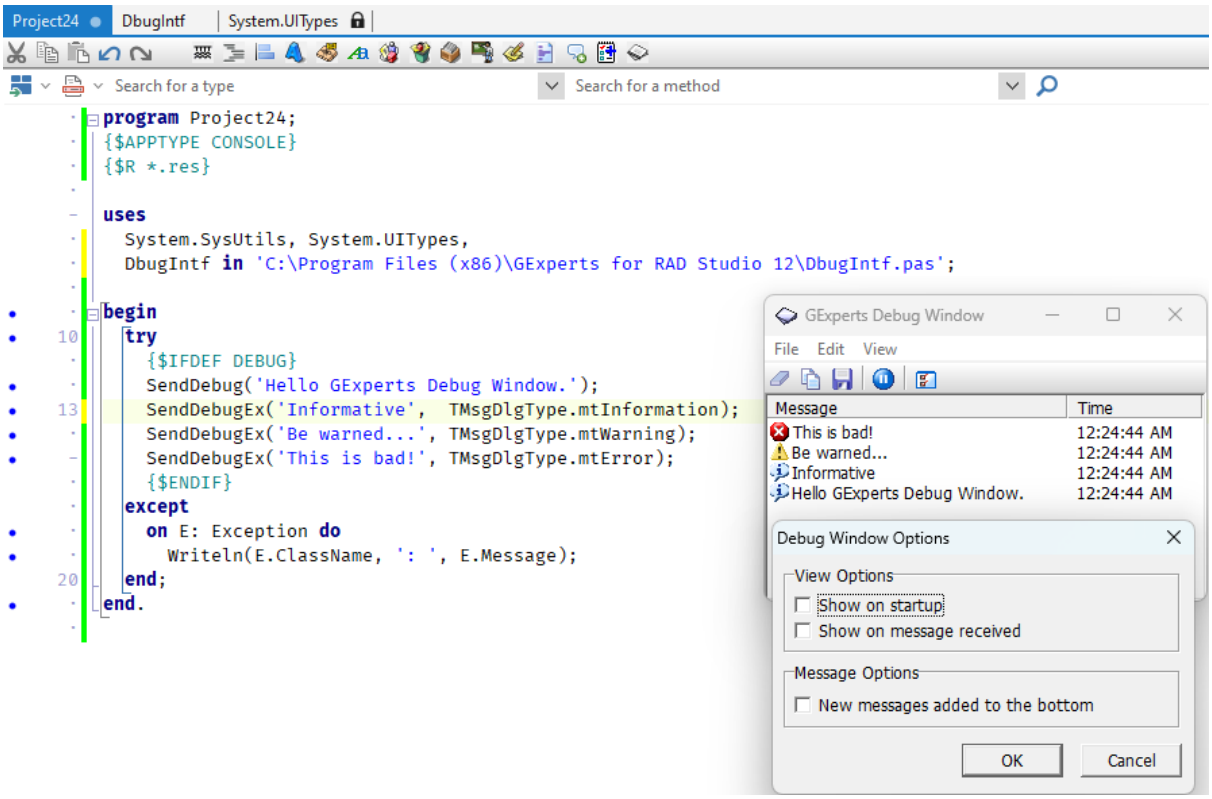
Whenever the window receives a new message, if it is minimized to the task tray then the icon will change from a plain paper icon to a paper icon with a pen writing on it. 

Three types of messages can be sent to the debug window: informational, warning, and error. These message types are sent using the mtInformation, mtWarning and mtError constants that are defined in

©2026 GExperts Team

the VCL's Dialogs unit and can be sent using the SendDebugEx command: `SendDebugEx('Debug message', mtInformation);`. DbugIntf.pas also defines several other useful methods to send data types directly and signal entry into and exiting from methods, etc.

The Debug Window allows you to clear the message display, copy the messages to the clipboard, save them to a file, pause collection of messages, and configure some basic options.



Here is the interface from `DbugIntf.pas` with all the supported functions

```
procedure SendBoolean(const Identifier: string; const Value: Boolean);
procedure SendDateTime(const Identifier: string; const Value: TDateTime);
procedure SendDebugEx(const Msg: string; MType: TMsgDlgType);
procedure SendDebug(const Msg: string);
procedure SendDebugError(const Msg: string);
procedure SendDebugWarning(const Msg: string);
procedure SendDebugClear;
procedure SendInteger(const Identifier: string; const Value: Integer);
procedure SendMethodEnter(const MethodName: string);
procedure SendMethodExit(const MethodName: string);
procedure SendIndent;
procedure SendUnIndent;
procedure SendSeparator;
procedure SendDebugFmt(const Msg: string; const Args: array of const);
```


©2026 GExperts Team

```
procedure SendDebugFmtEx(const Msg: string; const Args: array of const;
MType: TMsgDlgType);
function StartDebugWin: hWnd;
procedure SendDebugPause;
procedure SendDebugResume;
```

1.6 Change Log (2006-2015)

VERSION 1.38 (September 2015)
- General: Added support for RAD Studio 10 Seattle.
- General: Improved Unicode support for several GExperts tools (Thomas Mueller).
- General: Various minor bug fixes.

VERSION 1.37 (September 2012 - May 2015)
- General: Added support for RAD Studio XE3, XE4, XE5, XE6, XE7, and XE8 (with help from Jeroen Pluimers). Various other minor bug fixes.
- Grep: Add Ignore Delphi comments feature (via Paul Gardner). Significantly speed up searches by updating the search status details less often. Add ability to auto-hide the Grep Results window when jumping to a match.
- Components to Code: Preview the generated code as you change the options.
- Message Dialog: Support leading spaces better for inserted code. Force focus to editor after code insertion.
- Set Component Properties: Close opened but not modified files to save resources. Force show modified files, to show changes without saving. Ignore some problematic TWebBrower properties causing read errors.

VERSION 1.36 (December, 2011)
- General: Add official support for RAD Studio XE2, Allow filtering expert list in configuration dialog (Ulrich Gerhardt), Update SynEdit (syntax-highlighting text editor) version
- Replace Components: Ignore Touch/TTouchManager properties, since they crash 2010/XE when copied to a new component, add more error checking.
- Backup Project: Update Abbrevia component version used to generate .zip files
- Set Component Properties: Allow setting properties on forms and data modules
- Grep: Ignore symbolic link directories during recursive searches
- Uses Clause Manager: Add shortcut to open selected file. Close dialog after opening a file.
- Message Dialog: Changed default shortcut to Shift+Ctrl+D to not conflict with source formatter in recent IDE releases.

VERSION 1.35 (June, 2011)
- General: Fix problems with the bug reporting tool and about box disappearing.
- Grep: Search and replace supports regular expression subgroubs (Egon Elbre).
- Favorite Files: Support relative filenames in saved files lists, support multiple groups of favorite files. These changes may be useful for project-specific favorite files lists. (Ulrich Gerhardt, Erik)
- Clipboard History: Show combined text for multi-selected history entries.

©2026 GExperts Team

VERSION 1.34 (September, 2010)
- General: Added support for RAD Studio XE. Minor bugfixes and updates to the help files. (Erik)
- Set FocusControl: Add tool to assign focus control of labels by selecting the related label and wincontrol or a group of wincontrols, and the tool tries to guess the label to associate with each control. (Daniel Maltarollo and Erik)
- Grep: Allow excluding any number of directories from a search. Always center the match line in the Grep Result context pane.
- Components to Code: Optionally generate code to free all created components (Peter Dzomlija).
- Hide/Show Non-Visual Components: Added support for TFrame designers (Erik).

VERSION 1.33 (May 31, 2009)
- General: Added support for RAD Studio 2009/2010. Note that some of the tools that require our source code parser (Procedure List, Class Browser, To Do List, Delimiter Editor Experts, etc.) do not have full unicode support. Other tools such as the Grep Search/Replace, Source Export, Code Librarian, etc. should support UNICODE files just fine. Also included are some shutdown optimizations, removal of all known memory leaks, and other minor tweaks. All GExperts forms use the OS default font face/size. You can also override the UI font in the General tab of the settings.
- Grep Search: There is a new regular expression engine that supports a large subset of the Perl regular expression syntax. Grep can now search and replace files in ANSI, UTF-8, and UTF-16 formats in Delphi 2009/2010. The stay on top feature was removed when running inside the IDE, since it could hide modal dialogs. Allow searching within previously matched files (John Hansen).
- Hide/Show Non-Visual Components: This new tool allows temporarily hiding non-visual components on a form, so you can visualize the form as it will appear at runtime.
- Editor Experts: The main menu contains a submenu item for each editor expert now in Delphi 8 or later.
- Favorite Files: Supports preview of jpg, png, and gif files if the VCL supports it in your IDE version, supports select all for files and move up/down for folders.
- Code Proofreader: Should be slightly more accurate with fewer false positive corrections made to your code.
- To Do List: Display the category and owner for to do items, indicate items marked DONE, support more priorities (Pestasoft, Erik).
- Code Librarian: Support C# and XML syntax highlighting.
- Help File: Updated to reflect the current release. (John Hansen, Erik)

VERSION 1.32 (September 11, 2007)
- General: Added support for RAD Studio 2007
  GExperts data files are saved under the user's Application Data\GExperts directory by default for new installs, to better support Windows Vista user restrictions. To change this location manually, go to GExperts, Configuration, File Locations.
- Grep: Allow searching of the active form's text while the editor is active
- To Do List: Allow scanning .inc files in searched directories for to do items

©2026 GExperts Team

VERSION 1.31 (April 7, 2007)
- General: Added support for Delphi 2007
  Fixed a possible shutdown exception when unloading GExperts
  Help switched to HTML Help format (.chm) for better Vista support
  Improve the font rendering on several forms
- IDE Enhancements: Added a new option to allow resizing and enhancing some
  IDE dialogs (for BDS 2006 or later). See the help file for full details.

VERSION 1.30 (October 26, 2006)
- General: Added support for Delphi 2006. Better syntax highlighting
  support for C#, HTML, and XML (Erik). Better ClearType support in
  several forms.
- Open File: New tool to quickly search for and open files in the project,
  search path, recent files, favorite files, etc. The file groups are
  configurable and the tool can override the IDE's internal open
  project/form/unit dialogs. (Paul Gardner, Erik)
- Replace Components: Comprehensive new system to map properties between
  component classes and map constants to properties for replacement
  components. Import/export of mapping sets via XML. Interactive and
  file-based logging of replacement results. Smarter preservation of
  properties such as TStrings, TWideStrings, TFields, component
  references, etc. (Oasis Digital Solutions, Piotr Likus, Erik).
- Rename Components: Allows interactively renaming components after
  they are placed on the form (Leonel Togniolli). Added ability to edit
  a configurable list of component properties (Caption, Text, etc.) in
  addition to the Name for each component class and supply default values
  for most new component properties. (Thomas Mueller, Erik).
- Macro Templates: Optional expansion of templates when pressing space.
  Ability to embed the selection intelligently inside a construct such as
  try/finally using the %SELECTION% macro. New macros such as
  %BEGINPARAMLIST%, %PARAMNAME%, %PARAMTYPE%, etc. to format individual
  parameters one per line. New default templates for XML method
  documentation, begin/end completion, for/while loops, etc.
  (Piotr Likus, Erik)
- Macro Library: Allows record and playback of saved editor keystroke
  macros (John Hansen, Erik)
- Find Component Reference: Allows quickly jumping from a component to the
  first usage in your source, or from a component name in the source to
  the component on the form. (Kristofer Skaug, Leonel Togniolli, Erik)
- Set Component Properties: New tool to set properties (Boolean, string,
  set, enum, ordinal, etc.) to some value when the a project is compiled.
  Useful to deactivate datasets/connections, etc. (Robert Wachtel,
  Gami, Erik)
- Editor Experts: New alignment tool to horizontally align variable
  declarations, comments, assignments, etc. (Stefan Pettersson, Erik)
- Editor Enhancements: Re-enabled for Delphi 8/2005, including the editor
  toolbar, etc (Rick Hollerich, Erik). Middle mouse button close support
  for the editor tabs in Delphi 6/7 (Gami). Note that the various editor
  tab options can not be supported in the newer IDEs.
- Procedure List: Option to view the selected procedure's source code
  (Paul Gardner, Erik). Optionally search in the class name. (Primoz)

©2026 GExperts Team

Support for scanning Delphi .inc files and upgraded C++ support.
- Code Proofreader: Added basic C# support (Erik)
- Component Grid: Allow editing the Hint property for components (Erik)
- IDE Enhancements: Changing the multiline component tabs setting in Delphi 6/7 no longer requires a restart (Erik)

1.7 Credits

GExperts would not exist without the help and advice of many people. This is probably an incomplete list as some names may have been lost to time. Many others have contributed bugfixes, bug reports and suggestions. A special thanks goes to the Delphi community members who help out in the newsgroups.

Major GExperts contributors

- Arentjan Banck
- Erik Berry
- Jim Campbell
- Primoz Gabrijelcic
- Paul Gardner
- Ulrich Gerhardt
- Benjamin Fournier
- John Hansen
- Taz Higgins
- Stefan Hoffmeister
- Rick Hollerich
- Achim Kalwa
- Per-Eric Larsson
- Piotr Likus
- Ray Lischner
- Jim McKeeth
- Thomas Müller
- Gerald Nunn
- Alex Petrov
- Puthoon
- Mahdi Safsafi
- Egbert van Nes
- Robert Wachtel
- Martin Waldenburg
- Ralph Zornig

©2026 GExperts Team

1.8 Configuration

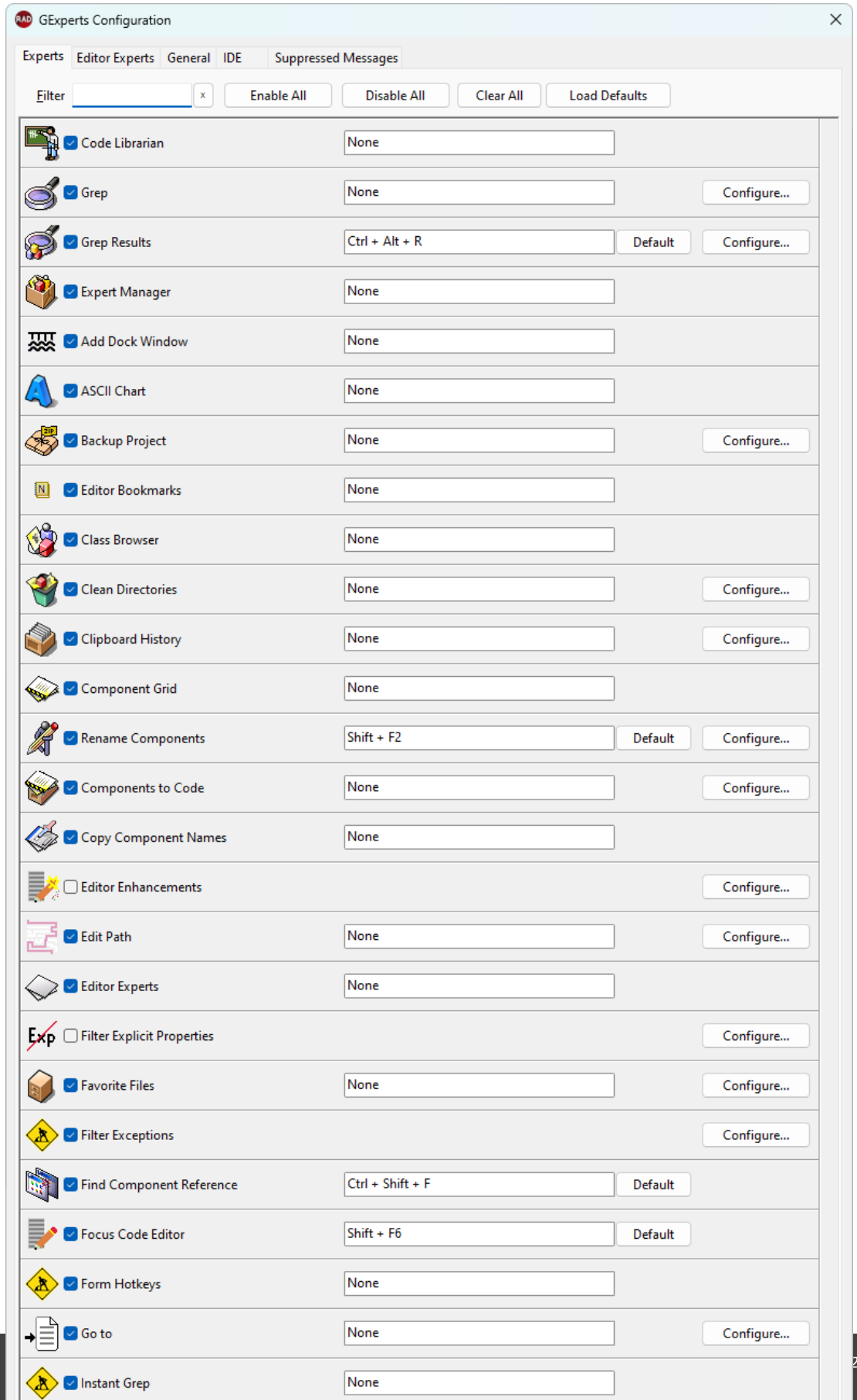
1.8.1 GExperts Configuration

Here you can configure the various parts of GExperts. The window has tabs for configuration of the individual experts, general settings, editor experts, IDE enhancements, and code editor enhancements.

The first tab, shown below, allows you to turn on and off the individual experts using the checkbox next to the expert name. If you uncheck an expert, it will be removed from the GExperts menu. You can also configure the shortcut key used to activate each expert. GExperts makes no attempt to ensure the assigned shortcut key is unique and many common shortcuts are already in use by the IDE's main menu, various context menus, or the code editor. If you find that one keystroke works and a second one doesn't, it is likely because the IDE has reserved the second shortcut for internal use. The IDE's help file lists most of the editor's reserved keystrokes for each keymapping, but not all of them. Note that you can scroll down the list on this tab to see the remainder of the experts.

The general settings, editor experts, IDE enhancements, and code editor enhancements are described in separate help sections.

©2026 GExperts Team

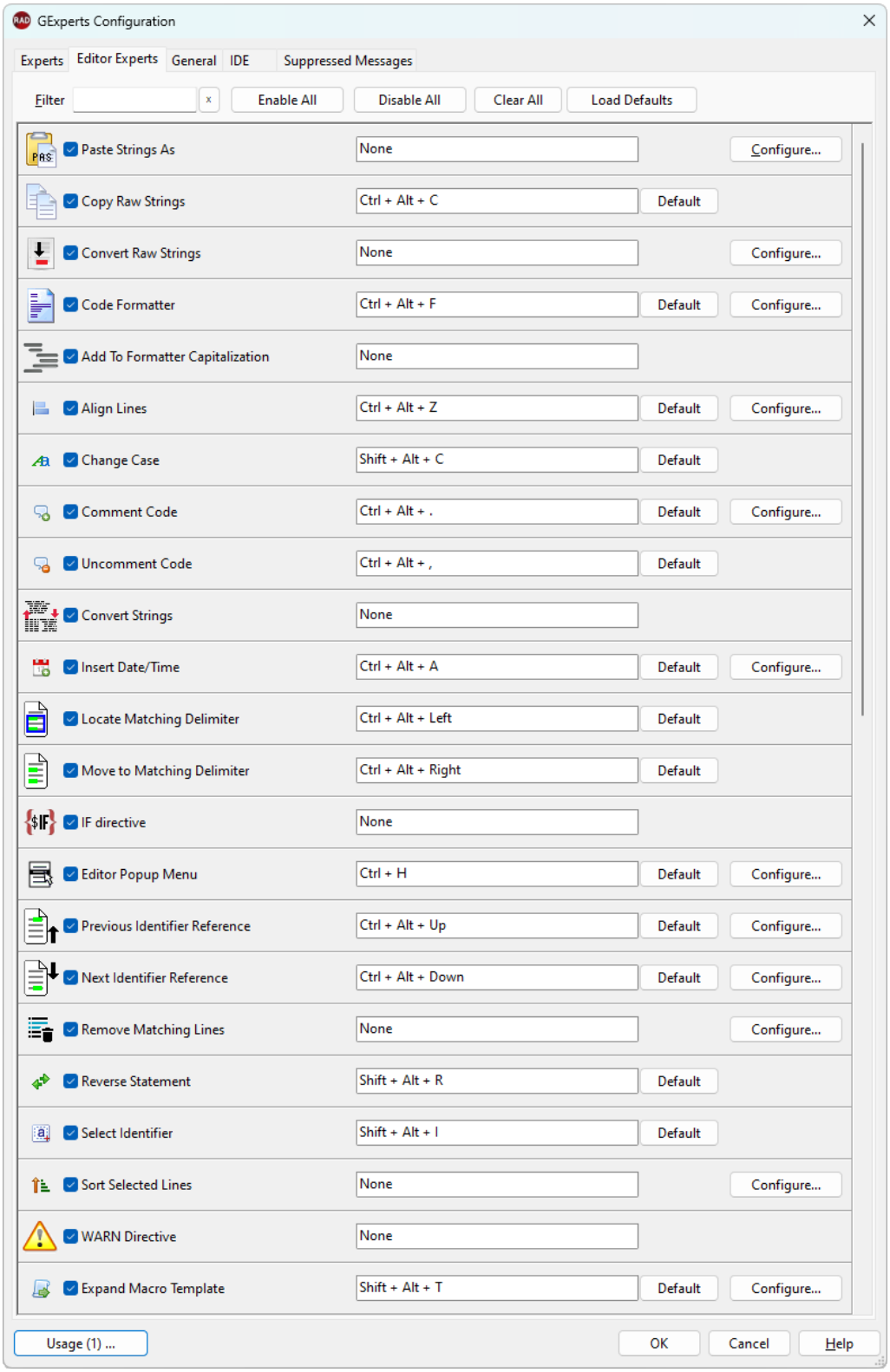

GExperts Configuration
Experts
Editor Experts
General
IDE
Suppressed Messages
Filter
Enable All
Disable All
Clear All
Load Defaults
Code Librarian
None
Grep
None
Configure...
Grep Results
Ctrl + Alt + R
Default
Configure...
Expert Manager
None
Add Dock Window
None
ASCII Chart
None
Backup Project
None
Configure...
Editor Bookmarks
None
Class Browser
None
Clean Directories
None
Configure...
Clipboard History
None
Configure...
Component Grid
None
Rename Components
Shift + F2
Default
Configure...
Components to Code
None
Configure...
Copy Component Names
None
Editor Enhancements
Configure...
Edit Path
None
Configure...
Editor Experts
None
Filter Explicit Properties
Configure...
Favorite Files
None
Configure...
Filter Exceptions
Configure...
Find Component Reference
Ctrl + Shift + F
Default
Focus Code Editor
Shift + F6
Default
Form Hotkeys
None
Go to
None
Configure...
Instant Grep
None

1.8.2 Editor Experts

The editor experts are active only when the IDE code editor has focus. They perform various functions on the source. The shortcut key combination for each editor expert can be changed via the Shortcut button, but you should verify the shortcut is unused by the editor itself (see Key Mappings in the IDE help file for more information) and the main menu.

Some of the experts have options that can be changed using the Configure button. Selecting such an expert enables the button. The function of each of these experts is described in the help memo at the bottom of the tab. There is extended help for the Macro Templates editor expert. The macro templates editor expert replaces the old procedure header and unit header experts. It offers the same style of replaceable `%macros%`, to insert values such as the current date, procedure name, user name, etc. into the editor.

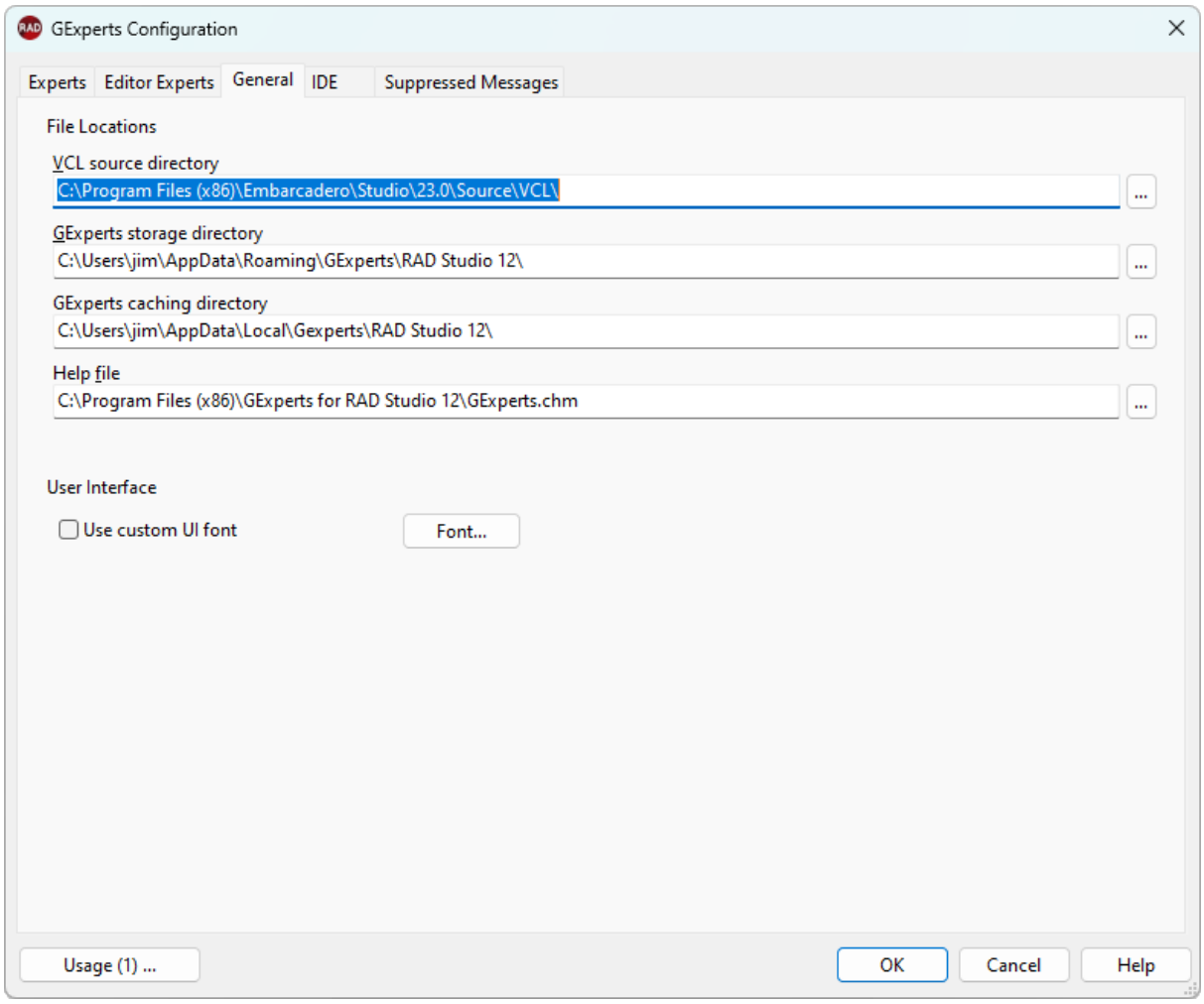
©2026 GExperts Team



©2026 GExperts Team

1.8.3 General Configuration

The General tab allows setting locations for the VCL source, the default GExperts data storage directory (where GExperts stores your .xml and other data files), and the GExperts help file. You can also choose to use a custom UI font for all GExperts forms. **Note** you should only choose 7-9 point variable width fonts, or you may find that the GExperts forms do not scale very well to fit larger text. If you don't specify a custom UI font, the OS default font is used (generally that is Tahoma 8-point for Windows XP and earlier releases, and Segoe UI 9 point for Vista and later).



©2026 GExperts Team

1.8.4 IDE Enhancements

The GExperts IDE enhancements use low-level techniques to interface directly with the IDE and add often-requested features. You can disable all of the IDE enhancements at once using the "Disable IDE enhancements" checkbox.

Menu

You can also choose to sort the GExperts main menu items in alphabetical order and place the GExperts menu in the Tools menu instead of as a top-level menu (this option requires an IDE restart).

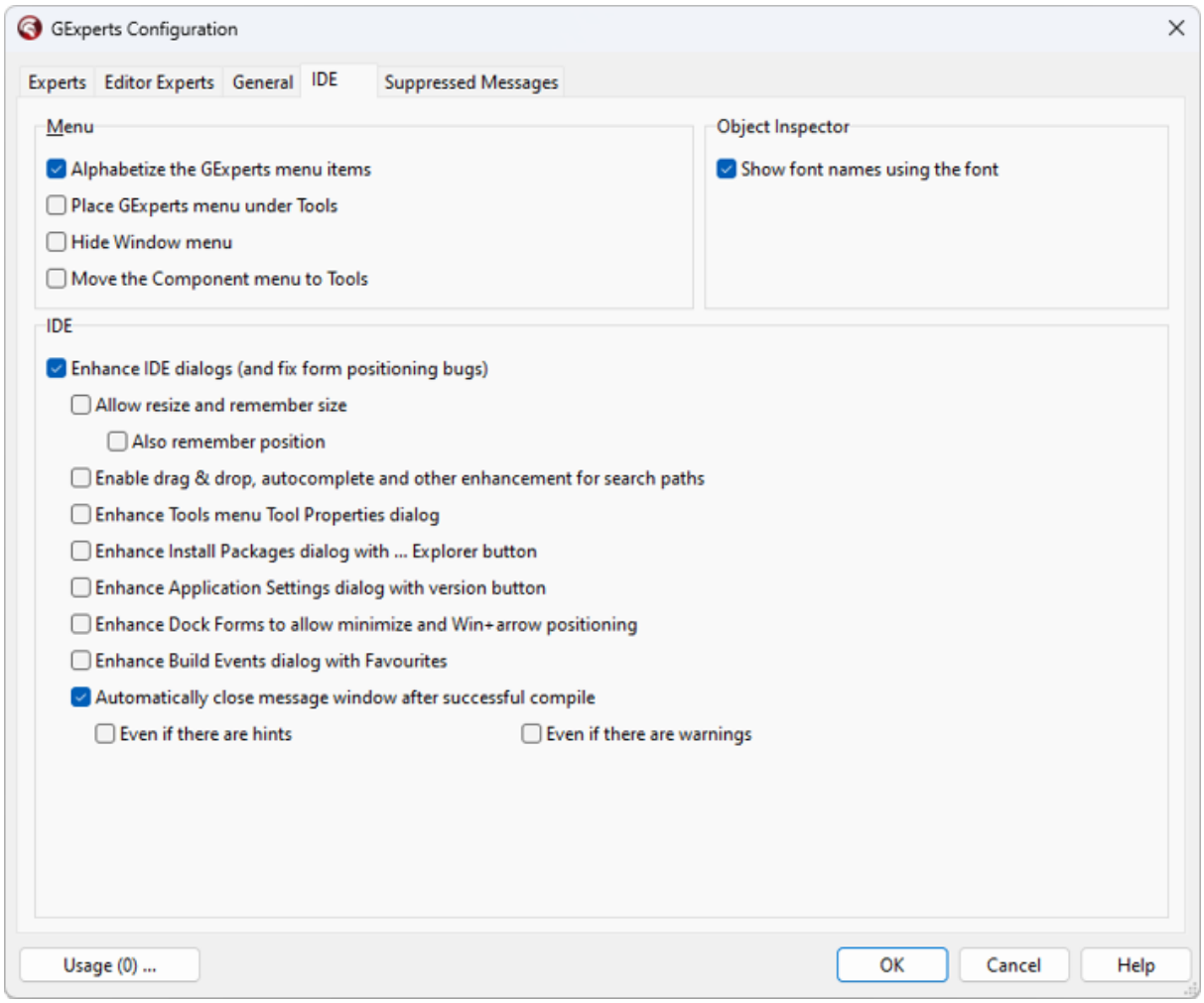
IDE

The Enhance IDE dialogs setting causes GExperts to perform the following enhancements:

- Makes the options dialogs remember their last position onscreen
- Makes the Environment Options and Project Options dialogs resizable (BDS 2006 or later only)
- Saves the splitter position on the options dialogs (BDS 2006 or later only)
- Increases all of the TComboBox DropDownCount values in the options dialog boxes to show at least 15 items
- Collapses the Together node in the options dialog (BDS 2006 or later only)
- Removes the product list horizontal scrollbar in the about box (BDS 2006 or later only)
- Remembers the width of the text search dialog
- Allows resizing the open picture dialogs for image lists and glyphs

You can also tell the object inspector to display font name dropdown selections using the actual font's face.

©2026 GExperts Team



Tab Dock Hosts

When docking multiple IDE tool windows together, GExperts can force the Delphi 6/7 IDE to use multiline tabs when the tabs do not fit in the available space. The multiline tab setting can be changed on a per-dock host basis from the right-click menu of the tabs, or set as the default for all new dock hosts.

Component Palette

GExperts can also enhance the Delphi 6/7 component palette in many ways. You have the option of showing multiple rows of tabs to see more component categories at once. If this option is set, you can also enable the ScrollOpposite and RaggedRight properties of the tab control. Scroll opposite determines where previous rows of tabs in a multi-line tab control move when the user selects a tab in another row. When scroll opposite is checked, previous rows of tabs are moved to the bottom of the tab control. Ragged right determines whether or not incomplete rows of tabs stretch to fill the width of the tab control. See the VCL help for more information on these TCustomTabControl properties.

©2026 GExperts Team

You can also choose to show the component palette tabs in a raised button style (see TCustomTabControl.Style) or a flat button style.

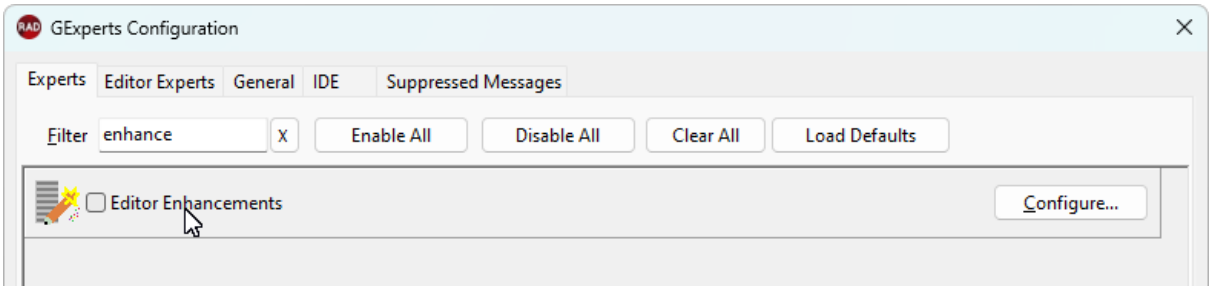
If you want quick access to lots of component categories, but don't want to enable the multi-line tab option, you can add the component categories to the palette's popup menu. Choosing one of these menu items selects the corresponding tab on the component palette. This option is also provided directly by the IDE in Delphi 6+.

Note: Starting with Delphi 8, there is no tabbed component palette, so these options do not apply.
Note: If the multiline option causes component icons to paint improperly when switching forms, it is suggested that you enable the flat button mode, which should fix the problem.

1.8.5 Code Editor Enhancements

The GExperts editor enhancements can improve the functionality of the IDE editor in the following ways:

- Make the code editor tabs multiline instead of utilizing horizontal scrolling
- Turn on/off hot tracking for the editor tabs (see TCustomTabControl.HotTrack)
- Show the code editor tabs in a raised or flat button style
- Enable and then position the GExperts editor toolbar to the top, left, right, or bottom of the code editor window
- In Delphi 6/7, you can configure whether the middle mouse button can be used to close tabs (starting with Delphi 2006, this feature is built into the IDE)

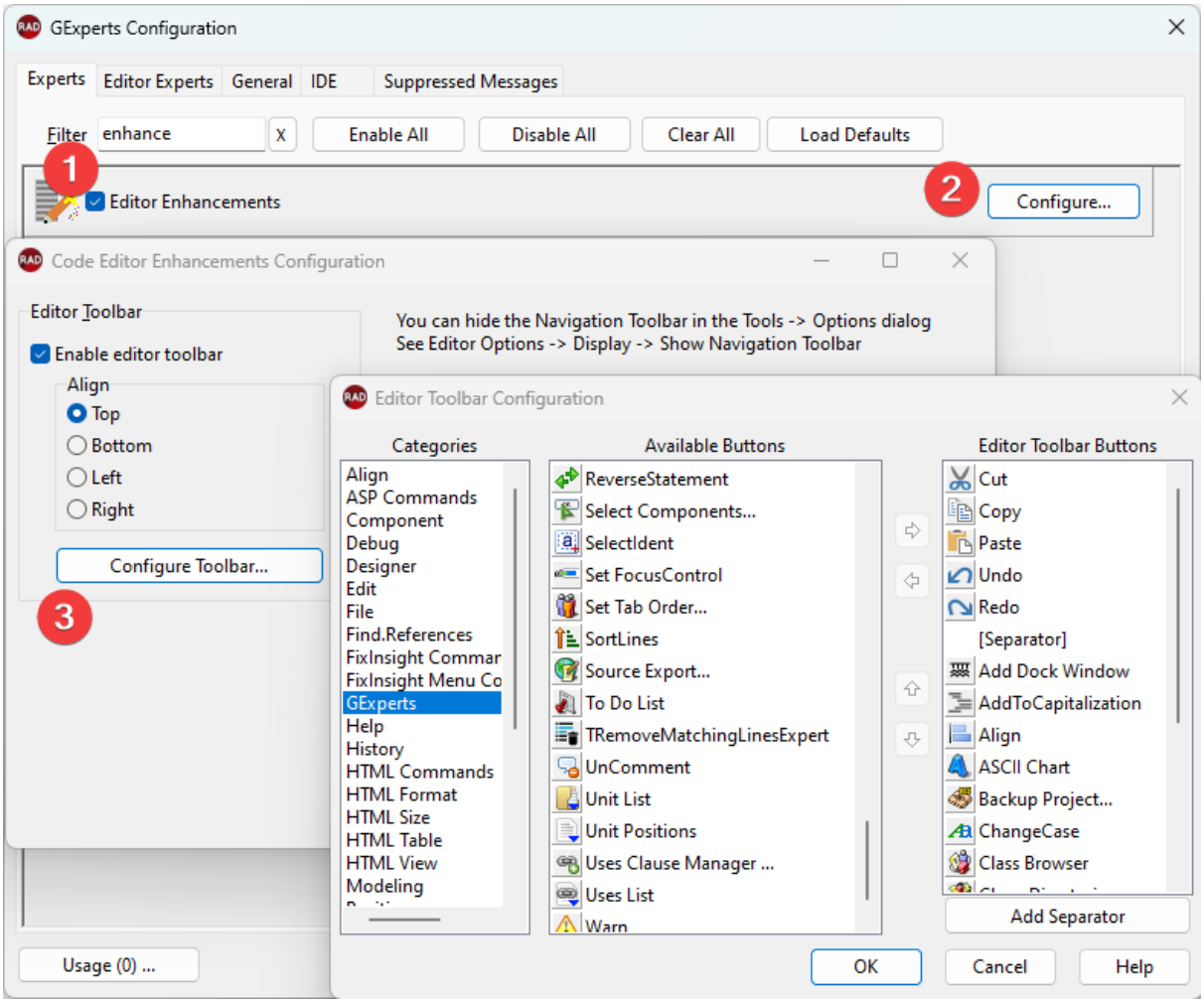


Note: Starting with Delphi 2005, the IDE's editor tabs do not support the multiline capability and the other related customizations.

1.8.6 Editor Toolbar

GExperts includes the ability to place a user-defined toolbar in the IDE code editor windows. Enable it from the Editor Enhancements under the GExerpts Configuration.

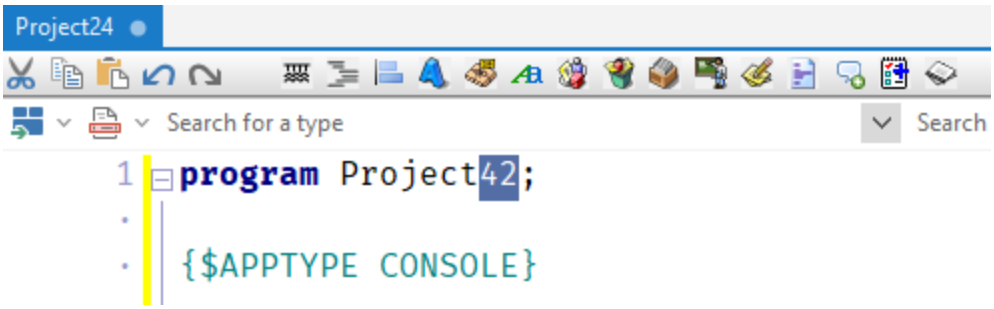
©2026 GExperts Team



To select which buttons are visible on the toolbar, drag buttons from the list of Available Buttons to the Toolbar Buttons list. Reordering buttons in the selected Buttons list is also possible using drag-and-drop or the Up and Down arrows. To remove a button from the toolbar, you can drag it from the Toolbar Buttons list and drop it back onto the Available Buttons list

The majority of the available buttons are taken from IDE's main menu, but there are also several special buttons in the GExperts category that perform custom actions. These custom actions can execute GExperts experts, execute editor experts, show a list dropdown list of forms or units in the current project, or allow you to select a component on the current form by class and name. There is also a button to jump to specific positions in the current Delphi unit. An example editor toolbar is shown here behind the toolbar configuration dialog:

©2026 GExperts Team



Note: Vertical toolbars do not support button separators.

©2026 GExperts Team

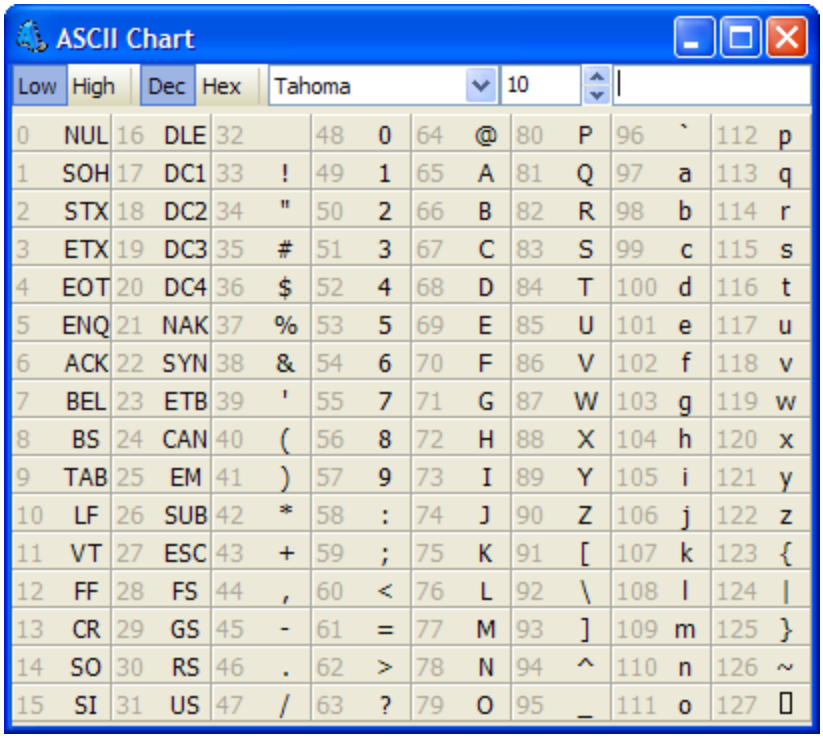
Experts

2 Experts

2.1 ASCII Chart

The ASCII Chart expert shows ASCII values for all of the characters in a given font. You can set various options by right clicking on the grid. You can choose to view the high or low half of the ASCII character range (0-127, 128-55) and choose between HEX and decimal number representations for a character using the buttons at the top of the window. You can also choose the display font and size for the ASCII chart. Several other options are available in the context menu for the ASCII chart.

Clicking a character in the ASCII character grid will copy that character to the edit box at the top right of the window. From there, you can copy any characters to the clipboard for use in your program.



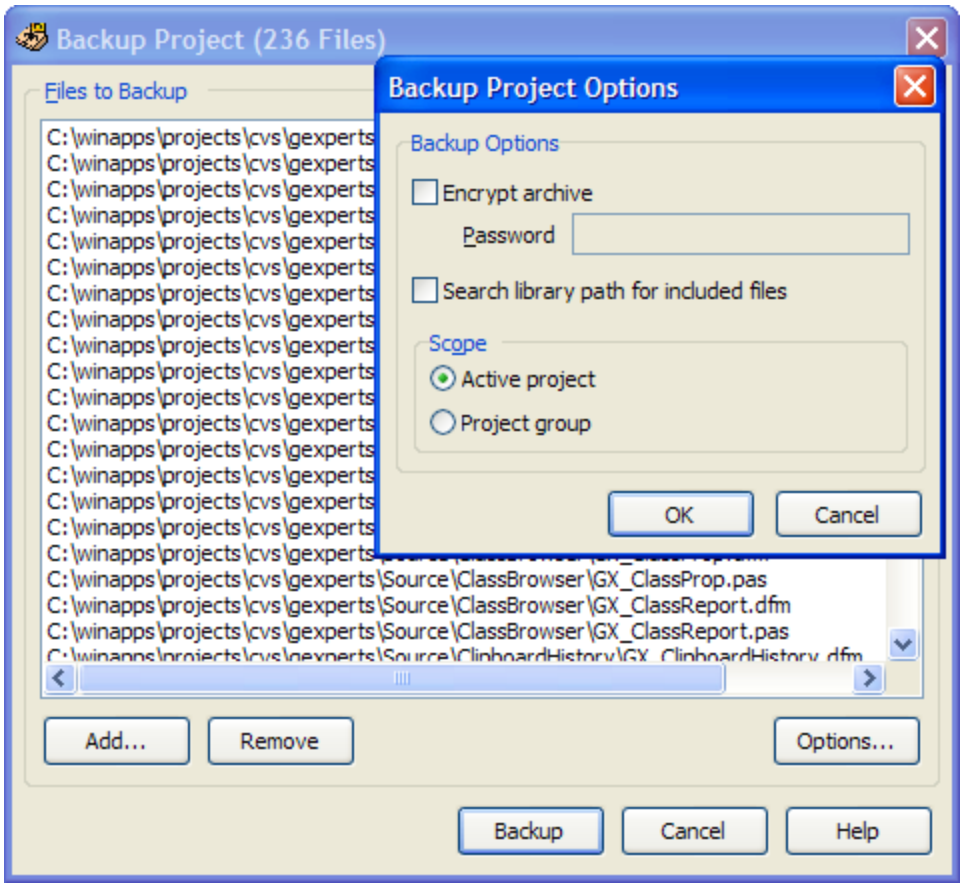
0	NUL	16	DLE	32		48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	TAB	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	|
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	□

2.2 Backup Project

The project backup expert enables you to backup an entire project to a zip file. Always save all files in your project before running this expert or your latest changes will not be included in the zip archive. When this expert is executed, a list of all files in the project appears in the dialog. You can add or remove files from the list using the Add and Remove buttons. To backup the project, click the Backup button.

Quick-access options such as password protection for your archive, the backup scope, and an option to search for included files on the library path are available via the Options button. These options are not saved between invocations of the expert; you must use the Backup Project Configuration dialog described below to set the default options.

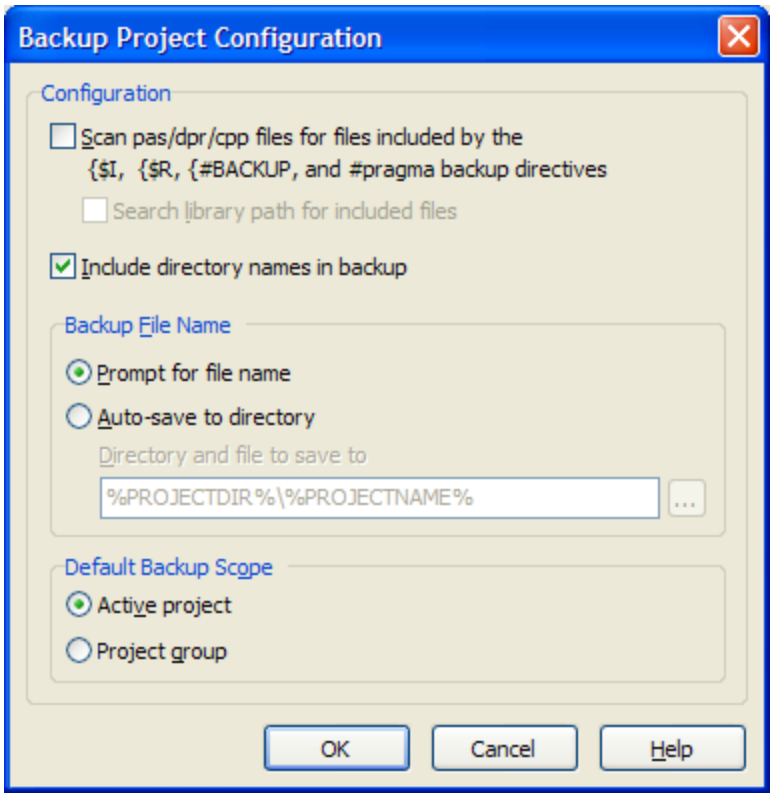
©2026 GExperts Team



The default configuration options are set from the GExperts Configuration window. You can configure the backup project expert to scan all of the project's source files for resource files and other files which should be included in the backup. These files are indicated by the $I, $R, $INCLUDE, and $RESOURCE directives for Delphi, and the included files can be anywhere in the project directory or the IDE/project library path. The backup can also default to backup the project group or just the current project.

If you have enabled the option to scan for included files, you can force certain files to be included in the backup by adding a command directive line to the interface of any unit in the project. For example, if you have a PAS file that always accesses a certain INI file and you want it included in the backup, add a `{#BACKUP MYINI.INI}` line to the top of the file and from then on GExperts will include it in the backup. This expert has support for wildcards and paths in the filename such as: `{#BACKUP *.INI}`, and `{#BACKUP 'C:\Program Files\test.dat'}`. For C++ files, use the syntax `#pragma backup Docs.txt` instead.

©2026 GExperts Team



The expert can prompt for a file name for each backup, or you can instruct the expert automatically backup to a specified directory and file without having to specify a file name each time. Various substitution macros are available that allow you to specify the directory and filename used, for example:

%PROJECTDIR%Backup\%YEAR%-%MONTH%-%DAY% will produce a backup file in a subdirectory called "Backup" off of your main project directory. The filename will consist of the year, month, and day the backup was made.

C:\Backups\%ProjectName%-%DAYSHORTNAME%-%DAY%.zip will produce a backup file named similar to C:\Backups\Project1-Wed-26.zip.

2.3 Class Browser

The class browser enables you to view the classes in your project or in external directories, such as the VCL classes. External class groups can be added using the add button on the toolbar or by selecting Add from the File menu. Your project classes are automatically added to the class browser when you open a project in the IDE. You can refresh the display of classes by pressing the refresh button.

Classes can be viewed as a flat list or tree-structured hierarchy. You can choose which method you prefer using the View menu. To find a specific class, choose Find. The details shown for the selected

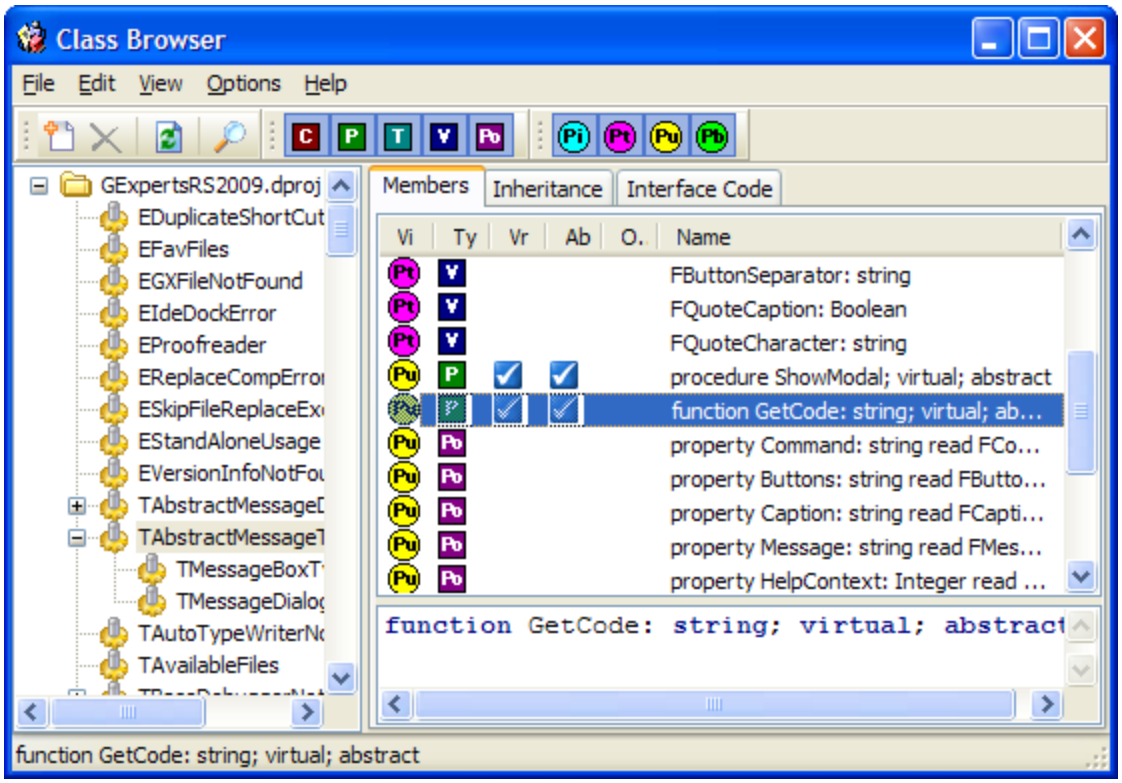
©2026 GExperts Team

class include the class members, the inheritance tree (this works best after you have added the RTL and VCL directories to your class database), and the interface code for the selected class member.

Two reports are available from the File menu. The first report prints the currently selected class and its methods. The second prints a hierarchical report based on the selected class. Both reports have print preview capability.

There is a filter toolbar to select which member types (properties, methods, etc.) and what visibility classes (Private, Protected, etc.) are listed in the Members tab. An options dialog provides access to the default filter settings, font settings, a setting to enable recursive parsing for classes, and other options.

You can jump to the source code for a class or method in the code editor by right clicking on the object and selecting the Goto menu option.

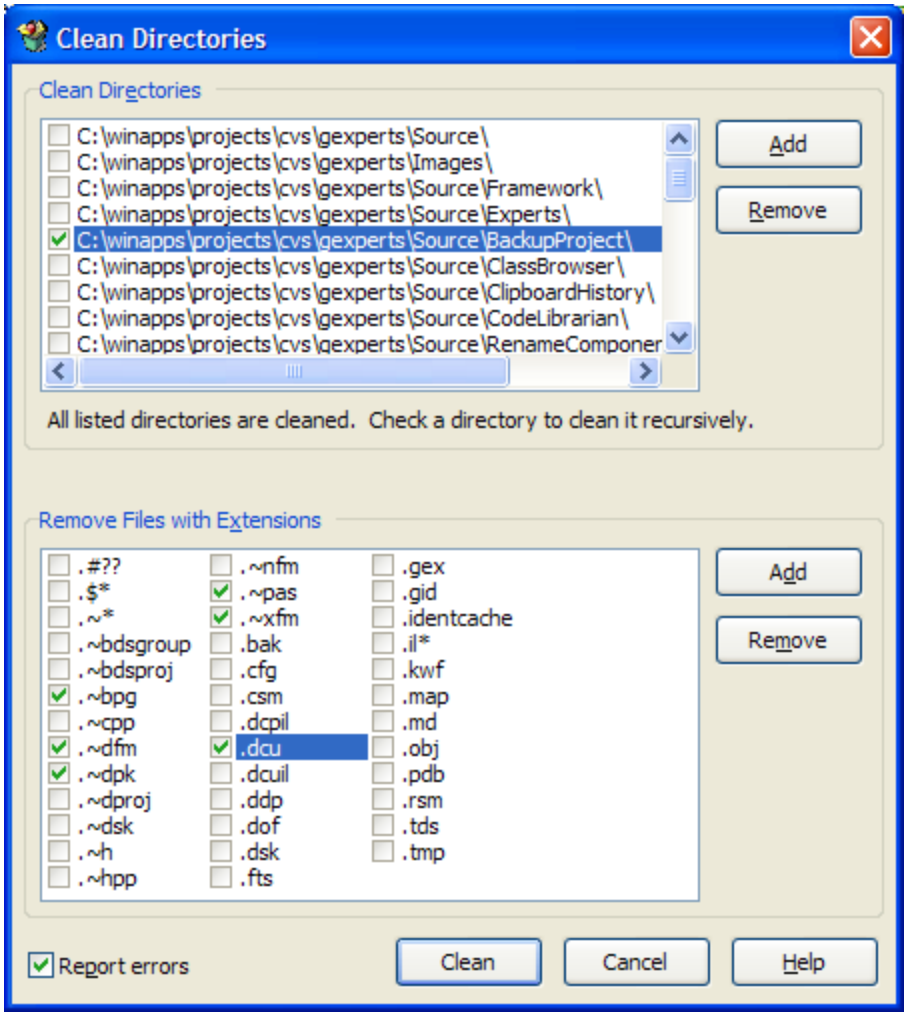


2.4 Clean Directories

This expert enables you to quickly delete several types of non-critical files that can accumulate during development. When it is activated, a list of all directories that project files reside in will appear. All of the listed directories will be searched for cleanable files. Check the checkbox next to a directory to scan it recursively. Select which file types to delete from those directories by checking the various checkboxes in the file extensions listbox. To start the cleaning process, press the Clean button. Once the process is complete, you will be notified how many files were deleted and how much disk space was reclaimed.

©2026 GExperts Team

You can add and delete directories and extensions to be cleaned using the provided Add and Remove buttons. By checking the "Report errors" checkbox you can request that GExperts notifies you when files can't be deleted, such as when the files are read only or in use by other applications.



2.5 Clipboard History

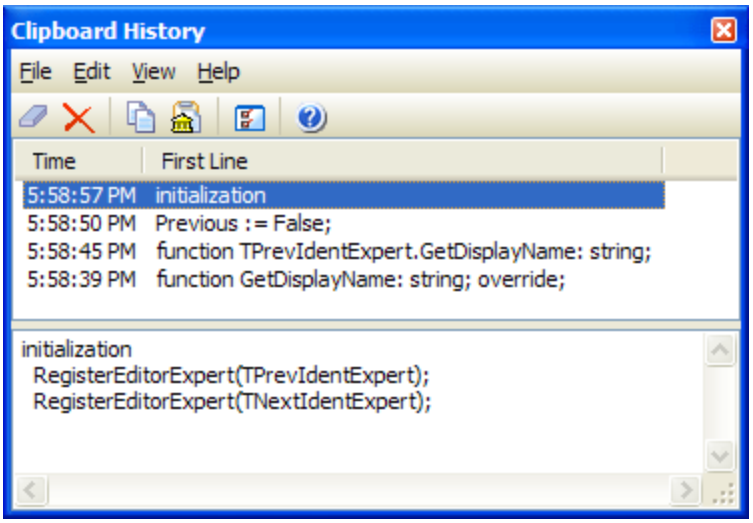
The Clipboard History expert allows you to view the current and previous text contents of the clipboard. You can re-copy any of the saved items by double-clicking the item or pressing the Copy button. The saved items may be purged at any time by choosing Clear. This expert supports IDE docking.

Several options can be set via the configuration dialog, including the maximum number of items to store, whether or not Clipboard History starts capturing automatically when the IDE starts, and whether or not to hide the form after pasting the selected item into the code editor. The clipboard history data is persistent across IDE sessions, and is stored in a file called ClipboardHistory.xml in your GExperts storage directory.

©2026 GExperts Team

Note: The Clipboard History expert only tracks standard ASCII (CF_TEXT) items on the clipboard with a size under 512 kilobytes. It does not maintain a history of other clipboard items such as bitmaps, components, etc.

Note: Some applications (such as Remote Desktop) may improperly hook the Windows clipboard chain or create desktops that break the clipboard chain and prevent GExperts from receiving notification of new items on the clipboard. If you experience this problem, try using the Rehook Clipboard menu item in the File menu.



2.6 Code Librarian

The Code Librarian enables you to organize and access often-used code snippets. It is an easy way to store the various code items that you use often, but do not fit into a general-purpose library unit. The snippets are stored in a custom structured storage file called CodeLibrarian.fs.

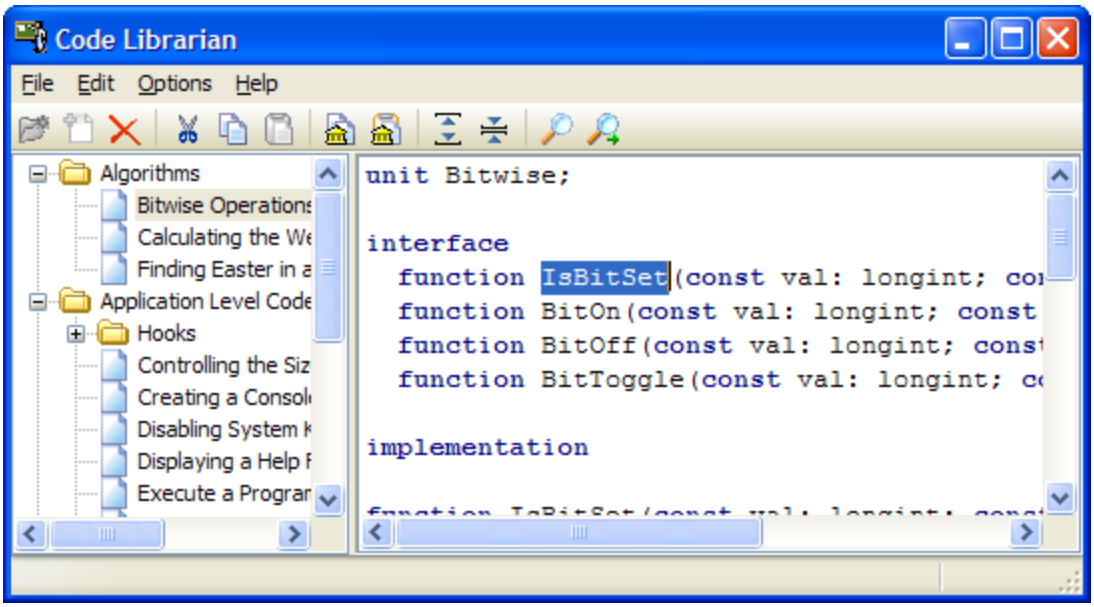
The Code Librarian is composed of two panes. The pane on the left is the folder view that allows you to create nested categories to organize for your code snippets. You can rename, add, and delete folders, or move them using drag and drop. Folder titles can be edited by clicking a selected folder.

The Code Librarian supports copying code from the IDE into the code pane and vice versa.

You can search for keywords by clicking the Find button. Both the topic title and the code pane are searched.

The Options selection allows you to change the display fonts, layout mode, and the location of the Code Librarian snippet file. The syntax highlighting mode for the code snippet can also be toggled between none, Delphi, C++, HTML, and SQL from the context menu of the source editor.

©2026 GExperts Team



Tip: A starter database of code snippets (shown in the screen shot above) that you can integrate directly into the Code Librarian is available on the GExperts home page.

2.7 Code Proofreader

The Code Proofreader is a powerful tool that can correct your typing in the code editor as-you-type based on a list of custom dictionary words, a list of AutoCorrect entries, or context sensitive information supplied by the Delphi compiler. It supports Delphi, C++, SQL, and C# code.

The corrections can be customized per source file type and even for different syntax entities within a file by using the dropdown box at the top of the window. You can import or export dictionary words from/to ASCII files that contain one word per line, or you can insert or delete single words using the provided buttons.

AutoCorrect entries wait for you to press a certain sequence of keys, and then replace that string with another string you define. This can be used to correct typing mistakes you commonly make, or it can be used to expand simple macros. AutoCorrect entries support usage of the "|" character to determine the position of the cursor after the correction takes place, similar to the IDE's Code Templates. If you want to create more complex macros containing multiple lines or dynamic text substitutions, try using the Macro Templates feature instead.

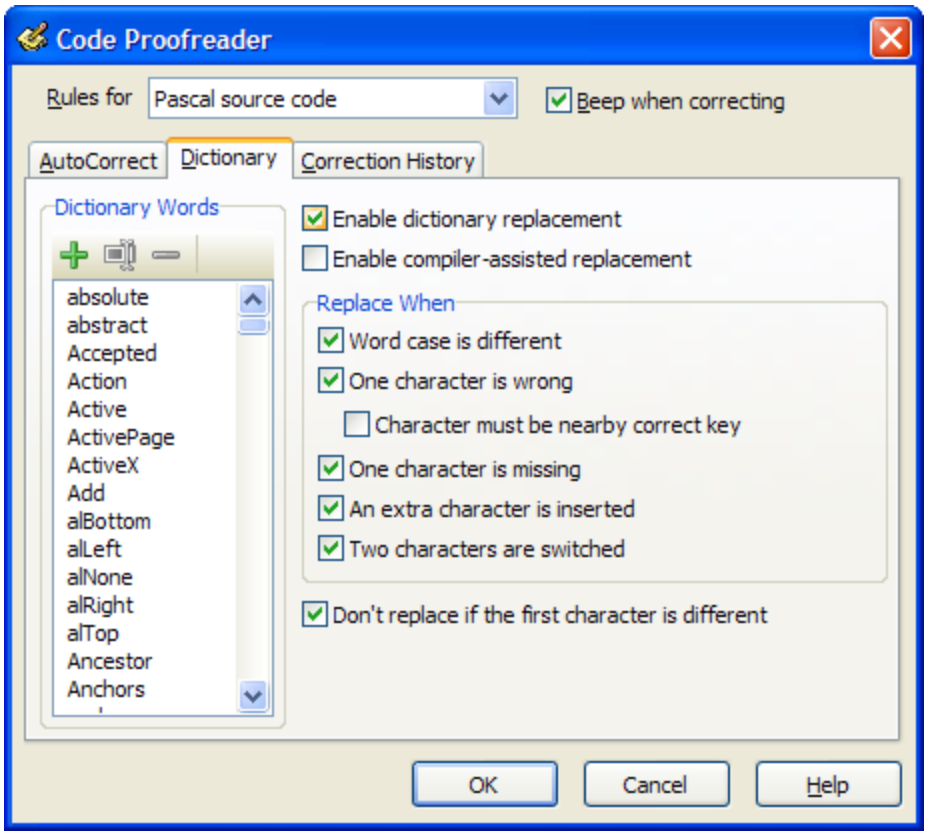
The compiler replacement option allows GExperts to interface with the Delphi compiler to generate a context-sensitive list of potential matches for a typed identifier and use that list to attempt to correct the typed identifier. Be careful, since this option causes the IDE to attempt to recompile parts of the project each time you press a non-alphanumeric keystroke in the source code window, and can slow down your coding on older machines. This option is useful to auto-correct identifiers to match the case they were declared in. Note that if you have dictionary replacement enabled and a typed identifier is present in the

©2026 GExperts Team

dictionary, compiler replacement is not attempted for that identifier, to allow you to easily override replacements where the compiler's suggested results are not ideal. For example, you may want to make sure the dictionary contains the language keywords, so those are never corrected, since the compiler correction feature (which uses the code completion dropdown identifier list) does not support keywords in all IDE versions.

Several options allow you to tune the dictionary and compiler correction. You can choose to correct or not correct based on word case; errors based on incorrectly hitting nearby keys on the keyboard; and missing, inserted, or switched characters in the word. Finally, the Code Proofreader can issue a beep after correcting to make sure you know it was activated.

The Correction History tab on this form shows the last few corrections that were made, so you can review exactly what the Code Proofreader has changed and what rule triggered the correction, since this isn't always obvious. You can also disable any of the recent corrections, if you find a correction is not helpful.

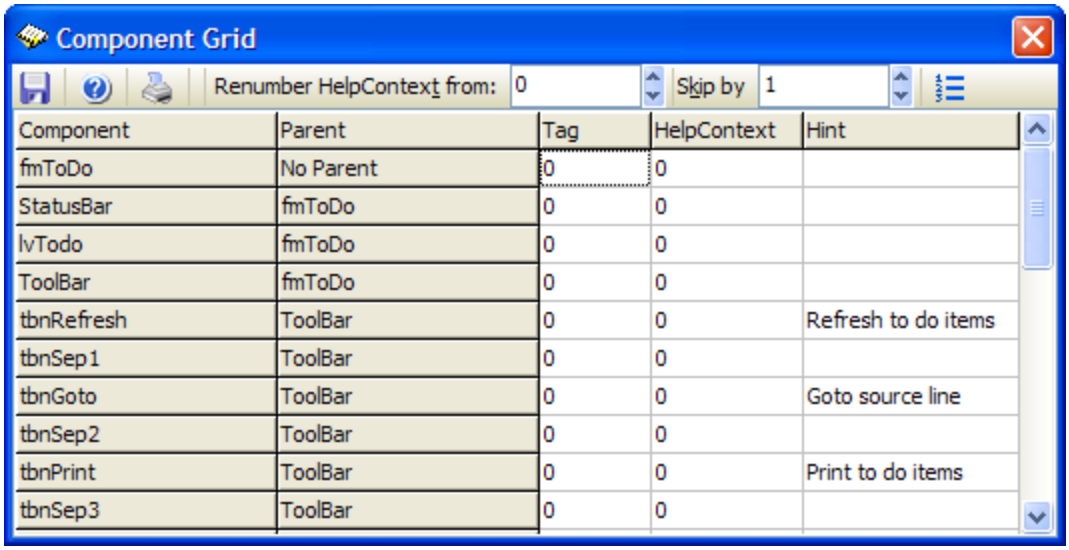


2.8 Component Grid

This expert enables you to edit the Tag, HelpContext, and Hint properties of all components on a form at once instead of having to individually select components. You can sort the component list by name or parent by clicking the relevant column header. When you want to save your changes, press the Save button.

©2026 GExperts Team

The toolbar controls provide support for renumbering help context IDs starting from a given number and skipping by another number and the ability to print a simple report listing all of the components on the current form.



2.9 Components to Code

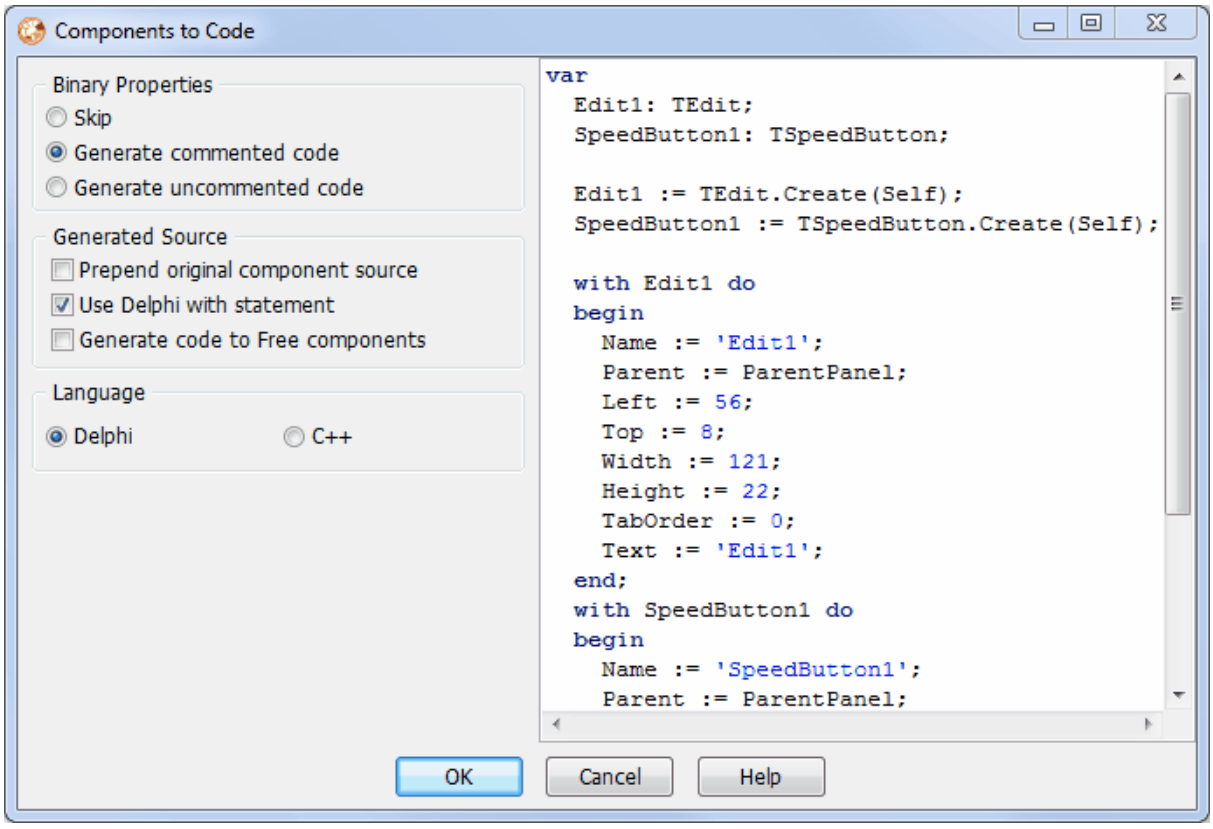
This expert allows you to select one or more components on a form and copy the appropriate code to the clipboard that would create those same components at runtime with any modified property settings intact. This expert is also available from the Form designer's context menu.

You can set the following options in the configuration dialog:

- **Binary Properties**: When encountering binary properties such as bitmaps, the tool can completely skip them, create a comment in the generated code that contains the binary data, or create uncommented code you then manually change to setup your own custom binary data.
- **Prepend Original Component Source**: Include a copy of the original DFM source so you can later paste it back on a form, if needed.
- **Use Delphi With Statement**: Use a "with" statement to set each component's properties, when generating Delphi code.
- **Generate Code to Free Components**: Enable if you want to manually handle memory management by generating the code to free each component and set the component owner to nil/null, rather than making the owner/form automatically free the component by setting the owner to Self/this.
- **Language**: Choose the target language for the generated code (Delphi or C++).

As you change the options in this dialog, a sample view of the component creation code is updated to reflect your options.

©2026 GExperts Team

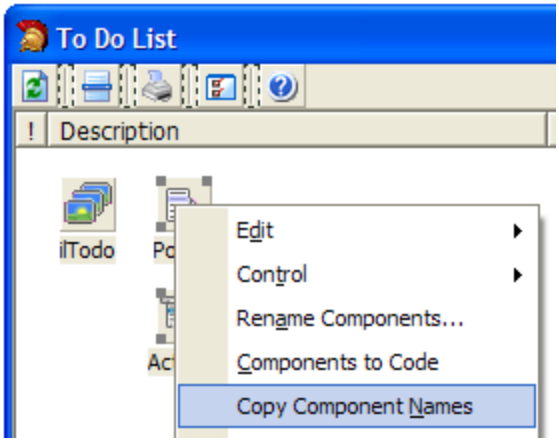


Note: This expert will not work perfectly with some components with subcomponents such as TTeeChart/TDataSet/TTreeView and third party tabsheets that do not descend from TTabSheet, because the sub-components need component-specific code to create or register them with the parent/owner component.

2.10 Copy Component Names

This simple expert allows you to copy the names of a form's selected components to the clipboard. The format with one component name per line. This might be useful to select several components and then paste the names into the code editor, so you can easily operate on the components without retyping the names. This expert is also available from a Form's context menu.

©2026 GExperts Team



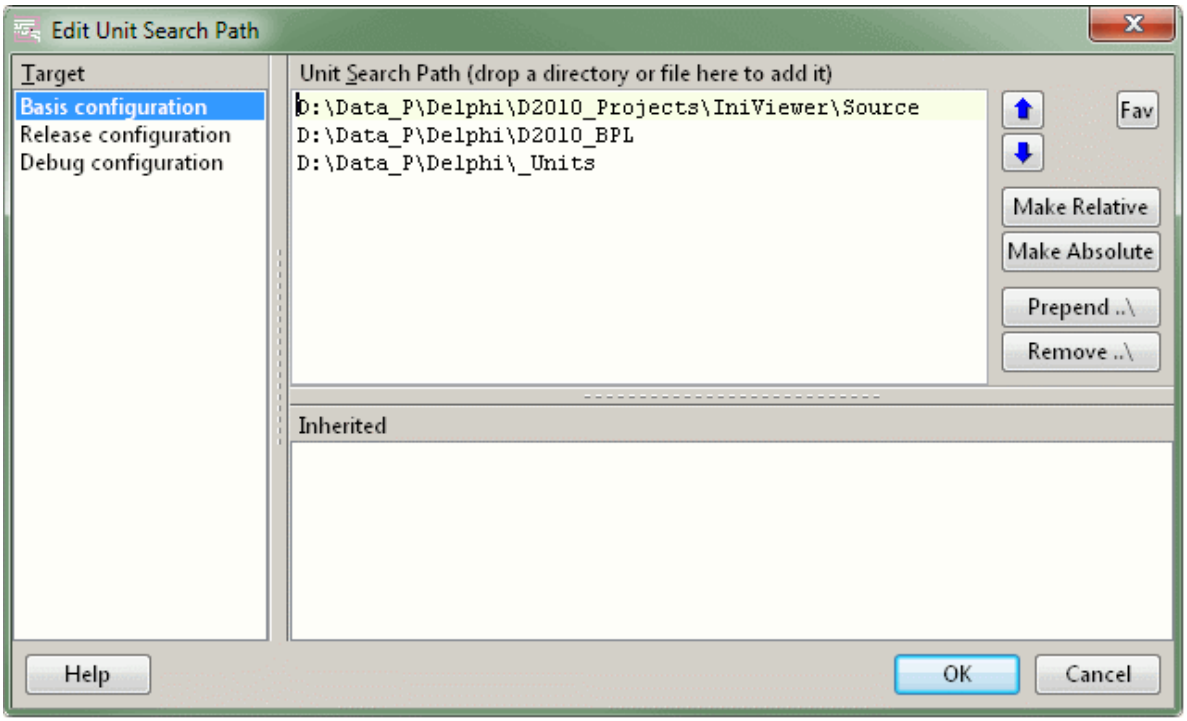
2.11 Edit Unit Search Path

this page ..	see also ..
Overview	General
Details	Favorites
Target	Favorites, Add or Edit
Unit Search Path	Configuration (Options)
Inherited	

Overview

The "Edit Unit Search Path" expert helps you edit the unit search path of the current project. It provides easy access to the unit search paths of all targets configured for it. A configurable favorites menu gives access to frequently used paths.

©2026 GExperts Team



Notice
- This expert is only available in Delphi versions from Delphi 2009 and later because the OTAPI did not offer access to the target list in earlier versions.
- In all Delphi versions from Delphi 6 and later the "IDE Enhancement for search paths" should work - the functionality of the enhancement is similar to the capabilities of this expert, albeit more restricted.

Details

Target

- The target list shows all compiler targets configured for the current project.
- By default, the currently active compiler target of the project is preselected. This behavior can be changed in the configuration; the configuration dialog is accessible from in the GExperts configuration dialog.
- You can select the desired target with the *mouse* or with *Alt + up/down arrow keys*.
- The list does not provide options for creating or deleting targets; this must be done in the IDE.

Unit Search Path

©2026 GExperts Team

- This is your central work area for managing search paths. It shows all defined search paths for the selected compiler target in the target list.
- The list is actually no list but an editable TMemo; this allows you to quickly delete multiple lines or transfer paths and individual folder names using copy and paste.
- It is possible to drop directories from the Windows Explorer on the memo to add them.
- The alternative way to add directories is to use the *"Fav"* (favorites) button. The button shows a pop up from which you can select a favorite or open the favorites configuration dialog. If no favorites have been configured, the button immediately shows the favorites configuration dialog. A single favorite entry can contain multiple directories, so it is e.g. possible to add all directories necessary for libraries like Indy, JCL or JVCL with one click rather than having to add each of them separately.
- With the *up / down buttons* or with *Ctrl + up/down arrow keys* you can change the order of the directory entries and thus their search priority and the compiler speed.
- Using the buttons *"Make Relative"* or *"Make Absolute"* all directories in the memo will be made relative to the project file directory or absolute respectively.
- With the buttons *"Prepend ..\"* or *"Remove ..\"* you can add or remove a relative path prefix for the currently selected line(s) to specify a parent or child directory as the starting point for the path specification.

Notice
- The ability to edit data without restrictions requires disciplined work, as the accessibility of the paths is not checked by the application. This option is very powerful, but you should know exactly what you are doing and whether your path specification references a correct directory under all circumstances.
- Most of the time you should prefer relative paths to absolute paths where possible.

Inherited

- The lower list on the right is not editable, the list shows all inherited search paths for the current selected compiler target.

2.12 Editor Bookmarks

this page ..	see also ..
Overview	General
Setting bookmarks	Add and edit bookmarks
Edit and delete bookmarks	Select function or procedure
Bookmarks list	Configuration (Options)

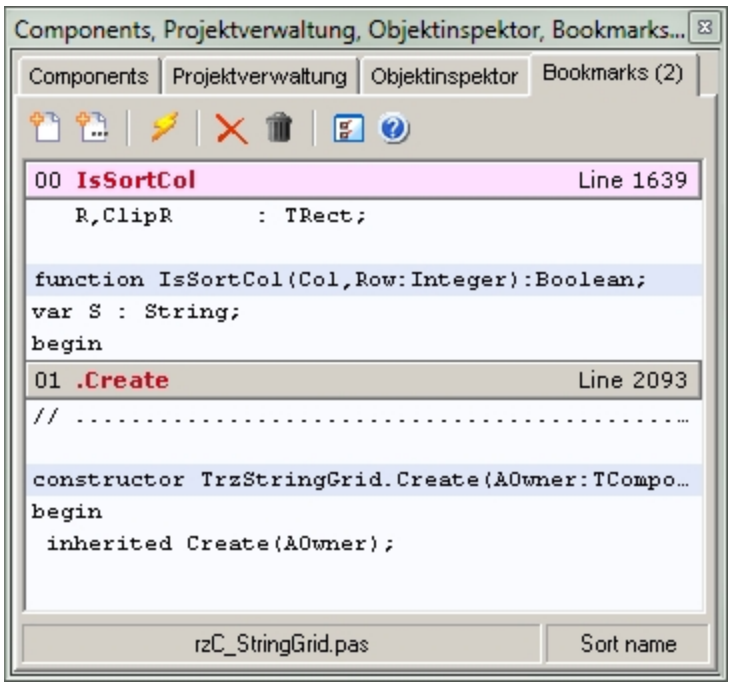
Overview

The "Editor Bookmarks" expert offers convenient ways to create, handle and use bookmarks. The expert has a dockable main window; this mode should also be the

©2026 GExperts Team

primary form of use. The expert complements the bookmark function available in the IDE and offers new ways to keep a good overview of the bookmarks set and to access them specifically.

Notice
- In Delphi 7, bookmarks are not saved by the IDE. In Delphi 2010, this happens automatically, allowing the expert to be used optimally.
- Unfortunately, due to system constraints, the experts help cannot be called with F1 in docked windows. If a dockable window is currently undocked, the F1 key can be used.
- In current Delphi versions the plugin "Bookmarks" available via "GetIt" can be used.



Setting bookmarks

Bookmarks now can be set in three different ways:

- As usual via the IDE (Bookmarks 0 ... 9).
- The 1st toolbutton is used to set the first free bookmark for the caret position in the editor (Bookmarks 0 ... 19). Duplicate entry for the same line is prevented.
- If specified in the options, this automatic function can be deactivated and the editor window known from earlier version will be called by the 1st toolbutton to optionally set any bookmark.
- The 2nd toolbutton opens an additional window in which the position of the new bookmark can be set from a list of all functions and procedures of the active file in the IDE.

©2026 GExperts Team

Notice
- Try to keep your methods as short as possible and, if useful, place the bookmark on the method header!
- If enabled, the expert prevents the creation of duplicate bookmarks within a line. However, this is generally possible within the IDE (and in "Editor Bookmarks" Edit form).

Edit and delete bookmarks

- If specified in the options, a 3rd toolbutton (as shown above) appears and let you modify an existing bookmark.
- Using the toolbuttons with the red cross or the trash can symbol, individual or all existing bookmarks can be deleted.
-

Notice
- Of course the bookmarks can also be deleted directly via keyboard command in the IDE editor.

Bookmarks list

- Activated bookmarks are displayed in the listbox of the expert window. The header of each bookmark item contains its index, the name of the assigned function/procedure (a leading dot marks a class method) and the line number.
- Below the header, an excerpt of the source text before and after the bookmark is shown, the bookmark line is highlighted. In the options, you can set whether 3 or 5 lines are shown at this point.
- The file name of the source text file for which the bookmarks are set is displayed in the 1st panel of the statusbar.
- By repeatedly clicking on the lightning symbol in the toolbar, the list can be sorted by the attributes *BookmarkIndex*, *MethodName* and *LineNumber*. The active sort mode is displayed in the 2nd panel of the statusbar.

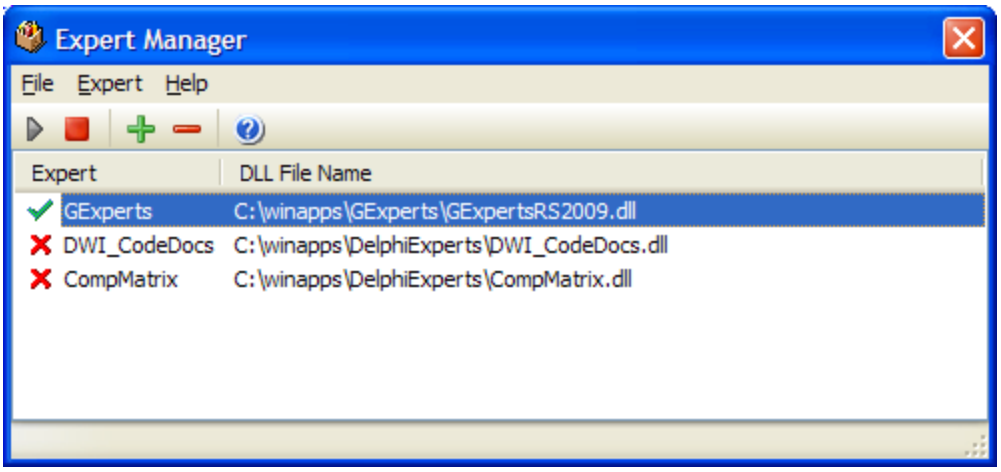
Notice
- Unfortunately, the determination of the method identifier within INC files is still faulty.

2.13 Expert Manager

The expert manager enables you to add, enable, disable, and remove DLL experts from the IDE without having to directly edit the registry. The experts marked with a red **X** are disabled and those with the green checkmark are enabled. If an expert has a red dot in front of the graphic, the expert's state has been changed since the IDE was started, and you will not see the effect of the change until you restart the IDE.

Note that the expert manager does not allow you to turn on and off the various GExperts experts. This can be done in the GExperts Configuration window using the checkboxes on the Experts tab. Package-based experts should be enabled and disabled using the IDE's Project Options dialog.

©2026 GExperts Team



2.14 Favorite Files

The Favorite Files expert provides a convenient way to organize commonly used files. You can create as many nested folders as you like, and each folder can be one of several types representing bitmaps, source code, help files, folders, glyphs, etc. To add a new folder, right click the folder pane and select the new menu. A window will appear prompting you to enter a folder name and type. The folder type is used to determine which icon is used represent the folder.

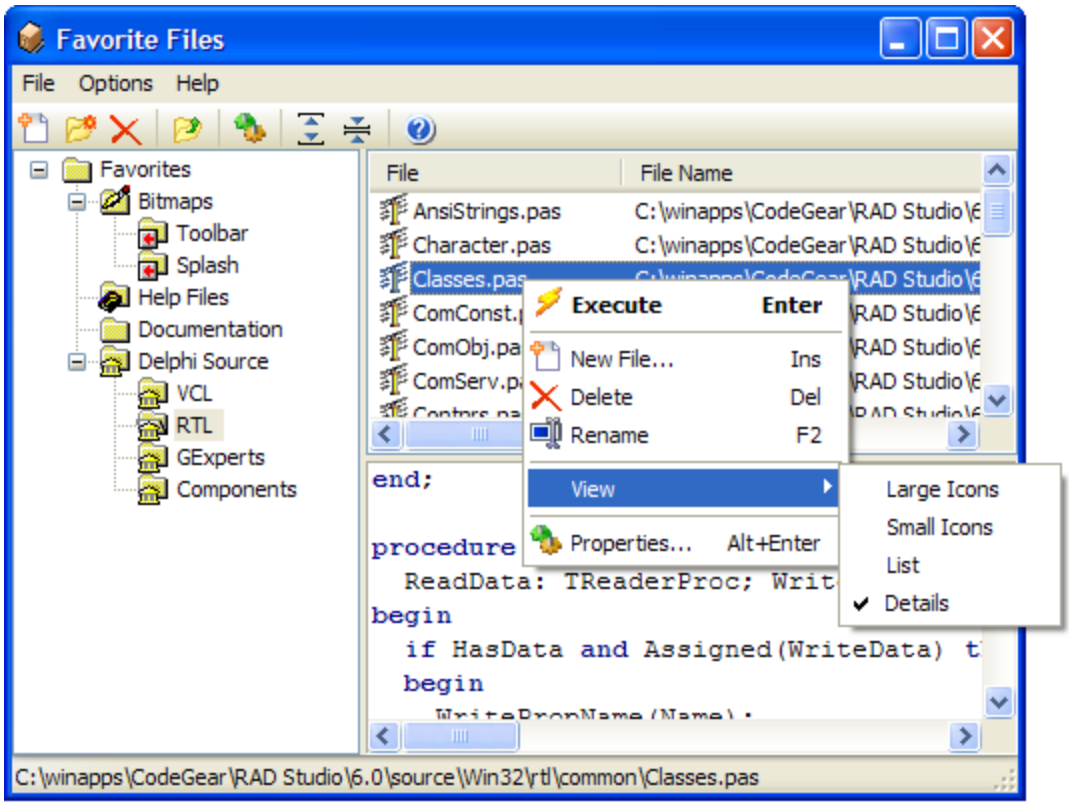
Similarly, you can add files to a folder by right clicking on the file pane and selecting new. An open file dialog will appear, from which you can select multiple files to add to the current folder. After adding the files, you can edit their properties by bringing up the property dialog. Each favorite item has a file name, a name, a description, and an execution type. The execution type determines what to do with the file when it is executed as follows:

IDE: Load the file into the IDE
Shell: Use the ShellExecute API call to run the file with the associated program
Custom: Manually specify an application to use to open the file
Project: Load the file as a new project into the IDE

A file viewer is available in this expert to preview files as they are selected. It can preview several file types such as PAS, DPR, RTF, HTML, BMP, SQL, C, CPP, TXT, etc. To speed up the process of selecting files, you can hide the preview panel via the Options menu.

Tip: You can drag and drop files from Windows Explorer into the file listing to quickly add new entries.

©2026 GExperts Team



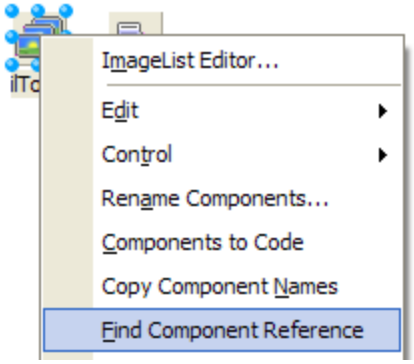
2.15 Find Component Reference

The Find Component Reference tool operates in two different modes:

- If a form designer is active and there is a currently selected component, the tool focuses the code editor and then tries to locate the first reference to that component in the source file. For Delphi source files, it will try to locate and select the second occurrence of the component, since the first occurrence of the component name is usually the declaration instead of the first usage of the component. Component references in comments will be found and selected.
- If the code editor is active, the tool grabs the identifier under the editor cursor and tries to locate a component on that unit's form named the same as the identifier and selects that component.

Note: Find Component Reference can not locate references to components on other forms/frames or source code references in units other than the current one.

©2026 GExperts Team



2.16 Formatter

this page ..	see also ..
Introduction	
Limitations and known problems	
Options dialog	
Indent tab	
Spacing tab	
Line breaks tab	
Capitalization tab	
Align tab	

Introduction

The GExperts code formatter is based on DelFor by Egbert van Nes. The following is from the original help file that came with DelFor (Nobody has yet found the time to update it, sorry. If you want to volunteer, contact the project lead.)

Delfor is a customizable source code formatter. It can improve the indentation, spacing, capitalization and the use of blank lines of Delphi source code. To do so the source code is parsed and lots of special constructs are tested. At default, the style of the Borland source code is followed closely, since this is the most widely accepted style. But formatting of Pascal code is much a matter of taste and debate. Therefore, some alternative rules are included.

Limitations and known problems

- Compiler {$IFDEF} + {$ELSE} directives may be nested to 3 levels and break into blocks of code. After the third nested level the right indentation is not guaranteed.
- After some options (aligning, adding line breaks) the indentation might be not correct. Rerunning DelFor can fix the problems.

©2026 GExperts Team

- It is tried to indent function directives after function declarations. In some cases this does not happen.

Options dialog

In the options dialog some formatting rules can be deactivated or customized.

The following tab sheets are included:

- Indent
- Spacing
- Line breaks
- Capitalization
- Align
- Miscellaneous
- Preview

Indent tab

Spaces per indent

Defines the number of spaces per indent. In all situations the same amount of spaces is used.

Extra indent after begin or try:

If checked an extra indent is added after if-then-else, with-do, on-exception-do and while-do statements:

1. with extra indent:

```
if i > 10 then
  begin
    l := 3;
  end
else
  begin
    l := 0;
  end;
```

2. no extra indent (default):

```
if i > 10 then
begin
 l := 3;
end
else
begin
  l := 0;
end;
```

Extra indent after else in try block:

If checked an extra indent is added after else in the following kind of try block:

Without option:

©2026 GExperts Team

```
try
  DoRun;
except
  on ESomeException do raise;
  on ESomeOtherException do raise;
else
  ShowMessage( );
end;
```

With option:

```
try
  DoRun;
except
  on ESomeException do raise;
  on ESomeOtherException do raise;
  else
    ShowMessage( );
end;
```

Extra indent after else in case block:

If checked an extra indent is added after else in the do case statement:

Without option:

```
case i of
  someValue1:
    DoSomething1;
  someValue2:
    DoSomeThing2;
else
  DoSomeThingElse;
end;
```

With option:

```
case i of
  someValue1:
    DoSomething1;
  someValue2:
    DoSomeThing2;
  else
    DoSomeThingElse;
end;
```

Never indent else if

With this option checked, the indent never increases after else if . The default behavior is that the indent only increases if there is a line feed between if and else

Default:

```
if t then
  k
else
  if t then
    j
  else if k then
    t;
```


©2026 GExperts Team

With option:

```
if t then
  k
else
if t then
  j
else if k then
  t;
```

Indent comments

If this option is checked comments are aligned with the surrounding text, else comments remain unchanged.

With indent comments

```
if t then
begin
        {this is
            a comment
           }
    j;
end;
```

becomes:

```
if t then
begin
  {this is
     a comment
  }
  j;
end;
```

Indent compiler directives

If this option is checked compiler directives are aligned with the surrounding text, else compiler directives are not indented.

With indent compiler directives

```
if t then
begin
           {$IFDEF debug}
j;
      {$ENDIF}
end;
```

becomes:

```
if t then
begin
  {$IFDEF debug}
  j;
  {$ENDIF}
end;
```


©2026 GExperts Team

if this option is unchecked the result is:

```
if t then
begin
{$IFDEF debug}
  j;
{$ENDIF}
end;
```

Spacing tab

In this tab sheet you can define the spacing of various operators.

The spacing of each operator has four values:		
None	Nospaces around the operator	1+1
Before only	Only spaces before the operator	1 +1
After only	Only spaces after the operator	1+ 1
Before and after	Spaces around the operator	1 + 1
The operators are grouped in the following way:		
Equals	:=	
Math	+ - / * < > = and as div downto in is or shl shr xor	
Colon	:	
Comma	,	
Left parenthesis	(	
Right parenthesis	)	
Left bracket	[	
Right bracket	]	

Line breaks tab

Always break lines

Line breaks can be added:

After var, type etc. :

```
        procedure Test(var i: Integer; const t: Byte);
var t: Integer;
type Pword = ^Word;
```

becomes:

©2026 GExperts Team

```
        procedure Test(var i: Integer; const t: Byte);
var
        t: Integer;
type
Pword = ^Word;
```

Before end :

```
        if i > 10 then
        begin
        l :=3; end;
```

becomes:

```
        if i > 10 then
        begin
        l :=3;
end;
```

After semi colons (except directives):

```
        if i > 10 then
        begin
        l :=3; k := 3; s:=3;
        end;
```

becomes:

```
        if i > 10 then
        begin
        l :=3;
k := 3;
s:=3;
        end;
```

Note: no lines are added after directives like virtual, abstract, message. A construct like procedure test(k: integer; l: word); forward; remains unchanged.

Between else and if:
Add line between else and if. Note that this option is opposite to the After then option. Check the Never indent else if option to prevent indenting.

After then, else(except else if), do, of :

```
        if i > 10 then l :=3 else if t=0 then k := 3;
        for i:=0 to count-1 do inc(k);
```

becomes

```
        if i > 10 then
l :=3
else if t=0 then
k := 3;
for i:=0 to count-1 do
inc(k);
```


©2026 GExperts Team

Use the Wrap long lines option to break large if then statements. (Wrap text breaks text on then or do if possible).

Note that the lines are not removed if this option in unchecked, therefore it is irreversible.

Except single lines:
If this option is selected in combination with the previous option (After then, else(except else if), do, of :) then single lines after if then, on : are NOT processed:

```
if i > 10 then l := 3;
case i of
1: t := 0;
2: t := 3;
end;
```

Never before then and do :

```
if i > 10
then
l := 3;
```

becomes

```
if i > 10 then
l := 3;
```

Note: if the Wrap Long lines option is checked then still might be moved to a single line.
Remove double blank lines
Check this option to remove all double (or more) blank lines. At most one blank line is then allowed.

Blank lines
Optionally blank lines can be added after
main procedures or
local procedures.

Wrap long lines
It is possible to let DelFor break long lines into smaller ones.
If a formatted line is longer than the Wrap position, it is tried to break it in smaller lines. If the long line contains then, else or do, the line is broken on that reserved word, otherwise a line feed added after the rightmost space. Lines without spaces and comments are not changed.
Note: the wrapped lines are not always indented correctly. Rerun DelFor to correct this.

Begin style:
1. unchanged
1. hanging after then, else do :
1. breaks before and after

The code:

```
if i > 10 then begin l :=3
end;
```

stays the same if option 1 is selected. If option 2 is checked:

©2026 GExperts Team

2. hanging:

```
if i > 10 then begin
l :=3
end;
```

3. breaks before and after:

```
if i > 10 then
begin
l :=3
end;
```

Capitalization tab

Reserved words/Standard directives

The reserved words and standard directives can be written in:
1. lower case
2. UPPER CASE
3. First Character Uppercase
4. remain unchanged.

Note that after one of the first three options is chosen the forth option has no effect anymore. Use the File with user defined capitalization option if you want different capitalization for some reserved words. If standard directives are used as identifier, in most cases the capitalization is unchanged (the program may fail to recognize the context).

Compiler directives
The compiler directives {$XXX} can optionally be placed in upper case. This option is irreversible.

Numbers
The hexadecimal numbers and the exponent (E) of real numbers can optionally be placed in upper case. This option is irreversible.

File with user defined capitalization
This file contains identifiers and other word that are always written the same way.

The file is a text file with words on single lines. Example of the content of such file:

True
False
OnClick

Using this file

```
i := TRUE;
```

is always replaced with

```
i:= True;
```

The file can be created manual or automatically. If there are reserved words included in the file, the Capitalization of reserved words option is always overridden by this option.

The file can be used in different ways:

©2026 GExperts Team

Do not use file: Nothing happens
Add new words only: With the option Add new words only mode the file is filled automatically with all new identifiers that the program encounters. In this mode all words in the code remain unchanged.
Use file: In this mode the capitalization of all words is changed according to the notation in the capitalization file. If there are reserved words or standard directives included in the file, the notation of the file is always used.
Use file (except standard directivess: This option is the same as the previous one except that standard directives are only changed if these are used as identifiers: e.g.

```
PriVate
property PriVate: Boolean; read FPrivate write FPrivate;
```

changes in:

```
private
property Private: Boolean; read FPrivate write FPrivate;
```

if Private is included in the capitalization file and Standard directives are set to lower case
Add and use: This mode is a combination of Add new words mode and the Use file mode. New words are added automatically.
Add and use (except standard directivess: This mode is a combination of Add new words mode and the Use file (except standard directives) mode.

With the option Add new words modes the file is filled automatically with all new identifiers that the program encounters. In this mode all words remain unchanged. After editing the file, the Add new words mode should be unchecked to make it work.
Words that should not be changed in the text can best be commented out by adding * before them, e.g.:

*i
*nl
True
False

The words with a proceeding * are not changed in the text. In advantage over deleting is that the Add words mode doesn t add these outcommented words.

Edit button

Use this button to view and edit the file with the identifiers, see: Edit file dialog.

Align tab

Align simple end of line comments

Simple end of line comments are comments that are one line, with proceeding code. It can be aligned to a certain position (if possible). E.g.

```
var
 i:integer; //comm
 j{comm.}:integer; {comm.}
 tooooooooooooooo, long:integer; {comm.}
 {comm.}
```

changes in (position = 30):

```
var
```


©2026 GExperts Team

```
 i: integer; //comm
 j: {comm.} integer; {comm.}
 tooooooooooooooo, long: integer; {comm.}
 {comm.}
```

Align var/const

Var and const declarations can be aligned in the same way: e.g.

```
const
 i =0;
var
 j:integer;
 tooooooooooooooooooo, long:integer;
```

changes in (position = 30):

```
const
 i = 0;
var
 j : integer;
 tooooooooooooooooooooo, long: integer;
```

2.17 Grep Regular Expressions

Grep regular expressions allow you to formulate complex searches that are not possible using a basic text search. GExperts implements a subset of the Perl regular expression syntax, as described below.

Simple Matches

Any single character matches itself, unless it is a metacharacter with a special meaning, as described below.

A series of characters matches that series of characters in the target string. So, the regular expression search pattern "foo" would match the characters "foo" in the target string.

You can cause characters that normally function as metacharacters to be interpreted literal characters by 'escaping' them by preceding them with a single backslash ("\"). For instance, the metacharacter "^" matches beginning of string, but "\^" matches the literal character "^", "\\" matches "\" and so on.

Examples:

foobar	Matches string 'foobar'
\^FooBarPtr	Matches '^FooBarPtr'

Escape Sequences

Some special characters may be specified using escape sequences similar to those used in languages like C and Perl: "\n" matches a newline, "\t" a tab, etc. More generally, \xnn, where nn is a string of hexadecimal digits, matches the character whose ASCII value is nn. If you need to specify a unicode character, you can use '\x{nnnn}', where 'nnnn' are the hexadecimal digits for the unicode code point.

\xnn	ASCII Char with hex code nn
\x{nnnn}	Character with hex code nnnn (use nn for ASCII or nnnn for Unicode)
\t	Tab (same as \x09)

©2026 GExperts Team

\n	Newline (same as \x0a)
\r	Carriage return (same as \x0d)
\f	Form feed (same as \x0c)
\a	Alarm/bell (same as \x07)
\e	Escape (same as \x1b)

Examples:

foo\x20bar	Matches 'foo bar' (note space in the middle)
\tfoobar	Matches 'foobar' preceded by a tab

Character Classes

You can specify a character class (to denote any one of a set of characters), by enclosing a list of characters in square brackets [], and this will match any single character from the list.

If the first character after the "[" is "^", the class matches any character not in the list.

Examples:

foob[aeiou]r	Finds strings 'foobar', 'foober' etc. but not 'foobbr', 'foobcr' etc.
foob[^aeiou]r	Find strings 'foobbr', 'foobcr' etc. but not 'foobar', 'foober' etc.

Within a list, the "-" character can be used to specify a range, so that [a-z] represents any ASCII characters between "a" and "z", inclusive.

If you want "-" itself to be a member of a class, put it at the start or end of the list, or escape it with a backslash. If you want ']' you may place it at the start of list or escape it with a backslash.

Examples:

[-az]	Matches 'a', 'z' and '-'
[az-]	Matches 'a', 'z' and '-'
[a\-z]	Matches 'a', 'z' and '-'
[a-z]	Matches all 26 lower case ASCII characters from 'a' to 'z'
[\n-\x0D]	Matches any of the characters #10, #11, #12, and #13
[\d-t]	Matches any numeric digit, '-' or 't' (see "Predefined Classes" below)
[]-a]	Matches any character in the range of ']'..'a'.

Metacharacters - Line Separators

^	Start of line
$	End of line
\A	Start of text (equivalent to ^ since all searches are single-line)
\Z	End of text (equivalent to $ since all searches are single-line)
.	Any character (period)

Examples:

^foobar	Matches 'foobar' only if it is at the beginning of line
foobar$	Matches 'foobar' only if it is at the end of line

©2026 GExperts Team

^foobar$	Matches 'foobar' only if those are the only characters in the line
foob.r	Matches strings like 'foobar', 'foobbr', 'foob1r' and so on

The "." metacharacter matches any character, by default, but if you can switch off the /s modifier to stop '.' from matching embedded line separators.

Unicode line separators are supported (based on http://www.unicode.org/unicode/reports/tr18/: \x2028, \x2029, \x0B, \x0C, and \x85):

Metacharacters - Predefined Classes

\w	Matches any alphanumeric character (including "_")
\W	Matches any non-alphanumeric character
\d	Matches any numeric digit
\D	Matches any non-numeric character
\s	Matches any whitespace character (equivalent to [\t\n\r\f])
\S	Matches any non whitespace character

Note that you may use \w, \d, and \s within custom character classes.

Examples:

foob\dr	Matches strings like 'foob1r', "foob6r' and so on but not 'foobar', 'foobbr' and so on
foob[\w\s]r	Matches strings like 'foobar', 'foob r', 'foobbr' and so on but not 'foob1r', 'foob=r' and so on

Alphanumeric characters only include only the basic low-ASCII upper and lower case Latin alphabet, the digits 0-9, and '_'.

Metacharacters - Word Boundaries

\b	Match a word boundary
\B	Match a non-(word boundary)

A word boundary (\b) is a spot between two characters that has a \w on one side of it and a \W on the other side of it (in either order), counting the imaginary characters off the beginning and end of the string as matching a \W.

Metacharacters - Iterators

Any item of a regular expression may be followed by an iterator metacharacter. Using these metacharacters, you can specify the number of occurrences of the previous character, metacharacter, or subexpression.

*	Zero or more ("greedy"), similar to {0,}
+	Zero or more ("greedy"), similar to {1,}
?	Zero or one ("greedy"), similar to {0,1}
{n}	Exactly n times ("greedy")
{n,}	At least n times ("greedy")
{n,m}	At least n but not more than m times ("greedy")

©2026 GExperts Team

*?	Zero or more ("non-greedy"), similar to {0,}?
+?	One or more ("non-greedy"), similar to {1,}?
??	Zero or one ("non-greedy"), similar to {0,1}?
{n}?	Exactly n times ("non-greedy")
{n,}?	At least n times ("non-greedy")
{n,m}?	At least n but not more than m times ("non-greedy")

The digits in curly brackets of the form {n,m} specify the minimum number of times to match the item as "n" and the maximum as "m". The form {n} is equivalent to {n,n} and matches exactly n times. The form {n,} matches n or more times. There is no limit to the size of n or m, but large numbers will use up more memory and be slightly slower. If a curly bracket occurs in any other context, it is treated as a regular character.

"Greedy" means the iterator will match as many characters as possible, whereas "non-greedy" matches takes as few characters as possible. For example, 'b+' and 'b*' applied to the string 'abbbbc' matches 'bbbb', 'b+?' matches 'b', 'b*?' matches the empty string, 'b{2,3}?' matches 'bb', 'b{2,3}' matches 'bbb', etc. You can switch all iterators into a "non-greedy" mode using the modifier /g.

Examples:

foob.*r	Matches strings like 'foobar', 'foobalkjdflkj9r' and 'foobr'
foob.+r	Matches strings like 'foobar', 'foobalkjdflkj9r' but not 'foobr'
foob.?r	Matches strings like 'foobar', 'foobbr' and 'foobr' but not 'foobalkj9r'
fooba{2}r	Matches the string 'foobaar'
fooba{2,}r	Matches strings like 'foobaar', 'foobaaar', 'foobaaaar' etc.
fooba{2,3}r	Matches strings like 'foobaar', or 'foobaaar' but not 'foobaaaar'

Metacharacters - Alternatives

You can specify a series of alternatives for a pattern using a pipe "|" to separate them, so that fee|fie|foe will match any of "fee", "fie", or "foe" in the target string (as would f(e|i|o)e). The first alternative includes everything from the last pattern delimiter ("(", "[", or the beginning of the pattern) up to the first "|", and the last alternative contains everything from the last "|" to the next pattern delimiter. For this reason, it is common practice to include alternatives in parentheses, to minimize confusion about where they start and end.

Alternatives are tried from left to right, so the first alternative found for which the entire expression matches, is the one that is chosen. This means that alternatives are not necessarily greedy. For example, when matching foo|foot against "barefoot", only the "foo" part will match, as that is the first alternative tried, and it successfully matches the target string. This distinction is important when you are capturing matched text using parentheses. Also remember that "|" is interpreted as a literal within square brackets, so if you write [fee|fie|foe] You're really only matching [feio|].

Examples:

foo(bar|foo)	Matches strings 'foobar' or 'foofoo'

Metacharacters - Subexpressions

The bracketing construct using parenthesis (...) may also be used to define subexpressions so you can separate the matched text into several subparts. Subexpressions are numbered based on the left to right order of their opening parenthesis. The first subexpression has the number '1', etc. The whole

©2026 GExperts Team

expression match is given the number '0'. You can also use sub-expressions to perform search and replace ($0 or $& would match the whole expression, $1 would match the first subexpression, $2 the second, etc.

Examples:

(foobar){8,10}	Matches strings which contain 8, 9 or 10 instances of the 'foobar'
foob([0-9]|a+)r	Matches 'foob0r', 'foob1r' , 'foobar', 'foobaar', 'foobaar' etc.
(John) (Doe)	Matches the literal name "John Doe" with each word being a subexpression (John = $1, Doe = $2). You can then use the replacement string of "$2, $1" to replace that with "Doe, John".

Metacharacters - Backreferences

Metacharacters \1 through \9 are interpreted as backreferences, so in general, \<n> matches previously matched subexpression #<n>.

Examples:

(.)\1+	Matches 'aaaa' and 'cc' (a single character followed by itself, one or more times)
(.+)\1+	Also match 'abab' and '123123'
(["']?)(\d+)\1	Matches "'13" (in double quotes), or '4' (in single quotes) or 77 (without quotes) etc.

Modifiers

Modifiers are used to change the behaviour of the regular expression engine. Any of the modifiers may be embedded within the regular expression itself using the (?...) construct, as described below. The case sensitivity modifier also has an equivalent option on the grep search screen.

i	Do case-insensitive pattern matching (based on your system locale settings). In this mode, a lower case character can also match the equivalent upper case character, and vice versa.
m	Treat the string to search as multiple lines. That is, change "^" and "$" from matching at only the very start or end of the string to the start or end of any line anywhere within the string. This modifier won't generally be useful, because the grep search operates a single line at a time.
s	Treat the string to search as a single line. This changes "." to match any character (even a line separator) which it normally would not match.
g	Non standard modifier used to switch all of the subsequent operators into non-greedy mode. So, if this modifier /g is specified, '+' works like '+?', '*'works like '*?' and so on. Greedy mode is enabled by default.
x	Allow whitespace and comments (see explanation below), to enhance the expression's legibility.
r	Non-standard modifier used to simplify Russian character range entry. If this is turned on, the range à-ÿ automatically also includes the Russian letter '¸', À-ß includes '¨', and à-ß includes all Russian symbols. This modifier is off by default.

©2026 GExperts Team

The modifier /x needs a little more explanation. It allows the regular expression to contain whitespace that is neither escaped nor within a character class. You can use this to break up your regular expression into more readable parts. The # character is also treated as a metacharacter introducing a line comment, for example:

```
(
(arc) # comment 1
  |   # You can use spaces to format your search expression, and they are ignored
(efg) # comment 2
)
```

In this mode, if you want literal whitespace or # characters in the pattern (outside a character class, where they are unaffected by /x), you will either have to escape them or encode them using hex references.

Perl Extensions To Modifiers

(?imsxr-imsxr)
You can use some Perl extensions to change the modifiers on the fly. If this construction is inlined into a subexpression, it only effects that subexpression.

Examples:

(?i)Frog-Soup	Matches 'Frog-soup' and 'Frog-Soup'
(?i)Frog-(?-i)Soup	Matches 'Frog-Soup' but not 'Frog-soup'
(?i)(Frog-)?Soup	Matches 'Frog-soup' and 'frog-soup'
((?i)Frog-)?Soup	Matches 'frog-Soup', but not 'frog-soup'

Perl Extensions for Comments

(?#text)
Text inside a comment like the above is ignored. The comment ends at the first ")", so there is no way to put a literal ")" inside the comment.

Source Code

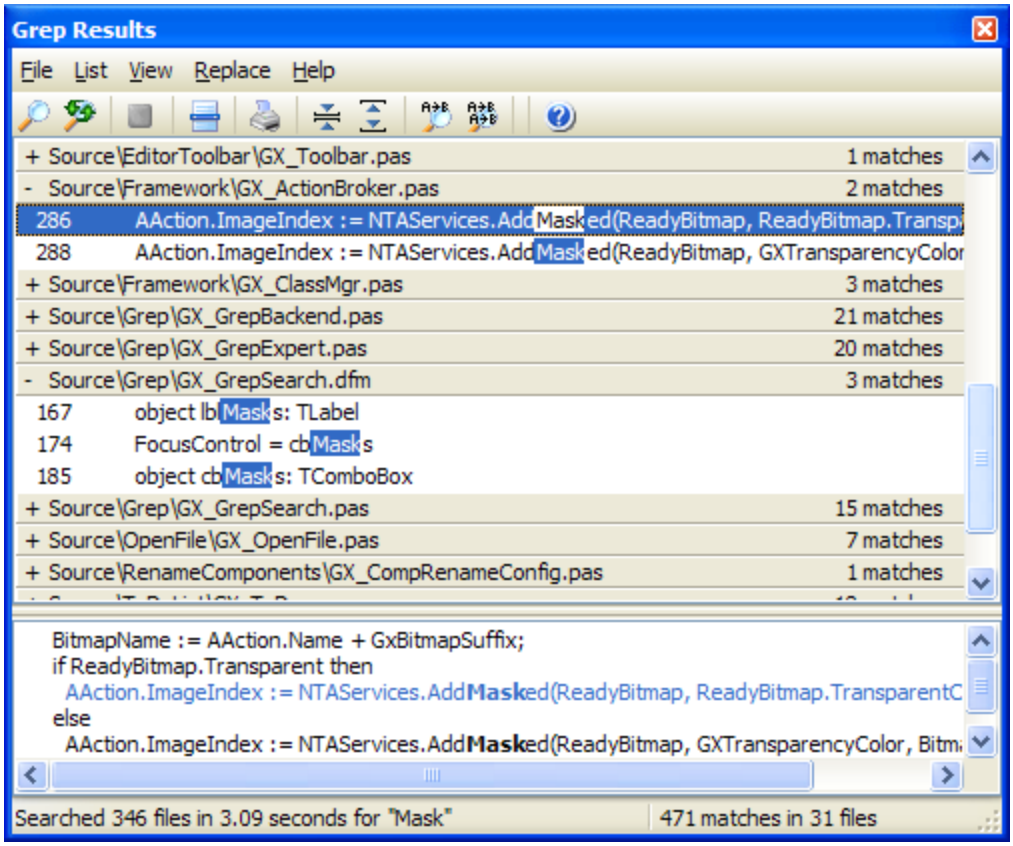
Note: The free TRegExpr Delphi library is used to implement regular expression searching.

2.18 Grep Results

The Grep Results window is where the results of a Grep Search are shown. It also provides an interface for multi-file search and replace on matches. The window uses a folding display of matches to allow you

©2026 GExperts Team

to easily locate a particular match. This expert supports IDE docking. An example of this window appears below.



The results window displays all files which contained one or more matches for the search term. Under each file, a list of matches for that particular file can be shown. To expand or contract a file's matches, click on the filename, press enter, or use the '+' and '-' keys. When a specific match line is selected, the window can show any number of lines of match context using the Show Match Context and Set Context Lines menu items. Note that the match context might not be accurate if the file has changed since the search.

The number to the left of each match is the line number where the match was found. The results list highlights the matching characters in each entry to indicate where the match occurred.

To jump to a match in the IDE editor, double click the desired line, press enter, or use the Goto button. You can control whether the result is positioned at the top of the IDE editor or in the middle through an option in under Vies,Options or via the GExperts Configuration dialog in the section for the Grep Results expert.

To start a new search, click the Search button and the Grep Search dialog will appear. As the search progresses, the new search button will be disabled and the abort button will be enabled to cancel the current search. Once a search is completed, the results window displays the number of files searched, the search time, and the total number of matches.

©2026 GExperts Team

You can expand all items in the list by clicking the expand button in the toolbar. Similarly, clicking the contract button in the tool bar will contract all result items. The entire match list can be coped to the clipboard or saved to a file using the items on the File menu.

Using the Options dialog, define how selected matches are shown in the editor, whether to expand all matches by default, the number of context lines to show, and the list and context fonts/colors.

Search and Replace on Matches
You can do a search and replace operation on all of the matches in the list or only the selected file/match. When you choose one of those options, a dialog appears prompting for the string to use in place of the matched text. The replace operation supports regular expression subexpressions. See the section on Subexpressions for more details. Note that forms that are currently open can not have their text replaced, due to limitations in the IDE. Please close all forms before trying to replace text in them.

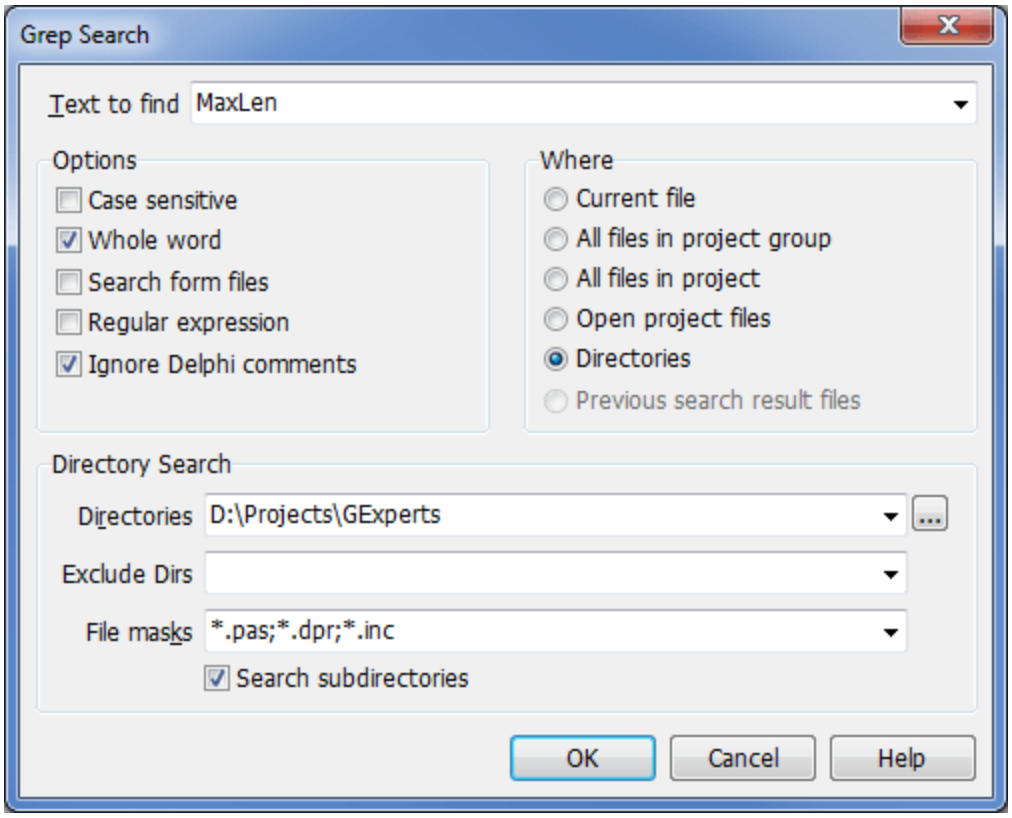
Tip: You can drag and drop a file from the results list onto a running program like notepad, or you can drag it onto a program icon or shortcut to execute the program with the selected file. When running the standalone GExpertsGrep.exe program, double clicking a file will open the associated program for that file type.

2.19 Grep Search

GExperts incorporates a powerful search facility that enables you to quickly locate text strings in files (ANSI/ASCII, UTF-8/UTF-16 unicode, and dfm/nfm/xfm form files). Using Grep, you can search the current file, all open project files, all files in your project or project group, or all files in a directory (optionally including sub-directories). The grep search is also available outside the IDE by executing the standalone GExpertsGrep.exe application.

To begin a search, select Grep Search from the GExperts menu. A dialog will appear like the one below where you can enter your search criteria. Note that if you have a string highlighted in the code editor when calling up this dialog, it will be used as the default search string.

©2026 GExperts Team



The various options in the search dialog are as follows:

Text to Find: The text or regular expression to search for

Options:

Case sensitive:	Search is case sensitive (a and A are treated as different characters)
Whole word:	Return matches that are whole words (delimited by whitespace or punctuation such as "().,<>-{}!@#$") Note that 0-9 and _ are treated as part of a word.
Search form files:	Include form files in the search (GExperts can search inside binary format form files)
Regular expression:	The text to find is a regular expression
Ignore Delphi comments:	Don't look for matches inside Delphi source file comments

Where (search scope):

Current file:	Only the file that is currently active for editing
All files in project:	All files that are part of the current project. Note that this only includes the files the IDE reports as directly in the project, or those listed in the View Unit(s) dialog. As an example, for Delphi applications, the file must be present in the .dpr uses clause with a proper 'in' specifier.
All files in project group:	All files that are part of the current project group

©2026 GExperts Team

Open project files:	All project files that are currently open in a source code editor
Directories:	All files specified by the Directory Search options (see below)
Previous search result files:	Within files that had matches from the previous search (only enabled if a previous search found at least one match)

Directory Search:
Note: This portion of the dialog is only enabled if "Search in directories" is selected.

Directories:	A semicolon separated list of directories to search
"..." Button:	Allows browsing for a search directory
Exclude Dirs	A semicolon separated list of directories to exclude from the search (the exclusion is done via a substring match on the full directory and file name)
File masks:	Limits the search to a semicolon separated list of file extensions
Include subdirectories:	Enables recursive searching of the search directories

Once you have entered the search criteria, click the OK button or press enter to initiate the search. As the search progresses, results will be shown in the Grep Results window. From the results window, you can also perform multi-file search/replace on the matches.

The GExperts Configuration dialog allows setting the Grep Search option to use the current editor identifier as the default search string, if no block is selected.

2.20 Hide/Show Non-Visual Components

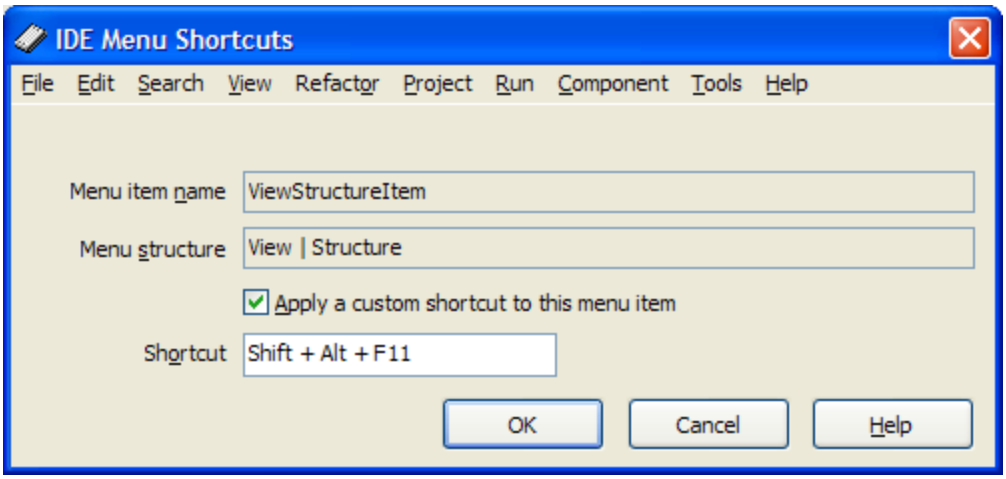
This allows you to temporarily toggle the visibility of non-visual components on VCL TCustomForm designers. The effect is only temporary until you close the form, but should allow you to visualize the form without any non-visual components present. Note that the non-visual components are not actually removed from the form, so you can still click on and select them, if you know where they are located.

Note: This tool only works for Windows VCL TCustomForm and TFrame designers (not data modules, CLX forms, .NET forms, etc.).

2.21 IDE Menu Shortcuts

This expert enables you to configure any of the IDE menu shortcuts. To change the shortcut of a menu item, select the appropriate menu item, click "Use Shortcut Expert for this menu item" and type the desired shortcut into the Shortcut edit box. If you find your chosen shortcut does not work, it is likely because the shortcut is reserved by the code editor or another menu item, and you'll need to select a different, unused shortcut. This expert can not change the shortcuts for the GExperts menu items; for this purpose, use the GExperts Configuration dialog.

©2026 GExperts Team



2.22 Macro Library

The IDE's code editor has a built-in keyboard macro facility, but there is a limitation that it can only store a single macro and that macro is lost when the IDE exits. The GExperts Macro Library allows you to save and later playback an unlimited number of keyboard macros in future IDE sessions.

To begin recording a macro, press the record toolbar button. Then, begin typing your editor keystrokes. When you are done recording the macro, use the record button to stop recording. At that point, the Macro Library windows should add your new macro to the window with the current date and time as the timestamp. You can then give the macro a name and a long description. All of the listed macros will be automatically saved to an XML file called MacroLibrary.xml. Delphi 7 and later also have the ability to automatically add all editor recorded macros to the library, without using the record toolbar button (you can start/stop recording by pressing Shift+Ctrl+R with the default editor keymapping). This feature can be turned off using the Pause button in the toolbar.

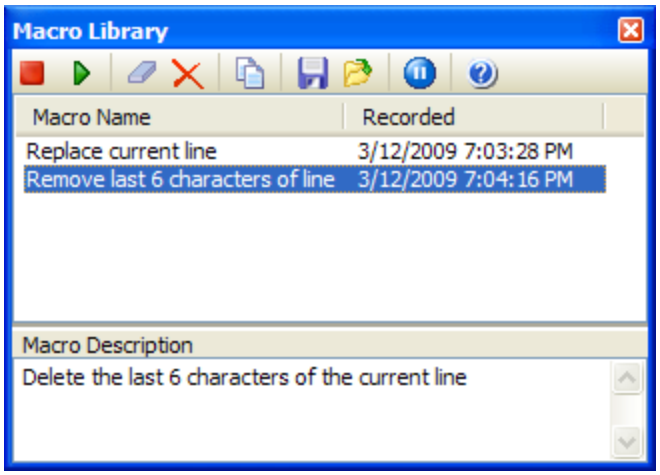
To playback a macro, first place the editor cursor in the desired location, and then press the playback button, double-click the macro, or use the copy toolbar button and press Shift+Ctrl+P to execute the macro as many times as you need.

Once you are done using a macro, you can delete it using the delete button or clear all of the macros at once.

If you want to share individual macros with other systems, you can use the save and load buttons to save or load individual macros to a file. To share all of your macros, you can exit the IDE and then copy the entire MacroLibrary.xml to another computer.

Note: It is the IDE that implements the actual recording and playback of macros. You should find that it properly records and plays back most standard editor keystrokes (arrow/movement keys, letters/numbers/symbols, block selection) and some types of menu item selections, but certain other types of IDE actions (creating new event handlers from the designer or object inspector , etc.) can not be recorded or played back.

©2026 GExperts Team



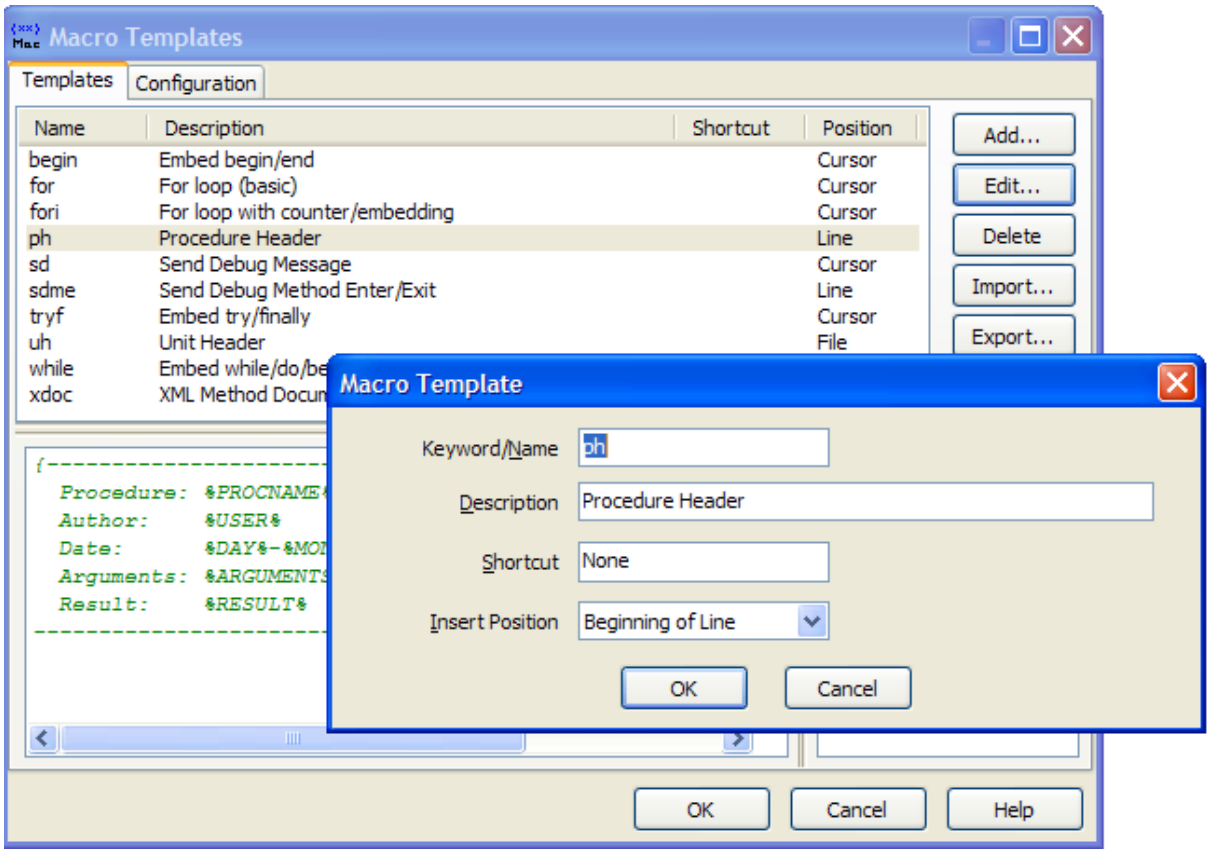
2.23 Macro Templates

Macro Templates is an editor expert that allows text snippets to be quickly copied into the code editor and those snippets can contain any number of %-delimited macros that can insert the current date/time, user name, procedure name, unit name, clipboard contents, etc. For a full list, see the Substitution Macros topic. Note that some of the macros are only supported for Delphi source code. As an example, the following template would produce a Delphi unit header comment:

```
// Unit:    %UNIT%
// Purpose: |
// Author:  %USER%
// Date:    %DATETIME%
// Version: 1.0
```

For each macro template you define, you can configure the position where template will be inserted. It can be the current cursor position, the start of the unit, or the beginning of the current line in the editor. You can also configure a shortcut to activate your most frequently used templates. The "|" (pipe) character can be used inside your templates to define the desired cursor position after a template is inserted. The screenshot below shows the configuration interface for Macro Templates:

©2026 GExperts Team



You can also define a list of units that should be present in the interface or implementation for each template to function. If the unit you define is not already used by the unit where the template is being inserted, the defined units are added to the appropriate uses clause. Right-click the implementation/interface list to add/delete items from the unit lists. Note that this feature is only supported in Delphi source code.

On the Configuration tab, you can enter your name, initials, and an auto-incremented sequence number to use in generated templates (you could use it to create unique error numbers, etc.). The values you type there can be substituted into your templates using the Substitution Macros (% PROGRAMMERNAME%, etc.). You can also configure your templates to be able to expand after pressing space and a short delay.

One time-saving method of using the macro templates is to create macros using %SELECTION% that allow you to embed the selected lines inside a begin/end, try/finally, try/except, region, or any other type of block.

There are several methods available to insert templates into the editor:

- Press the shortcut key you have assigned to the specific template you want to insert.
- Type the template name in the editor and press the shortcut assigned to the Macro Templates expert. If your typed name matches a template name, the template is inserted. If there is not a

©2026 GExperts Team

match, or no name was typed, the dialog described below is shown. The template name you typed in the editor will only be deleted if it matches the inserted template name.
- Press the shortcut assigned to the Macro Templates expert and choose the template from the list in the resulting dialog. The dialog supports incremental searching through your templates.
- Enable the option to expand templates when pressing space in the Configuration tab. Type the template name and press space. After your configured delay, the template will be inserted into the editor and the template name you typed will be automatically deleted.

Note: Starting with BDS 2006, you may prefer to use the IDE's built-in live templates, which have added features such as ordered code insert points and custom scripting within the templates.

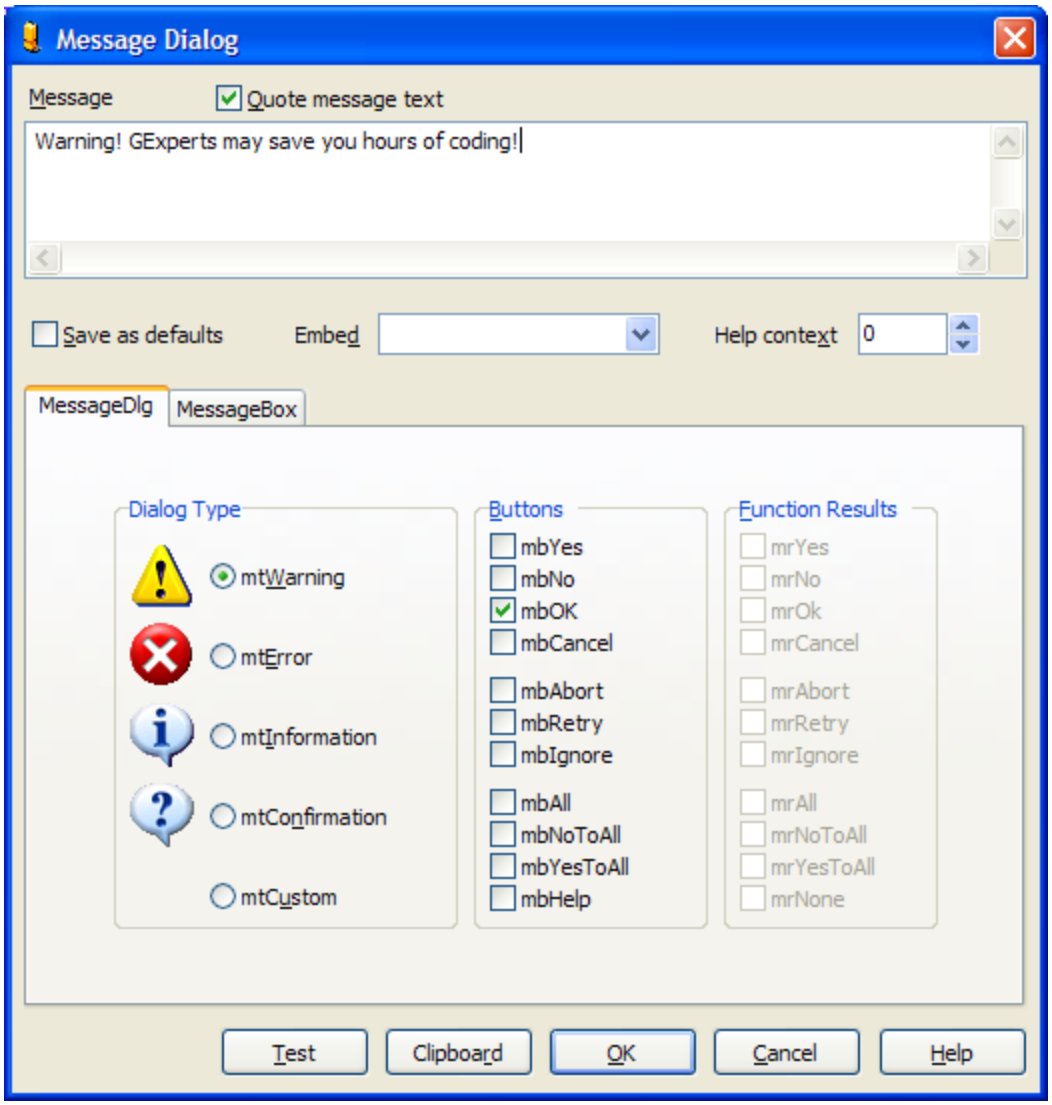
Note: Please see C++ and C# Compatibility for some limitations with those file types.

2.24 Message Dialog

The Message Dialog expert provides a convenient way to enter MessageDlg or Windows MessageBox function calls code into your source code. To use this expert, fill out the fields shown and press the OK button. The expert will then generate the required function call and insert it into the editor. If your message contains embedded carriage returns, those will be reflected in the message shown. You can test the execution of your dialog using the Test button or copy the code to the clipboard using the Clipboard button.

This expert can generate an if, negative if, or case/switch statement in Delphi or C++ code that tests the user's response to the dialog box against the responses marked in the Function Results box. To set your the current options as the default, check the "Save as defaults" checkbox before pressing OK. See the VCL and Windows API documentation for help with the options available for each type of message dialog.

©2026 GExperts Team



Tip: You can press Ctrl+Enter while typing in the message memo to confirm the dialog (click OK).

2.25 Open File

The Open File tool allows you to search for and quickly open any type of file. The filter string you type will cause the file list to only show files containing that filter. The type dropdown allows you to select the groups of files to show in the lists.

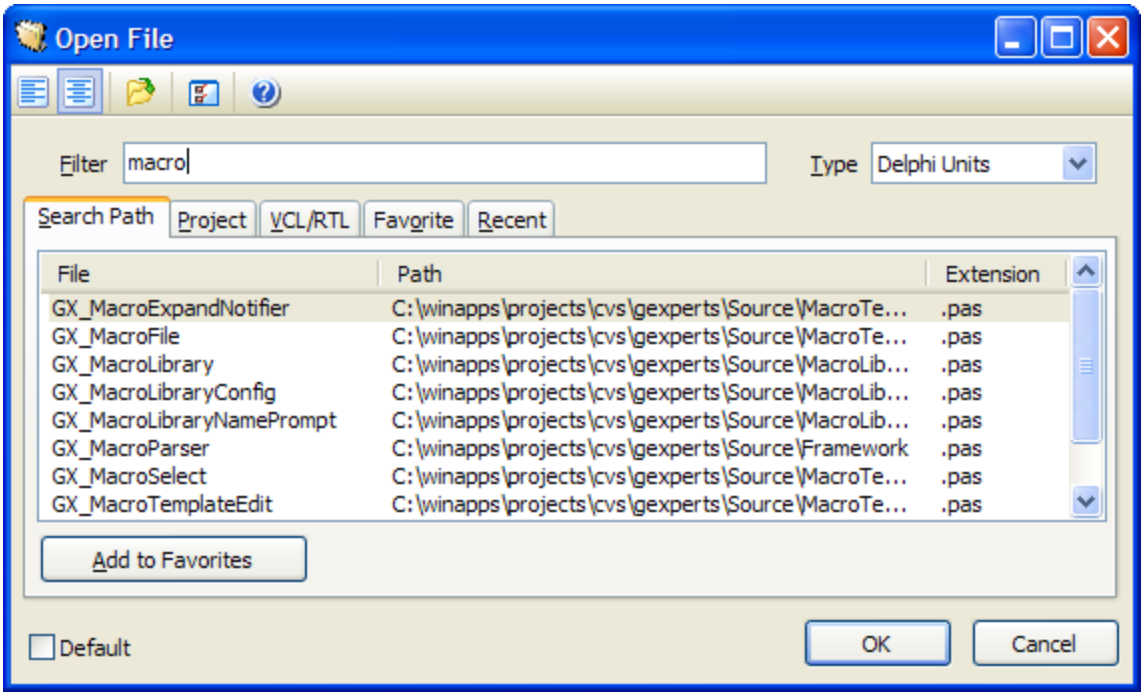
There are several tabs that show different groups of files:

- **Search Path**: Files on the project search path and the IDE-global search path

©2026 GExperts Team

- **Project**: Files in the project
- **VCL/RTL**: Files in the IDE's Source directory
- **Favorite**: Files you have specifically indicated as a favorite file
- **Recent**: Files that have been opened in the IDE recently

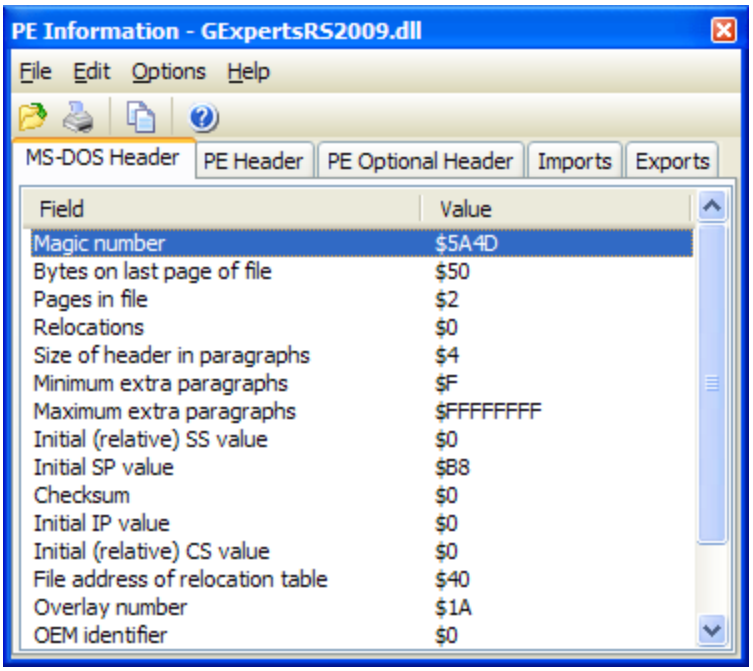
Using the Options button, you can define custom file groups to use when filtering the file list using the Type dropdown. You can also configure custom directories to search and for files to add to the search path tab when that file group is selected. You can also choose to override the IDE's Open Project, Unit List, and Form list dialogs to bring up the Open File tool with a specific file group pre-selected.



2.26 PE Information

The PE Information expert enables you to examine the contents of a Portable Executable (PE) file. Almost all Win32 EXEs and DLLs are PE format files. Many other files are also PE files, such as Borland/CodeGear packages (DPL/BPL), Active X controls (OCX), etc. This expert supports IDE docking and drag and drop of files from Windows Explorer.

©2026 GExperts Team



To view the contents of a PE format file, click the Open button and select the file or drag and drop a file from your file manager. There are five tabs in the expert as follows:

- **MS-DOS Header:** All windows programs contain the MS-DOS header, sometimes referred to as the MS-DOS stub.
- **PE Header:** The portable executable header contains data such as what machine the binary is targeted for (Intel 0x86, AMD x64, Alpha, MIPS, etc.).
- **PE Optional Header:** Contrary to its name, this header is required in PE files. It contains information regarding the subsystem required, the version number of the OS required, etc.
- **Imports:** This section contains a list of all files required by this file to run, including Borland packages. Selecting a file will cause a list of all functions imported from the selected file to appear.
- **Exports:** Exported functions are the ones that are made publicly available for other applications call. This tab is mostly of use when examining DLLs or packages.

Note: This tool can detect 64-bit binaries, but will not be able to read all of the data from them (imports, exports, etc.).

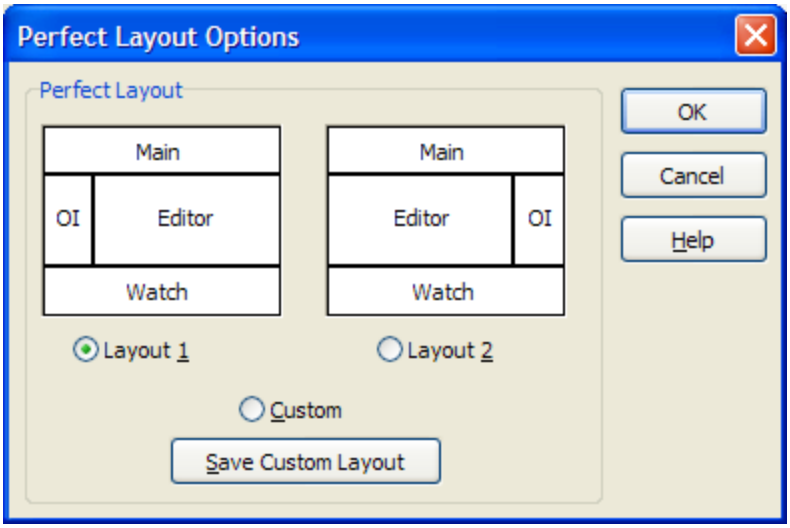
©2026 GExperts Team

2.27 Perfect Layout

The Perfect Layout expert repositions the IDE's windows so that they are located exactly where you want them. This expert supports two pre-defined layouts, which can be selected via the GExperts Configuration dialog.

You can also create a custom layout by positioning the IDE windows as you want them, and using the Save Custom Layout button. You can change your custom layout at any time by repeating this process.

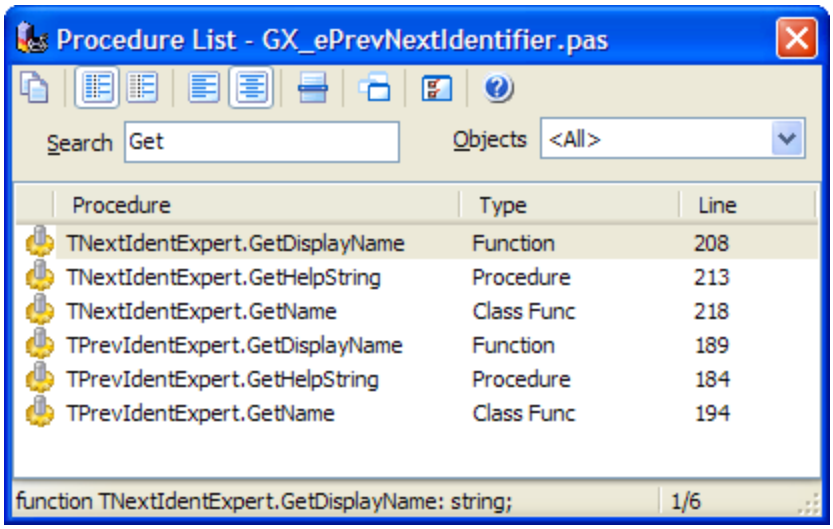
Note: This expert is deprecated and does not work well with docked window configurations. We recommend you use the named desktop saving features of your IDE instead.



2.28 Procedure List

The procedure list enables you to view a list of Delphi or C++ procedures in the current unit and quickly jump to the implementation of a given procedure. The procedure list window is pictured below:

©2026 GExperts Team



To search for a procedure, start typing in the Search edit box. As you type, the characters will appear at the top next to the word **Search**. To jump to a selected procedure, double click or highlight it and press enter.

Two search modes are provided. In the first search mode (match only from the start), searches are conducted only on the beginning of the procedure name (after an optional object reference). In the second search mode (match anywhere), the search string can match at any point in the procedure name. For example, if you had two functions, **TMyObject.Assign** and **TMyObject.AssignWidget**, searching for "assign" would return both methods in either mode, whereas searching for widget would return neither when searching from the start, while **TMyObject.AssignWidget** would appear if searching for a match at any point.

Using the Objects combobox on the right, you can filter the procedure list to display all procedures, only those without an associated object, or only those associated with a specific object. The Copy button will copy all of the visible procedure details to the clipboard.

Using the options button, you can configure several items:

- Optionally show the selected procedure's source code in a pane inside the procedure list
- Determine whether the search filter matches within class names as well as procedure names
- Configure the fonts used for display
- Select whether or not the class name of any methods are displayed

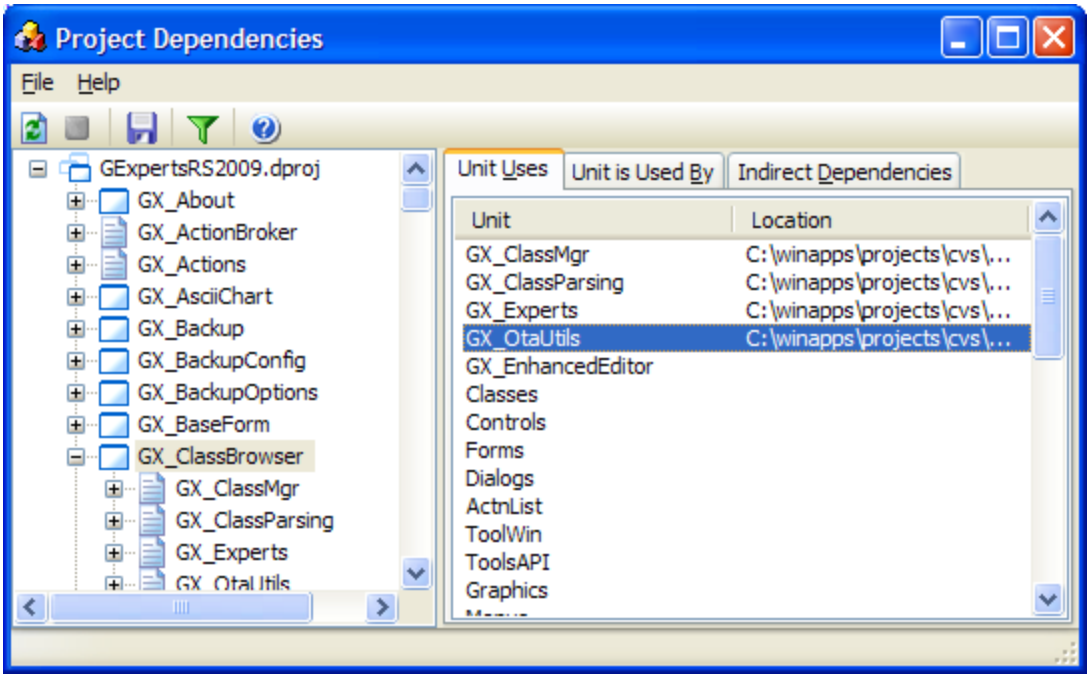
2.29 Project Dependencies

The project dependency expert enables you to see what units a particular unit uses, and in turn what units use a particular unit. When this expert is activated, it parses all of the current project's source code for uses clauses and builds up a list of dependencies. To view the dependency information for a

©2026 GExperts Team

particular unit, click on it in the left pane. The right pane will contain the dependency information. Indirect dependencies are units that are used by used units of a particular unit.

You can refresh the dependency information at any time by clicking the refresh button on the toolbar and you can sort the file listing by clicking on the column headers.



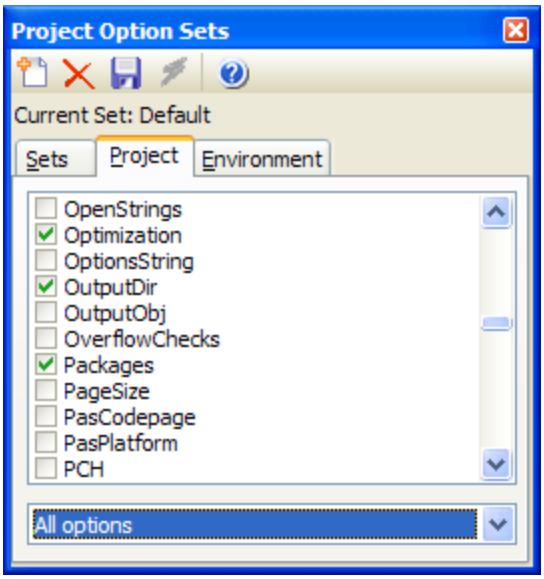
2.30 Project Option Sets

With this expert, you can save off sets of project options and recall them at a later time. Along with the project options, a few environment options can be saved as well. This expert supports IDE docking.

To save a given set of project options, you should first execute the Project Option Sets expert and click Add to name the new set (such as "Debug" or "Release"). Next, open the IDE's Project Options dialog and verify that the settings you wish to save are set correctly. Note that you can open the Project Options dialog from the right-click menu of settings listbox. Let the IDE store those settings by confirming the Project Options dialog. Next, check the settings on the Project Settings or Environment Settings tabs that you want to be saved. Settings you don't check here are not affected when you load the stored settings later. Finally, use the Save button to save the settings. Your saved option sets are stored in an XML file called ProjectOptionSets-*.xml. The storage location of this file is defined in the GExperts Configuration dialog. To load a previously saved set of options, select the set name in the list and use the Apply button.

Note: The option name lists match those used internally by the IDE, so it may take some searching to determine which option name matches a given setting in the project options dialog. The listbox provides some popup hints to help you, and you can use the combobox below the list to filter the available options based on certain categories.

©2026 GExperts Team



2.31 Property Value Constants

For features such as the Replace Components constant property specifiers and the Rename Components "Other Properties" default values, you can specify constant values as strings that get converted into property values at runtime. Here are the basic types of properties that can be set using these constant specifiers:

- **Ordinals**: Integers, Int64, Extended, and other ordinal values can be specified by specifying the integer or float value, such as '557' or '5.57'.
- **Strings**: Short strings, long strings, unicode strings, and wide string properties can be set just as expected, by entering normal strings such as 'Value', 'OK', etc. In some cases, you can specify an empty string using two single quotes, like this: ''.
- **Enums**: You can set an enum by specifying either the ordinal value or string name of the enum value such as 'mbNo' or just '1'.
- **Sets**: Sets modifications can be entered such that all of the members are defined using syntax such as '[akLeft,akTop]' or 'akLeft,akTop'. To remove a set member, use a minus sign, similar to '-akBottom', and to add members do the same with a plus sign: '+akBottom'.
- **Variants**: Specify the value using either a string notation or an ordinal/float value, similar to the above.
- **Boolean**: Specify either 'True', '1', 'False', or '0'.

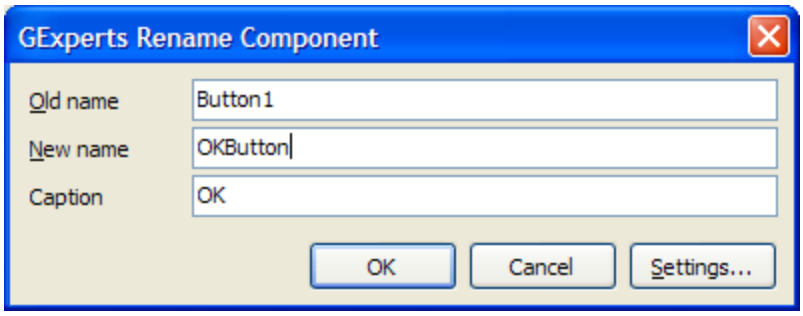
Other published property types such as tkMethod, tkInterface, tkDynArray, tkClass, tkArray, tkRecord, and tkUnknown are not supported.

Note: Some IDE versions have a bug that does not allow you to set boolean and enum properties on VCL.NET components. WinForms components are also not supported.

©2026 GExperts Team

2.32 Rename Components

The Rename Components tool allows you to define custom rename rules for various component classes and have those rules executed when components are added to a form at design time. The renaming can be completely automatic, or you can be prompted for each component to customize the suggested name.



In the configuration screen, you can click Add to add a new rule, Remove to remove the selected rule, or Defaults to add some basic default rules to the list.

The rename rule consists of a default name for the component, and (optionally) any of the following patterns:

%d Causes the automatic renaming to insert the smallest positive integer possible to create a unique component name on the form

| Defines the default position of the caret in the interactive rename dialog

|foo| Causes the interactive rename dialog to select the text 'foo' by default

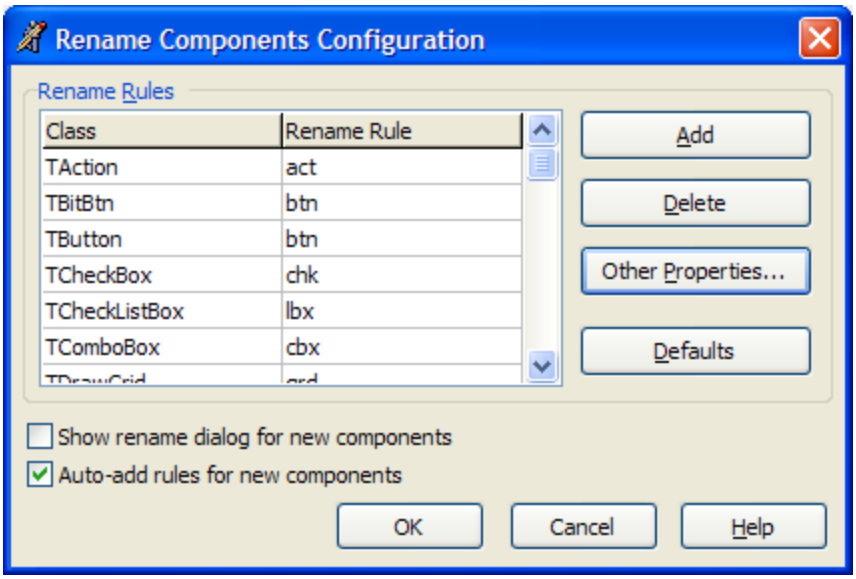
The "Other Properties" button's dialog allows you to enter any extra properties you want to be able to edit when the interactive rename dialog pops up. Those properties will be shown below the name property entry area in the rename dialog. For example, you might use this to set the Caption of a Button while you are renaming or, or to set the checked status of a checkbox. You need to enter the other properties one per line in the resulting dialog. If you want any of the properties to have a default value different from the component's default, add an equals sign and then the default value after the property name. If you have chosen not to show the interactive rename dialog, you can still specify "Other Properties" with defaults and those defaults will apply to any new component you place on the form. For example, for TButton, you could define this set of properties in the other properties list to set some defaults and also be able to edit the default Hint property in the rename dialog:

```
Caption=OK
Hint
Enabled=False
```

Options:

- **Show rename dialog for new components**: Show the interactive rename dialog for all newly added components. Without this checked, the renaming is done automatically without user interaction.
- **Auto-add rules for new components**: Each time a component is added to a form, a default rename rule will be added to the list so you can customize it. Note that you need to create one rename rule manually to active this expert first before it will begin creating rules for you.

©2026 GExperts Team



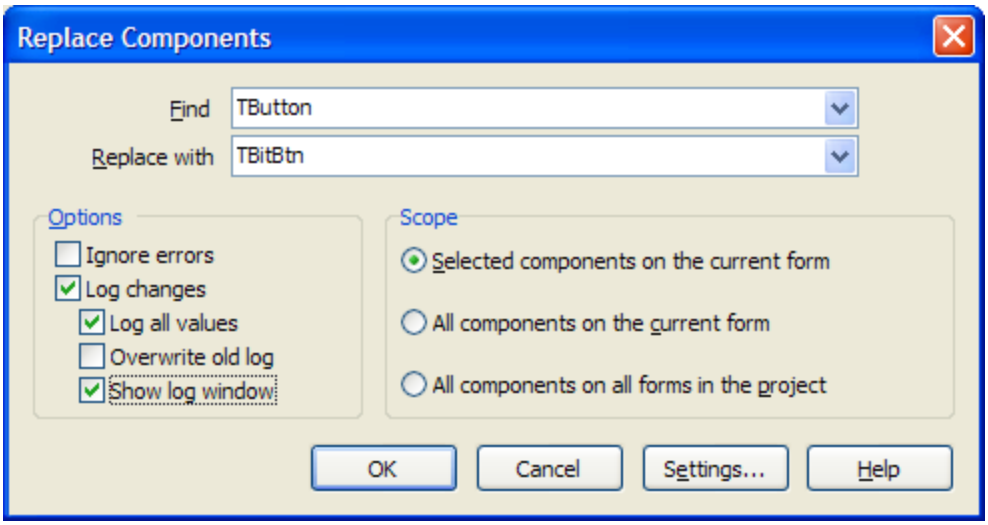
Note: This tool will only work for VCL and VCL.NET projects.

2.33 Replace Components

This expert enables you to easily replace components on a form or in a whole project. For example, have you ever wanted to replace all TBitBtn components in a project with TButton components? To use this expert, first choose the component you want to replace, followed by the component you want to replace it with. Then click OK to have the expert perform its magic. If you select the component to replace in the IDE before calling up this expert, you won't have to select it in this dialog box.

This expert does everything it can to preserve and map properties and can handle all of the basic types such as integers, strings, and enums. It can also handle many more complex properties such as component references, TStrings, TFields, properties that support calling TPersistent.Assign between each other (such as TFont/TBitmap), and conversions between properties of slightly differing but largely compatible types such as TStrings to a string and conversions between various ordinal types, within the limitations of the RTTI GetPropValue and SetPropValue functions. You will generally be able to map from ordinals to floats to strings, for example, but mapping from strings to integers might raise errors if the string does not represent an integer.

©2026 GExperts Team



Options

- **Ignore Errors**: Continue attempting to replace components, even if some property conversions or component creations fail. Otherwise, when errors happen, you will be presented with a dialog allowing you to ignore the error, ignore all errors, or abort the conversion.
- **Log changes**: Log the progress of the replacement to a CSV file named ReplaceComp.log (and optionally to the screen as well)
- **Log all values**: Log both common/direct property assignments, in addition to custom mapping assignments
- **Overwrite old log**: Overwrite any previous replacement log files, instead of appending to the existing log
- **Show log window**: Show the GUI log window after the conversion is done. This will allow you to browse the log, review any errors, and export the log to XML/CSV. You can also select individual log items to be copied to the clipboard.

Scope: Look for components to replace only in the currently selected components, the current form, or anywhere in the active project.

Custom Mapping Rule Details

If you access the configuration for this expert in the main GExperts configuration screen, you can configure custom mapping rules between component classes where the source and destination property names or property types are not direct matches.

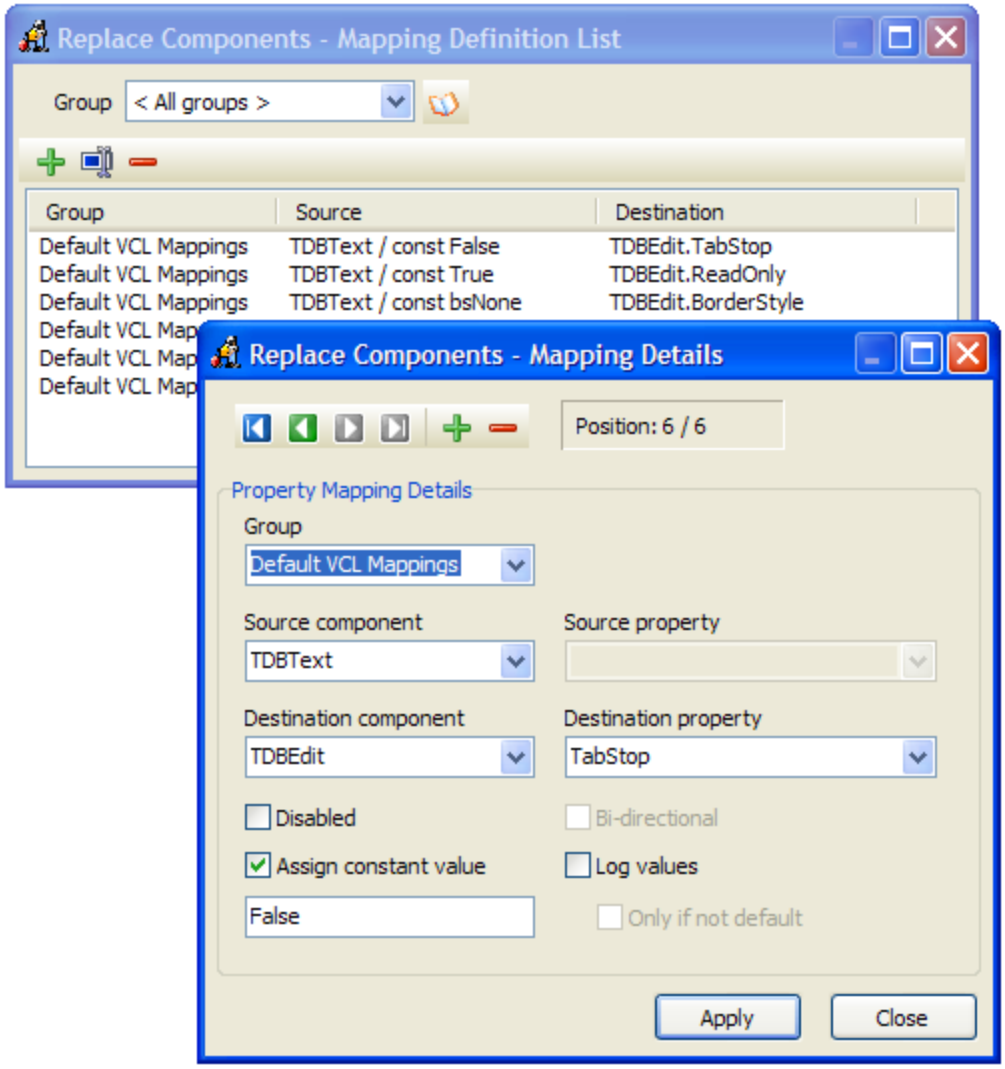
- **Group**: Each custom property conversion is a member of a specific group. The groups are listed in the Mapping Definition list screen in the Group combobox and the Group column of the custom mapping list. Groups can be imported and exported and then shared using XML files via the "Open group list" button.
- **Source component**: Select the component class being replaced for the defined mapping rule to be activated
- **Source property**: The replaced component property name to map to the defined destination property. For both the source and destination property names, if a subproperty descends from TPersistent, you

©2026 GExperts Team

can specify subproperties, such as mapping strings like Font.Name to a destination FontName property.

- **Destination component**: Select the component class that must be selected to replace the source component class for this mapping rule to be activated
- **Destination property**: Select the property of the destination component class that maps to the source property
- **Disabled**: Disable copying of the source property to the destination property. This is useful when two components share a same-named property, but the properties do not perform the same function or are of incompatible types.
- **Bi-directional**: Treat the source to destination mapping rule as also applying from the destination to the source (for example, to map TEdit.Text to TMemo.Lines, as well as TMemo.Lines to TEdit.Text)
- **Log values**: Log the property conversion to the log
- **Only if not default**: Only log the conversion if the source property is not set to the component's default value. This option is useful to log changed source properties where there is no direct equivalent destination property. You can then scan the log to determine if any manual/code changes must be done to complete a full conversion.
- **Assign constant value**: Assign a constant value to the destination property rather than using the source property value. This is useful to convert properties where there is no direct equivalent in the source component, but the destination component's property should not remain at the default value. With this option selected, the source property is no longer used for the conversion, but is instead replaced by the value typed as the constant value.

©2026 GExperts Team



Note: Several types of more obscure property conversions are not supported such as mapping set membership to a Boolean, mapping between non-name compatible sets or enums, mapping between TPersistent properties where calling Assign does not work (for example converting TBitBtn.Glyph to TImage.Picture), and conversions such as TTabbedNotebook to TPageControl, where custom coding/mappings would be required to create new tabs and move components to them. Also, note that you can not replace individual TField descendent objects, because the VCL prevents more than one component with the same FieldName on each dataset, and the Replace Components tool requires both objects to be present at once, to copy the old properties to the new object.

©2026 GExperts Team

2.34 Set Component Properties

This tool waits in the background until you compile a project. It then scans the current project's forms to check for components with certain properties and changes those properties to a defined value. This tool is useful to deactivate datasets or database connections before you compile your applications, but it can be used for any similar situations as well. To activate the scanning, enable the checkbox next to this expert in the GExperts Configuration screen.

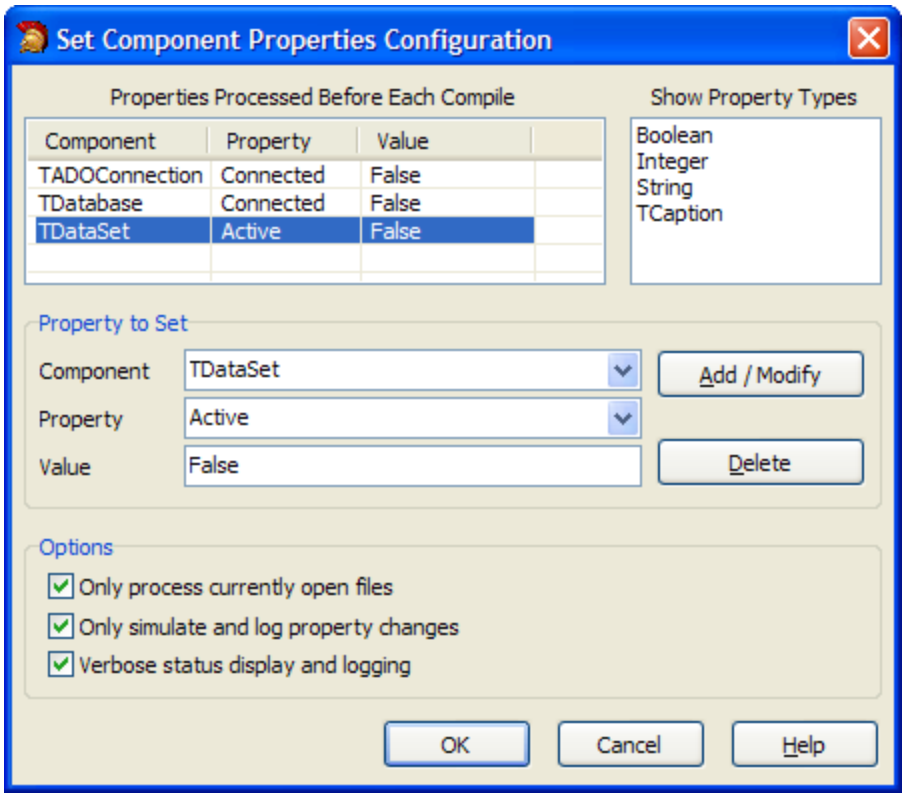
To add a new component class and property to search for, enter those values in the Property to Set area. Then type in the desired value of the property as the Value text (see Property Value Constants for details). Finally, click Add to add that property rule to the stored list of rules. To edit an existing property rule, select it in the list, make your changes, and click Modify. To delete a property rule, select the property in the list and hit Delete.

The property types that are listed in the property dropdown can be edited using the context menu in the Property Types area. You can add and remove property types to show only those you are interested in seeing.

Options:

- **Only process currently open files**: Only scan project forms that are currently open in the IDE. It is recommended that you keep this option enabled because it results in much faster scanning, because GExperts will not need to open a potentially large number of forms into the IDE for larger projects.
- **Only simulate and log property changes**: Do not actually change the value of any properties - just log the changes that would be made. This is useful for testing new property rules, to be sure they have the intended effect.
- **Verbose status display and logging**: This enables detailed logging of each property that is changed and enables the display of a small status window while the forms are being scanned for property changes. The log details are output to the IDE's message view. Note that important messages such as non-existent properties or errors while setting a property are always reported regardless of this setting.

©2026 GExperts Team



2.35 Set FocusControl

This tool enables you to quickly set the FocusControl property of labels on your VCL forms. There are two ways to use this tool:

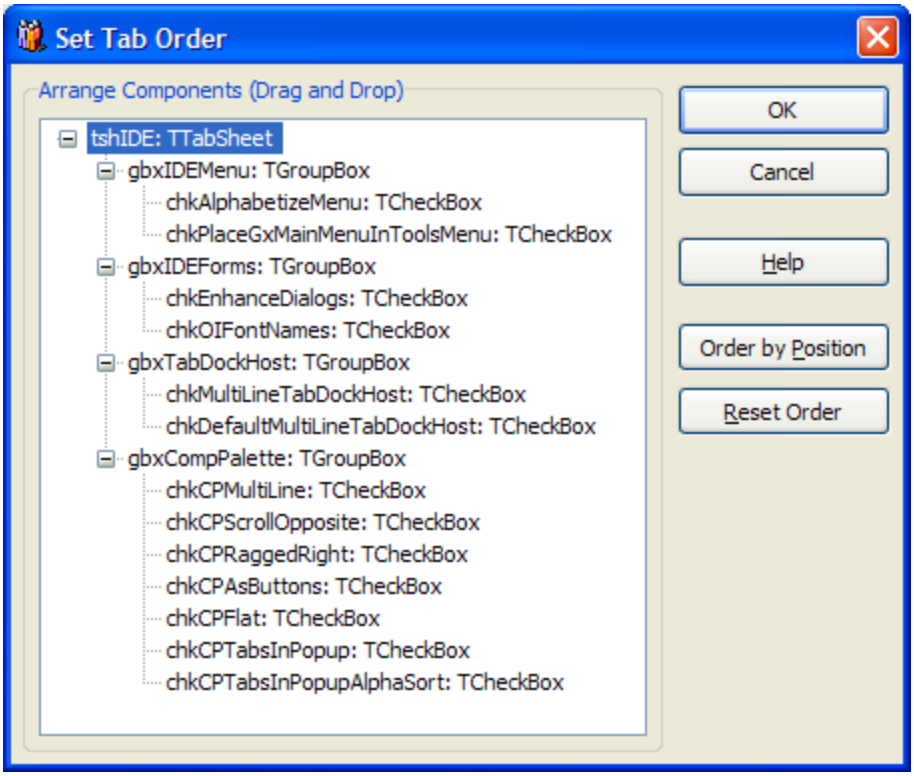
- You can multi-select a TLabel and the associated TWinControl descendant (in either order) and execute this tool. The FocusControl property of the label will be set to the TWinControl descendant.
- You can multi-select any number of TWinControls and the tool will try and guess what TLabel to associate with each control. It looks both above and to the left of the control a certain distance and tries to find the closest label. It won't associate labels with controls where TabStop is False, or with any TButtonControl descendants. Each label and control pair must have the same parent component. Labels that already have an assigned FocusControl are not changed unless you hold down the Shift key while activating the tool. If you want to locate a label for all TWinControls on the form at once, you can either use Edit, Select All or, equivalently, select the form itself to have the tool operate on the whole form.

After the FocusControl assignment process completes, any labels assigned a FocusControl will be selected in the designer, so you can see exactly which labels were modified.

©2026 GExperts Team

2.36 Set Tab Order

This expert enables you to quickly modify the tab order of a form's components. There are two major modes of usage for this expert. If you hold down the Shift key and select multiple components, the expert assumes you have selected components that share a parent and you wish to set the tab order for those components to the selection order (this mode is not supported in Delphi 2005, due to an IDE bug). If a single component is selected that contains child components, the expert attempts to guess a good tab order for those child components based on their relative Top/Left positions and displays the suggested order in the Arrange Components tree. You can rearrange the suggested tab order displayed in the tree using drag and drop. As you select a component in the tree, it will be selected on the underlying form to ease identification.



Notes: This expert does not modify the TabStop property of your controls and will ignore components without a TabOrder property.

2.37 Source Export

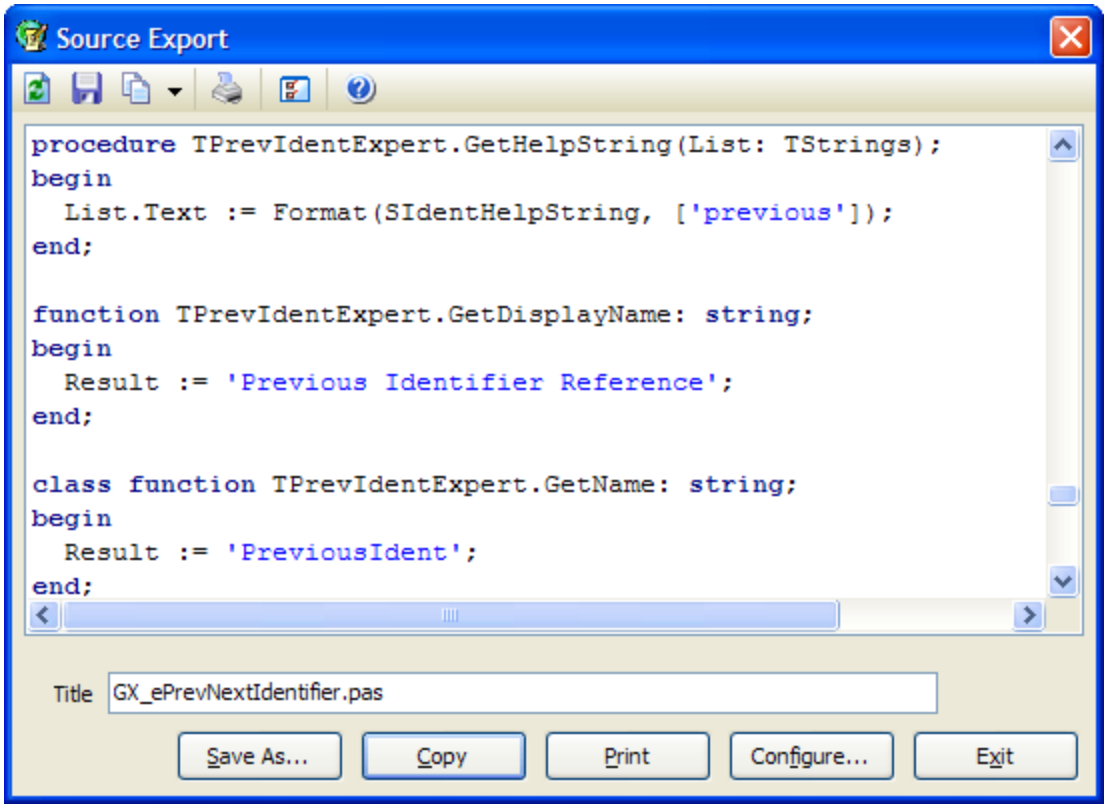
This expert enables you to save source code to an HTML or RTF file or to the clipboard with syntax highlighting preserved. Syntax highlighting of Delphi, C/C++, HTML, C#, XML, and SQL files is provided. You can export the whole file, the portion that is selected in the IDE, or you can edit the shown source to customize the output. The print button allows syntax-highlighted printing as well. You can also set

©2026 GExperts Team

the colors used for the syntax highlighting and the default clipboard format by clicking the Configure button.

The copy button dropdown has three choices:

- Formatted text/RTF/HTML - Preserves the fonts/colors of the source text and places it on the clipboard in all three formats. The application you paste the code into will choose the clipboard format it prefers.
- HTML fragment text - Places the plain text of the HTML source on the clipboard. This text can then be pasted into an existing HTML document's source code.
- RTF fragment text - Places the plain text of the RTF source on the clipboard. This text can then be pasted into an existing RTF document's source code.



Note: GExperts uses CSS to define HTML font styles. Because you are not supposed to duplicate CSS class definitions more than once per file, GExperts does not copy the CSS style definitions into every copied HTML fragment. The best way to preserve the styles when copying multiple fragments into a single HTML document or web site is to export a source code file to an HTML file and copy the CSS definitions from there to into the header of your HTML document or an externally referenced .css file if you want to share the styles across multiple pages. After this process, any further fragments you paste into your document will automatically pick up the shared CSS styles.

©2026 GExperts Team

2.38 Substitution Macros

Some GExperts tools such as the Macro Templates editor expert and the Backup Project expert can use substitution macros in their configuration. These macros allow various %-delimited phrases to be dynamically replaced with a context-sensitive string. The supported macros are:

Project/Unit:

%PROJECTNAME%	The project name
%PROJECTDIR%	The project directory
%PROJECTGROUPNAME%	The project group name
%PROJECTGROUPDIR%	The project group directory
%UNIT%	The current unit name
%UNITDIR%	The unit directory

Date/Time:

%DATE%	The system format date
%DATETIME%	The system format date/time
%YEAR%	The year, in four digits
%MONTH%	The month in two digits
%MONTHSHORTNAME%	The short month name
%MONTHLONGNAME%	The long month name
%DAY%	The day in two digit format
%DAYSHORTNAME%	The short day name
%DAYLONGNAME%	The long day name
%HOUR%	Hour in two-digit format
%MINUTE%	Minute in two-digit format
%SECOND%	Second in two-digit format

System:

%USER%	The currently logged in username
%CLIPBOARD%	The text contents of the clipboard
%SELECTION%	Selected text in the current IDE editor. For the macro templates, the selection text will be deleted from the editor and moved to the position specified by this macro.
%COPYSELECTION%	Selected text in the current IDE editor. For the macro templates, the selection is not deleted from the editor but instead is copied into the macro.
%INPUTVAR%	Queries the user for a string when executed. The format of this macro is: "%INPUTVAR,VarName,<caption>%", where the dialog caption is optional. For example: "%INPUTVAR,VarName,Enter a value%". After the user has input the desired value, the value may be used as a macro later as %VarName%.

Version Information (requires version information in the IDE project options):

%VERPRODUCTVERSION%	The project product version
%VERFILEVERSION%	The project file version
%VERMAJOR%	The project major version
%VERMINOR%	The project minor version
%VERRELEASE%	The project release version
%VERBUILD%	The project build version
%VERPRODUCTNAME%	The project product name
%VERINTERNALNAME%	The project internal name
%VERFILEDESCRIPTION%	The project file description

©2026 GExperts Team

Source Code Parsing (Delphi source only):

%IDENT%	The identifier under the editor cursor (works for C++)
%PROCNAME%	The name of the procedure after/before the cursor
%RESULT%	The result of the procedure after/before the cursor
%ARGUMENTS%	The arguments of the procedure after/before the cursor (all on one line)
%METHODCLASS%	The class and method name in the format "Class.Method" for the procedure after/before the cursor
%CLASS%	The class name after/before the cursor (searches declarations and method definitions)
%INTERFACE%	The interface name after/before the cursor
%BEFORE%	Toggles the state of looking after/before the cursor position for macro substitutions like PROCNAME, RESULT, etc. The default is to look after the cursor.
%BEGINPARAMLIST% %ENDPARAMLIST%	Starts and ends a section of the substitution where the appearance of any of the three below macros reports on only a single argument of the method before/after the cursor. This allows creating macros with an argument per line.
%PARAMNAME%	The name of a single argument (use after %BEGINPARAMLIST%)
%PARAMTYPE%	The type of a single argument (use after %BEGINPARAMLIST%)
%PARAMDEF%	The default value of a single argument (use after %BEGINPARAMLIST%)

For Macro Templates Only (configured inside Macro Templates):

%PROGRAMMERNAME%	Full name of the user
%PROGRAMMERINITIALS%	Initials of the user
%PGEN%	An auto-incremented integer sequence number

Note: To produce a literal percent (%) character in the output, type it as two consecutive percent characters (%%) to prevent an attempt to parse the percent as the start of a macro.

Examples
Examples of many of these macros can be seen by looking at the default macro templates accessed via GExperts, Configuration, Editor Experts, Expand Macro Template, Configuration.

2.39 ToDo Grid

this page ..	see also ..
Overview	General
Basic functionalities	Rules of syntax analysis
Using the string grid	Configuration (Options)
Select rows + Move selection	
Highlight rows	
Sort grid	
Filter grid	
Adjust column width	
Hide columns	

©2026 GExperts Team

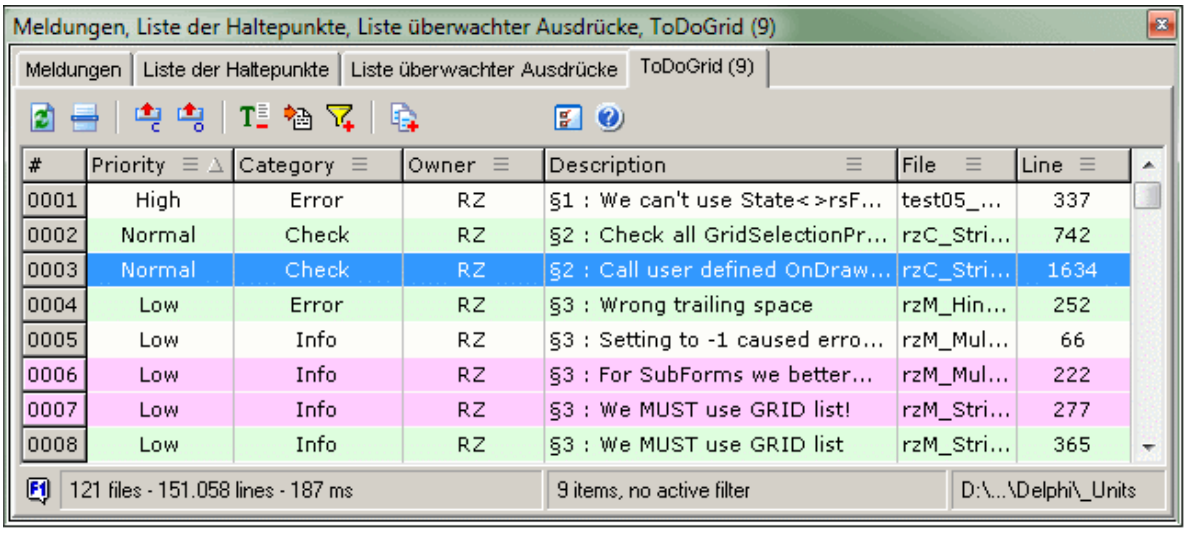
ToDo Grid Expert is not yet available in GExperts

Overview

The "ToDo Grid" expert offers a great overview of all ToDo items that are present in your source code. The expert has a dockable main window; this mode should also be the primary form of use. Disadvantage of the dockable window is, that only a small amount of rows from the data grid is visible. To compensate for this limitation, a second, large and freely movable window can be opened which can displays the data grid with a significantly larger number of rows.

The extended functionality complements the to do function available in the IDE and offers new ways to keep a good overview by sorting, filtering and temporarily highlighting the data. This expert follows the well-known "To Do List" in many respects, but expands it with new features and expanded display options through the use of a string grid. The increased flexibility, especially in configuration, significantly improves adaptation to individual work situations.

Notice
- The application does not monitor the files. If the files have been modified, it is the user's decision to manually initiate a new file scan.
- If you have added new ToDo to the source, but they are not listed in the string grid: Please note that only saved changes are taken into account during the scan!
- Users of older IDE versions that do not save bookmarks can use "ToDo Grid" as an alternative bookmark expert by defining a special "Jump Token".
- Unfortunately, due to system constraints, the experts help cannot be called with F1 in docked windows. If a dockable window is currently undocked, the F1 key can be used.



Basic functionalities

©2026 GExperts Team

Both forms (*DockForm* and *FreeForm*) offer the same functionalities for controlling and using the grid. The basic functionalities are presented in the order of the buttons provided for them:

	Data scan	The button or the keyboard shortcut *F5* you can rescan the files at any time according to the selection of tokens defined in the configuration. A data scan removes both the highlight filter and the column filter. The previous sort, individual column sizes or column visibility are not affected by the scan. Detailed information about the structure of the tokens considered by the scan and the various ways of placing them in the source code can be found on the help page "Rules of syntax analysis". If both forms are visible, a physical scan is performed only on the active form; the data in the inactive form is not automatically updated. If this data should also be updated, a rescan must be initiated in this form as well. By the way: When the *FreeForm* is opened, no current scan is performed; for speed reasons, the data from the DockForm is copied into the string grid of the FreeForm.
	Goto source line	With this button or the keyboard shortcut *Return* you can jump to the source code row which ist selected in the grid. If necessary, the file will be opened in the editor. Depending on the currently valid search path, sometimes an error message may be displayed when opening the file. This is harmless as long as the IDE doesn't remove components from the opened form. Therefore, you should make sure that the file isn't saved. If the line position in the source was changed after the last scan, an attempt will still be made to bring the correct line into the visible area. In this case, the application will ask whether the row number in the grid should be adjusted. If the token position cannot be found at all, the program abort jumping.
	Hide / show "Category" and "Owner"	With the buttons you can hide and show the columns "Category". If a column filter is set, the visibility state of this column cannot be changed. Data scan does not change the visibility of the columns. The status is saved and restored separately for the grids in the DockForm and the FreeForm.
	Show / hide token	After pressing the button the "Description" column will additionally show or hide the "Token" in front of the description. In the configuration, you can specify whether the "Token" is taken into account when sorting the "Description" column.
	Add grid items to message view	Pressing the button copies the current string grid rows to the IDE message view. The copy takes into account the current sorting and any filters. Before the copy is carried out, you will be asked whether you want to delete the

©2026 GExperts Team

		current contents of the IDE message view. The structure of the copied data rows is currently not configurable.
	Show / close undocked grid form	Clicking this button you can show and hide the *FreeForm* as a large and freely movable window. This grid is preferred when a larger number of rows need to be viewed clearly. Size and position of the form are saved and restored when called again. To close the undocked grid form you can press the button in the DockForm or in the FreeForm.
	Options	The button opens the configuration form. The form offers comprehensive options for adapting the expert's settings and behavior to individual requirements. Of particular note, however, are the settings that control the scan because they influence the run time requirements significantly. Detailed information can be found on the help page "Configuration (Options)".
	Help	Pressing this button opens the help (this page).

Using the string grid

Both forms (*DockForm* and *FreeForm*) offer the same functionalities for controlling and using the string grid:

Select rows + Move selection

Only a single row can be selected in the grid; multi-selection or extended-selection is not available. The selection can be moved using the usual keys or the mouse wheel.

The behavior of the mouse wheel is configurable: By default the row selection can be moved directly with the mouse wheel. To move the area marker in the scroll bar, you must hold down the Shift key additionally. In the opposite case, the area marker in the scroll bar is moved directly with the mouse wheel; to move the row selection, hold down the Shift key additionally.

Highlight rows

Double-clicking on a fixed cell in the first column of a row colors the entire row in a configurable color. The highlight status isn't saved beyond runtime and will be overwritten by a scan - nevertheless, the functionality is very useful for temporarily marking any selection of rows.

Notice
- Highlighted rows can be filtered (see "Filter grid").
- When the *FreeForm* is opened, highlight attributes set in the *DockForm* are applied once. Changes made in the *FreeForm* are not subsequently transferred to the *DockForm*.
- While the highlight filter is active, the row status cannot be changed.
- The row highlighting is removed by a data scan.

Sort grid

©2026 GExperts Team

All columns of the grid, with the exception of the row index column, can be sorted individually by clicking in the right area of the fixed cell in the top row. If the column is currently the primary sorted column, a triangle symbol appears there; the orientation of the triangle indicates the sorting direction (column is sorted up) and (column is sorted down).

Notice
- Typically, sorting is performed across multiple columns.
- Only the elements of the sorted column correspond to the selected sorting direction; columns sorted below are always sorted in ascending order.
- The elements in the columns "Priority" and "Category" are not sorted alphabetically, but rather by the relevance of the attributes. For example, "Error" is more important than "Check" and is therefore sorted before "Check." A good overview of the sort order can be found in the combo boxes in the options.
- When sorting the "Description" column, the optionally displayed token is only taken into account if this is set in the configuration.
- After data scan the previous sort will be restored.

Filter grid

Column filter - All columns of the grid, with the exception of the row index column, can be filtered individually by clicking on the symbol in the column header. You can activate multiple column filters at the same time; the filter works additively, i.e. each subordinate filter reduces the result set of the parent filter. If the filter for the column is active, the symbol is displayed. Right-clicking on any red (active) symbol resets the filter; a multi-level filter is always reset step by step in the reverse order of activation.

Highlight filter - If there are highlighted rows, they can be filtered using the button placed above the grid. An active highlight filter is indicated by the button; the filter can be reset by clicking the button.

Notice
- The "Category" or "Owner" column cannot be hidden when filtered by the column filter.
- Column filter and highlight filter cannot be activated at the same time.
- Filters are not transferred to the siblings form grid, so the data can be viewed independently.
- Data scan resets both filters.

Adjust column width

The width of the columns "Description" and "File" can be changed by the user. The action is started by pressing the left mouse button over the dividing line in the top row between the two columns; while holding the mouse button down, the width of both columns can be adjusted within specified limits.

Notice
- The column widths in the DockForm are always preset to a fixed ratio of 3 : 1
- The column widths in the FreeForm are preset in a ratio of 3 : 1 when first called; after that, the last widths set by the user are restored.

©2026 GExperts Team

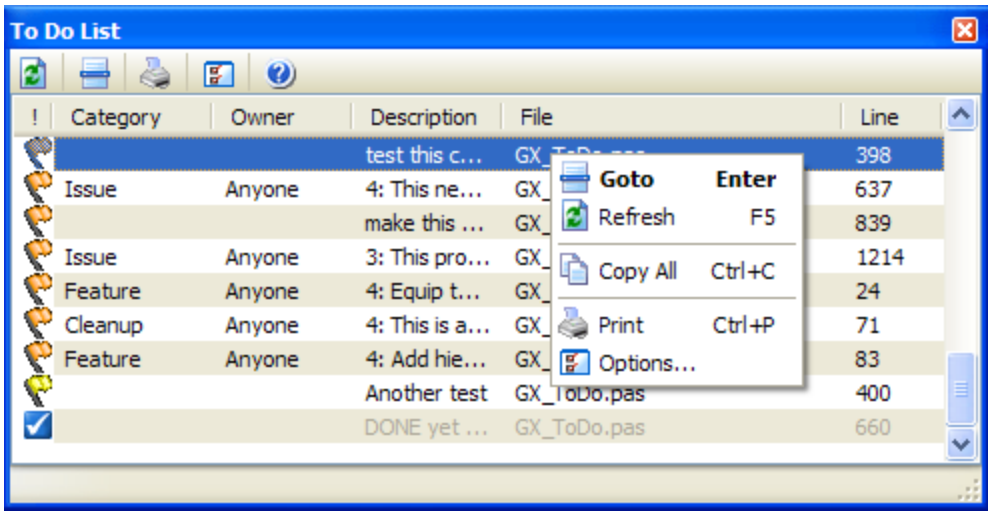
- Only the columns "Description" and "File" are sizable; the width of other columns cannot be changed.

Hide columns

see "Basic functionalities"

2.40 To Do List

The To Do List helps you organize a list of items in your source that need special attention. You can click a column header to sort by any column in the list.



To add new to do items, type comments in your code that start with one of your configured to do tokens, such as:

```
{#ToDo1 Rewrite this code to work under NT}
//#ToDo2 Add support for Oracle here later
```

The to do tokens (such as #ToDo1, #ToDo2, etc.) are configurable in the options dialog and can each have an assigned priority of Critical, High, Normal, Low, Lowest, Done, or Info, with each having its own icon in the list. By adding a token named TODO, this expert can read the standard to do entries generated by the IDE's built-in to do list. It can also extract the assigned owner and category of the to do item using the IDE's standard -o/-c switches as shown here:

```
// TODO 3 -cBug -oJohn: Find a way to handle invalid filenames
```

Based on settings, this tool can scan open project files, all files in the current project or project group, or a complete directory for to do items. It may be helpful to create an IDE Code Template to quickly create new to do items while you are coding.

©2026 GExperts Team

Note: You can not configure the To Do list to use tokens that begin with the dollar sign ($) because this conflicts with Delphi compiler directives.

2.41 Uses Clause Manager

this page ..	see also ..
Overview	General
Used units	Configuration (Options)
Available actions	
Context menu and ..	
Available units	
Tabs "Search path"..	
Tab "Favorite"	
Tab "Identifiers"	
Statusbar	

Overview

The "Uses Clause Manager" allows you to manage both uses clauses of the current unit. The used units are shown in the left part of the expert's dialog. There you can *delete* units from these lists or *move* entries from one of the lists to the other. However, the main task of this expert will usually be *add* units required for new code added in your source.

Although it is possible to *drop* files from Windows Explorer onto both uses lists, the units to be added are usually selected from the lists in the right part of the dialog and copied from there to one of the uses lists.

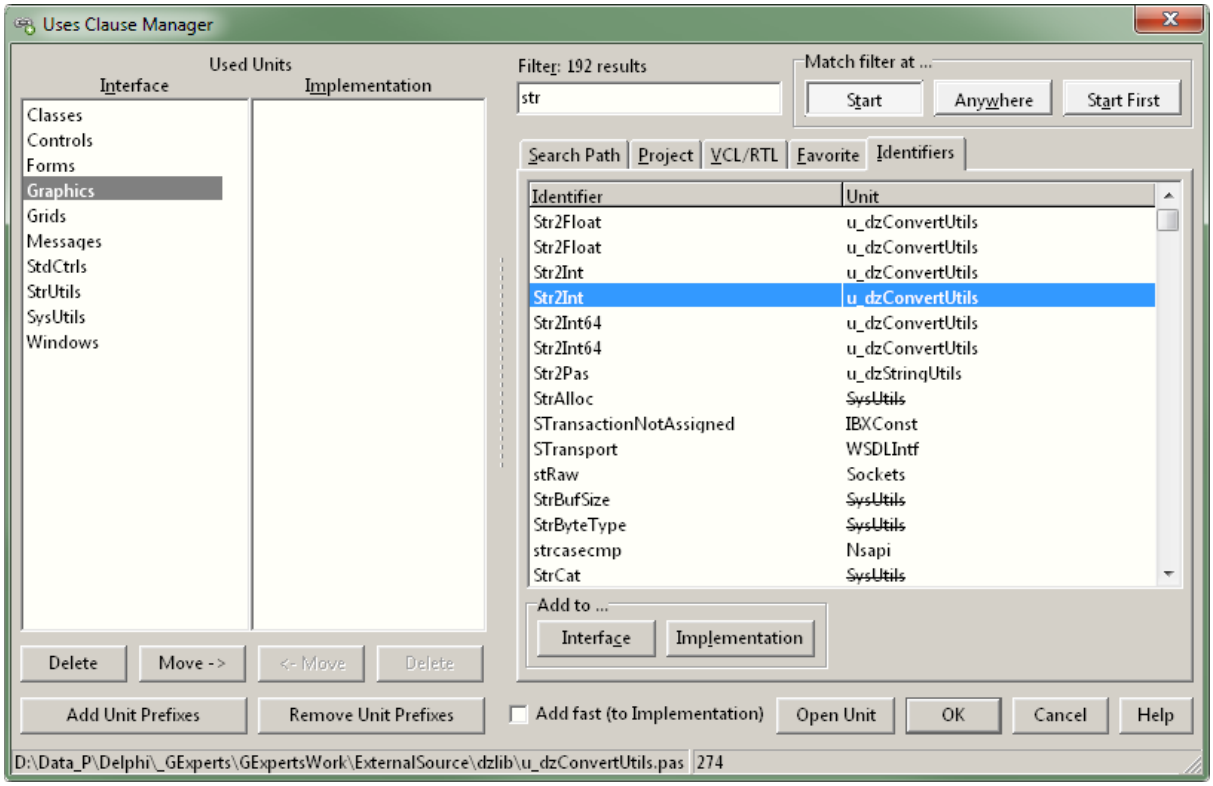
The check mark *Add fast (to interface)* or *Add fast (to implementation)* changes the behavior of the *OK* button. If checked, it adds the selected unit to the uses clause given in the caption and closes the dialog. Which uses clause is given in the caption depends on where the cursor in the code editor is located. If it is in the interface section, the caption says "to interface", if it is in the implementation section, the caption says "to implementation".

The *Open Unit* button opens the selected unit in the code editor and closes the dialog. Note that there is a shortcut *Ctrl+O* for it, or you can double click an entry. In this case the unit lists in the source will not be changed.

The *OK* button or *Enter* closes the dialog and commits your changes, the *Cancel* button or *Escape* closes the dialog and discards any changes.

Notice
- The Uses Clause Manager does not sort the units in the source, nor does it detect or remove unused units.
- The splitter only works if the window is not displayed in the minimum width.

©2026 GExperts Team



Used units (left part of dialog)

Available Actions

On the left the expert's dialog shows two lists with the units in the Interface and Implementation uses clauses respectively. The four buttons below these lists *Delete* from the corresponding list or *Move* entries from one of the lists to the other.

The units to be added are usually selected from the lists placed on the right part of the dialog and *added* from there to the uses lists; for further information see the next paragraph. Alternatively, you can drag files from the Windows Explorer to any of these two lists to *add* a unit.

Alternatively to using the Move button it is also possible to move entries with drag and drop between the two uses lists.

Further below the lists there are two buttons to *Add Unit Pre*fixes or *Remove Unit Prefixes* (name space qualifiers), e.g. they convert "Registry" to "System.Win.Registry" and vice versa. This functionality is helpful for people who develop software for multiple Delphi versions including those ancient versions that did not support name spaces.

Notice
- Adding these prefixes is a bit tricky because the expert needs to go through all the configured name spaces in the project and use the unit search path to find a unit matching these name spaces. If it does not find them, it will not change the unit name.

©2026 GExperts Team

- Although the two lists on the left are initially sorted, added or moved items are placed at the end of the lists.
- Dragging also works from the lists on the left to the lists on right for removing entries from Uses lists. But note that only the list on the Favorites tab accepts entries! Dragging an entry to one of the other three lists will still remove the unit from the source list. The drag cursor is not set to *No Drop* in this case to allow removing units in this manner.

Context menu and keyboard shortcuts

All Buttons also have corresponding context menu items and keyboard shortcuts:



Delete - Always deletes from the currently focused list. This is also the default action executed with a double click.

Move - Moves selected item from the current list to the other.

Open Unit - Opens the currently selected unit in the editor and closes the dialog without making any changes to the source code.

Add to Favorites - Adds the currently selected unit(s) to the favorite units list and switches to the favorites tab.

Unalias - Replaces units like WinTypes and WinProcs, which have been declared as aliases to Windows for ages, with their actual unit. The same goes for dbiTypes, dbiProcs and dbiErrs which will be replaced with BDE. This will also work for other unit aliases configured in the project.

Available units (right part of dialog)

On the right of the expert's dialog there are five tabs with lists from which the desired unit can be selected and added to the uses lists in the left part of the dialog. The content of the Favoriteslist is configured by the user, the other lists are filled automatically. If an entry is crossed out, this means that it is already in one of the Used units lists.

Tabs "Search path" + "Project" + "RTL/VCL"

The *Search Path* list contains all units in the current unit search path. The *Project* list shows all units of the current project. The *VCL/RTL* list holds all units of VCL/RTL. The caption is a bit of a misnomer, because the list also contains units from the Firemonkey framework.

With the *filter* above the lists you can narrow down the listed entries, e.g. entering "vcl" will only show items that contain that string in the unit name. The list is updated dynamically, so changing the filter will immediately show different entries. While the focus in on the filter edit, you can still move the selected line in the list using the up/down and page up/down keys. By holding the shift key you can also select multiple entries. The full path of the currently selected unit is shown on the status bar.

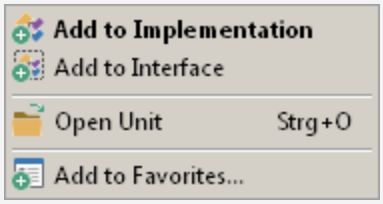
©2026 GExperts Team

The buttons in the *Add to..* group box add the selected unit(s) to the uses clause in the interface and implementation section respectively. Note that both buttons have two keyboard shortcuts: *Alt+C/Alt+L* adds the selected unit(s) and keeps the expert open, *Ctrl+C/Ctrl+L* adds the selected unit(s) and automatically closes the dialog.

It is also possible to select one or multiple entries with the mouse and drag the selected unit(s) to the Interface or Implementation lists on the left.

With the button *Add all to favorites* you can copy all list elements to the *Favorites* list; the current selection is not taken into account. *Favorites* should represent a manageable and exclusive selection, so all elements of a list should only be included in exceptional cases - even if the number of visible items can be reduced by the filter.

The lists on all *tabs* have corresponding context menu items and keyboard shortcuts:



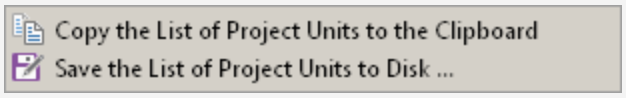
Add to Implementation - Adds the currently selected unit(s) to the implementation list.

Add to Interface - Adds the currently selected unit(s) to the interface list.

Open Unit - Opens the currently selected unit in the editor and closes the dialog.

Add to Favorites - Adds the currently selected unit(s) to the favorite units list and opens the favorites tab (see above notes).

On the *Project* tab, you will also find the *Copy/Save List* button, which allows you to copy the entire list to the clipboard for later use. Directly next to it is an arrow button that opens the following drop down menu:



Copy .. to clipboard - Copies the complete list to the clipboard for later use.

Save .. to disk - Opens the save file dialog to save the complete list to any file.

Tab "Favorite"

The list placed at the *Favorite* tab contains the users favorite units. It is recommended to limit list items to a manageable number of elements - even if the number of visible items can be reduced by the filter. Initially this list is empty, entries can be added via a file select dialog, by copying elements from other lists or dragging from the Used lists or the Windows Explorer.

With the *filter* above the list you can narrow down the listed entries, e.g. entering "vcl" will only show items that contain that string in the unit name. The list is updated dynamically, so changing the filter will immediately show different entries. While the focus in on the filter edit, you can still move the selected line in the list using the up/down and page up/down keys. By holding the shift key you can also select multiple entries. The full path of currently selected unit is shown on the status bar.

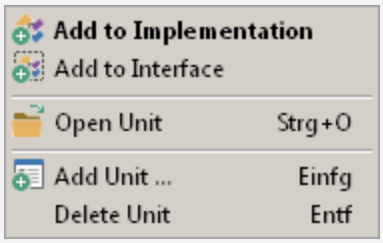
The buttons in the *Add to..* group box add the selected unit(s) to the uses clause in the interface and implementation section respectively. Note that both buttons have two keyboard shortcuts: *Alt+C/Alt+L* adds the selected unit(s) and keeps the expert open, *Ctrl+C/Ctrl+L* adds the selected unit(s) and automatically closes the expert.

©2026 GExperts Team

It is also possible to select one or multiple entries with the mouse and drag the selected unit(s) to the Interface or Implementation lists on the left.

With the button *Add Unit..* you can add any unit via a file select dialog to the favorites list, the button *Delete Unit* deletes the currently selected units(s) from the favorites list. The list will only be re-sorted alphabetically the next time the dialog is opened.

The *Favorites* tab has corresponding context menu items and keyboard shortcuts:



Add to Implementation - Adds the currently selected unit(s) to the implementation list.

Add to Interface - Adds the currently selected unit(s) to the interface list.

Open Unit - Opens the currently selected unit in the editor and closes the dialog.

Add Unit - Opens the file select dialog and let you add any unit file to the favorites list.

Delete Unit - Removes the currently selected unit(s) from the favorites list.

Tab "Identifiers"

The last list, placed on the *Identifiers* tab, is probably the most useful one. It lists all *identifiers* and *units* from the units search path, the list items are sorted alphabetically by identifier in the first column.

With the *filter* above the list you can narrow down the listed entries, e.g. if you type "TWinControl" the list will be filtered down to less than 10 entries of several thousand entries. The list is updated dynamically, so changing the filter will immediately show different entries. Even while the focus in on the filter edit, you can still move the selected line in the list using the up/down and page up/down keys. The full path of currently selected unit is shown on the status bar.

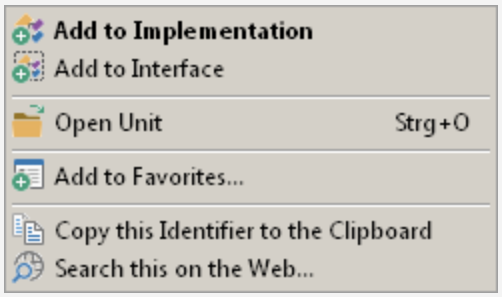
Below the list there are three buttons that control the way the filter works, these buttons can get the input focus, so they can be pressed using the keyboard:

- *Start* - Show items that start with the search string.
- *Anywhere* - Show items that contain the search string.
- *Start First* - Show items that start with the search string first, followed by others that contain the search string.

The buttons in the *Add to..* group box add the selected unit(s) to the uses clause in the interface and implementation section respectively. Note that both buttons have two keyboard shortcuts: *Alt+C/Alt+L* adds the selected unit(s) and keeps the expert open, *Ctrl+C/Ctrl+L* adds the selected unit(s) and automatically closes the expert.

©2026 GExperts Team

The *Identifiers* tab has corresponding context menu items and keyboard shortcuts:



Add to Implementation - Adds the currently selected unit to the implementation list.

Add to Interface - Adds the currently selected unit to the interface list.

Open Unit - Opens the currently selected unit in the editor and closes the dialog.

Add to Favorites - Adds the currently selected unit to the favorite units list and opens the favorites tab.

Copy this Identifier... - Copies the selected identifier to the clipboard for later use and keeps the expert open.

Search this on the Web... - Opens the default browser and starts an internet search for currently selected identifier.

Notice

- The Identifier tab can be used as a *replacement* for the Find Unit refactoring of the Delphi IDE (which for me doesn't work most of the time and if it does is very slow). And of course the Uses Clause Manager also works for older Delphi versions which didn't have this refactoring feature.
- If you filter for e.g. TWinControl the expert shows several matching entries, e.g. the two top ones in the unit Controls. While this looks like a bug at first, but in fact - *it's not a bug, it's a feature*. There are actually *two* declarations of TWinControl in the unit Controls (a forward declaration and the actual class declaration). The expert lists them both, so when the user opens the unit with Ctrl+O the cursor will be placed in the line with the selected entry.
- The Identifiers list is updated in a *background thread*. For this to work it parses all source files in the various search paths of the project. And because this takes a while, it caches the results and only updates this cache if a new unit is found or a unit has been changed. The directory for this cache is configured in the GExperts Configuration dialog on the General tab.

Statusbar

The status bar always shows the full path of the selected unit in the 1st panel; the second panel shows the files line number of the currently selected identifiers declaration (for "Identifiers" tab only). Right clicking on the status bar shows a context menu with the following menu items:



Copy this text to the clipboard - Copies the full file name to the clipboard.

Copy this file to the clipboard - Copies the file to the clipboard so it can be used as if copied from the Explorer.

Open location of this file - Opens the folder in which the file is located in the Explorer and selects the file.

©2026 GExperts Team

©2026 GExperts Team

- A -

About 5
Archive Project 29
ASCII Chart 29
Auto-Correct 35

- B -

Backup 29
Backup Project 29

- C -

C# Compatibility 10
C++ Compatibility 10
change log 17
Class Browser 31
Clean Directories 32
Clipboard 33, 38
Clipboard History 33
Code Editor Enhancements 25
Code Librarian 34
Code Navigation 72
Code Proofreader 35
Code Snippets 34
Code Templates 85
Component Grid 36
Component Naming 76
Components to Code 37
Configuration 18
Copy Component Names 38
Correction 35

- D -

Debug Window 12
Default Properties for Components 76
Delete Temporary Files 32
Disconnect Datasets 81

- E -

Editor Enhancements 25
Editor Experts 20
Editor Toolbar 25
Expand Macro Template 66
Expert List 18
Expert Manager 43
Experts 43

- F -

Favorite Files 44
File Locations 22
Find Component Reference 45
FocusControl 82

- G -

General Configuration 22
GExperts Configuration 18
Grep Regular Expressions 55
Grep Results 60
Grep Search 62
Grep Search and Replace 62

- H -

HelpContext Property 36
Hide/Show Non-Visual Components 64
Hint Property 36
HTML 83
https://en.delphipraxis.net/forum/31-gexperts 5

- I -

IDE Enhancements 23
IDE Menu Shortcuts 64
IDE Shortcuts 64
Imports/Exports 70
Introduction 5

- L -

Library Dependencies 70
License Agreement 10

- M -

Macro Library 65

©2026 GExperts Team

Macro Templates 66
Macros 85
Main Menu 64
Message Dialog 68
MessageBox 68
MessageDlg 68
Methods 72
MRU 69

- N -

Name property 76
Non-Visual Components 64

- O -

Open File 69

- P -

PE Information 70
Perfect Layout 72
Procedure Header 66
Procedure List 72
Procedures 72
Project Dependencies 73
Project Option Sets 74
Property Value Constants 75

- R -

readme 6
Regular Expressions 55
Rename Components 76
Replace Components 77
Required Packages 70
RTF 83
Runtime Component Creation 37

- S -

Saved Desktop 72
Search 62
Search and Replace of Component Types 77
Search Results 60
SendDebug 12
Set Component Properties 81
Set FocusControl 82
Set Property Values 75
Set Tab Order 83
Source Export 83
Subexpressions 58
Substitution Macros 85

- T -

Tab Order 83
Tag Property 36
To Do List 91

- U -

Unit Header 66
Units 73

- W -

Windows 72
Wizards 43

- Z -

zip 29

©2026 GExperts Team